

Simple Public Key Anamorphic Encryption and Signature using Multi-Message Extensions

Shalini Banerjee*, Tapas Pal*, Andy Rupp†, Daniel Slamanig†*

*KASTEL SRL, Karlsruhe Institute of Technology

{shalini.banerjee, tapas.pal}@kit.edu

†University of Luxembourg

andy.rupp@uni.lu

†*Universität der Bundeswehr München

daniel.slamanig@unibw.de

Abstract

Anamorphic encryption (AE) considers secure communication in the presence of a powerful surveillant (typically called a “dictator”) who only allows certain cryptographic primitives and knows all the secret keys in a system. The basic idea is that there is a second (anamorphic) mode of encryption that allows for transmitting an anamorphic message using a double key to a receiver who can decrypt this message using the same double key. From the point of view of the dictator, the encryption keys as well as the ciphertexts in the regular and anamorphic modes are indistinguishable. The most recent works in this field consider public key anamorphic encryption (PKAE), i.e., the sender of an anamorphic message requires an encryption double key (or no key at all), and the receiver requires an associated decryption double key. Known constructions, however, either work only for schemes that are mostly of theoretical interest or come with conceptual limitations, assuming additional unnecessary properties (e.g., randomness recoverability and CCA security).

In this paper, we ask whether we can design PKAE schemes without such limitations and be closer to practically used PKE schemes. In fact, such schemes are more likely to be allowed by a cognizant dictator. Moreover, we initiate the study of identity-based anamorphic encryption (IBAE), as the IBE setting seems to be a natural choice for a dictator. For both PKAE and IBAE, we show how well-known IND-CPA and IND-CCA secure primitives can be extended by an anamorphic encryption channel. In contrast to previous work, we additionally consider CCA (rather than just CPA) security notions for the anamorphic channel and also build upon CPA (rather than only CCA) secure PKE.

Finally, we ask whether it is possible to port the recent concept of anamorphic signatures, which considers constructing symmetric anamorphic channels in case only signature schemes are allowed by the dictator, to the asymmetric setting, which we denote by public-key anamorphic signatures (PKAS). Moreover, we consider IND-CCA security for the anamorphic channel of our PKAS.

Contents

1	Introduction	3
1.1	Our Contributions	5
2	Technical Overview	7
2.1	μ -Message Public Key Anamorphic Encryption Extensions	7
2.2	μ -Message Public Key Anamorphic Signatures Extensions	12
3	Preliminaries	13
3.1	Notations	13
3.2	Computational Assumptions	14
3.3	Secret Key Encryption	15
3.4	Public Key Encryption	16
3.5	Identity-Based Encryption	17
3.6	Signature Scheme	19
4	Public Key Anamorphic Encryption	20
5	μ-Message Public Key Anamorphic Encryption Extension	22
5.1	Extension of the ElGamal Encryption Scheme	23
5.2	Extension of the Dual-Regev Encryption Scheme	30
5.3	Extension of FO-Based PKE	38
6	Identity-Based Anamorphic Encryption Extension	41
6.1	Extension of the Boneh-Franklin BasicIdent Scheme	46
6.2	Extension of FO-Based IBE	54
7	μ-Message Public Key Anamorphic Signature Extension	55
7.1	Waters Signatures with Linear Encryption	58
7.2	BBS with ElGamal and Pseudo-Random Encodings	59
7.3	Anamorphism-Friendly Signatures with Compliant Encryption	60
A	Robustness of Public-Key Anamorphic Encryptions	72
B	Informal: Robustness of Public-Key Anamorphic Signatures	73

1 Introduction

Anamorphic encryption, introduced in [PPY22] and subsequently explored in [KPP⁺23a, BGH⁺24, CGM24a, CGM24b, PPY24, CGM25], is a very recent concept that allows individuals to confidentially exchange messages in a very restricted setting where a powerful surveillant (typically called a “dictator”) has full access to the secret keys in the system. Such a capability can foster a strong foundation for enabling privacy in the face of a powerful authoritarian government where every citizen is constantly surveilled. The basic idea behind a PKE scheme to be anamorphic, is that there is a second mode of encryption (the anamorphic mode) that allows encrypting an anamorphic message that can be recovered by the legitimate holder of the secret key. But from the perspective of the dictator, these ciphertexts look indistinguishable from regular ciphertexts, and so the dictator is oblivious of these anamorphic messages. Moreover, keys used for the anamorphic mode must also be indistinguishable from the ones used for the regular mode. The anamorphic functionality is achieved by having a second key (the so-called double key) that can be used by the sender and the receiver (and has to be exchanged covertly in advance). Techniques on how to secretly and efficiently exchange the double key have been recently discussed by Wan, Hang and Liu in [WHL25]. The general idea behind anamorphic encryption is to incorporate a covert channel using a symmetric or asymmetric encryption scheme on top of an existing PKE scheme. While the asymmetric setting will be the focus of this paper, we want to mention that for the symmetric setting, one usually relies either on some randomness recoverability property of the PKE [BGH⁺24, KPP⁺23b], on hybrid encryption, or an IBE-to-CCA transform [CGM24a]. Interestingly, there are well-known CCA-secure public-key encryption schemes (or generic approaches to construct such schemes) such as the Naor-Yung double encryption paradigm [NY90], (RSA)-OAEP [BR95] or Cramer-Shoup [CS98] that are inherently anamorphic.

The most recent works in this fast-pacing field focus on schemes with asymmetric anamorphic channels [CGM24a, PPY24]. Additionally, they investigate the limitations of constructing anamorphic schemes [CGM24b, CGM25]. Arguably, asymmetric anamorphic channels, so-called public key anamorphic encryption (PKAE), seem to be superior to symmetric ones. In a PKAE, the keys for the anamorphic mode are asymmetric, i.e., there is an encryption double key dk and a decryption double key tk . This is beneficial as in the case of multiple senders, no complex key management is required in order to prevent senders from reading anamorphic messages sent to the receiver by other senders.

Public-key anamorphism. Persiano, Phan and Yung in [PPY24] propose a variant of the PKAE notion where $dk = \epsilon$, which interestingly eliminates the need of communicating the encryption double key dk privately to the receiver. They complement this by providing a proof of concept of the notion using the CCA-secure Koppula-Waters (KW) scheme [KW19]. Additionally, they initiate a theoretical investigation on the link between CPA-to-CCA compilers and public key anamorphism to extract a set of generic “reduction-based” properties required in the underlying PKE, satisfied by the KW scheme. Worth mentioning is also that both the PKAE constructions based on KW proposed in [PPY24] and Naor-Yung (NY) proposed in [CGM24a] rely on a common feature of the underlying PKE scheme: the setup procedure generates multiple key pairs $(sk_i, pk_i)_{i \in [n]}$, while retaining only a subset of secret keys sk_j , which makes them a natural choice for constructing PKAE by embedding anamorphic message $amsg$ within ciphertext ct_j and allowing the receiver to retain the corresponding sk_j (which ideally should have been forgotten). Nevertheless, we observe that most of the well-known PKE schemes and, in particular, practically used ones do not fulfill

this property.

In another recent work, Catalano, Giunta and Migliaro [CGM24a] design PKAE schemes, where $\text{dk} \neq \epsilon$, which they call fully-asymmetric AE schemes. Such schemes are asymmetric but require sharing the encryption key to potential senders. In particular, the authors focus on Homomorphic Anamorphic Encryption (HAE) and demonstrate that NY instantiated with any IND-CPA secure fully-homomorphic encryption scheme and fully-homomorphic NIZK [ADKL19], the Gentry-Sahai-Waters scheme [GSW13] and Cramer-Shoup Lite [CS98] are fully-asymmetric. Moreover, in a follow-up work, Catalano, Giunta and Migliaro [CGM25] show that such a type of scheme can be obtained from any IND-CPA secure PKE with sufficiently high min-entropy ciphertexts using $i\mathcal{O}$. Also, here it is worth noting that this does not cover PKE schemes that (or variants thereof) are used in practice. Importantly, all these constructions in [CGM24a] are anamorphic triplets rather than anamorphic extensions (introduced in [BGH⁺24]), i.e., the double keys have to be generated at the time the normal keys are generated. However, it is desirable that schemes support *on-the-fly* anamorphic key generation, i.e., double keys can be generated based on existing keys of the normal PKE.

Moreover, interestingly, all these existing anamorphic schemes build on top of *CCA-secure* PKE schemes, but with respect to the anamorphic message (i.e., the indistinguishability of anamorphic mode from normal mode), they just achieve protection in the *CPA-sense*.

This makes us think whether these limiting restrictions, i.e., reduction-based properties, being limited to anamorphic triplets rather than extensions as well as relying on CCA-secure PKEs, are inherently linked to the design of asymmetric anamorphic channels. In particular, we ask

Can we design simple public key anamorphic encryption schemes without such limitations and ideally based on practically used PKE schemes?

Indeed, the ability to focus on simpler, practically deployed PKE schemes (or their close variants) increases the likelihood of acceptance by a perceptive or strategically aware dictator. We believe this is a promising direction to explore in the sense that a positive result would give us hope towards establishing PKAE as a practical cryptographic primitive, while a negative result would imply relying on the existence of specialized PKE schemes, that might be outlawed by the dictator.

Towards an affirmative answer, we first give constructions for PKAE (with $\text{dk} \neq \epsilon$) based on the ElGamal and Dual-Regev PKE schemes where we embed an anamorphic message in $\mu > 1$ ciphertexts of the respective PKE (which we call multi-message extension). Note that ElGamal is used as a KEM in the popular Elliptic Curve Integrated Encryption Scheme (ECIES) [ABR99] and Dual-Regev is the basic scheme underlying many modern lattice-based schemes such as Kyber [SAB⁺22] (standardized as ML-KEM¹). Second, many practically used schemes (such as Kyber and thus ML-KEM) apply the Fujisaki-Okamoto transform [FO99, FO13] to a weakly (i.e., OW-CPA or IND-CPA) secure scheme to obtain an IND-CCA secure one. Here, we show that when the ciphertext of the underlying PKE is pseudo-random in the ciphertext space (so called IND $\mathcal{S}$ -secure [KLR19]), then together with a suitable encoding, anamorphic messages can be embedded into the FO-transformed PKE such that the anamorphic encryption channel achieves replayable CCA (RCCA) [CKN03] security. To the best of our knowledge, this is the first work that considers security stronger than IND-CPA for the anamorphic message (though many works in the past relied on CCA secure schemes for the normal message).

Identity-based anamorphic encryption. When considering a setting where a dictator wants

¹<https://doi.org/10.6028/NIST.FIPS.203>

to control the entire communication, it seems natural to consider identity-based encryption [BF01]. In this setting, there is an entity which generates the private keys for all the users (identities) from a compact msk , and thus there is an inherent backdoor (this msk) already built into the system. Thus, it seems likely that a dictator would allow such encryption mechanisms, which in addition would save the dictator from the effort of collecting all the private keys. Consequently, it seems natural to ask

Can we port the concept of anamorphic encryption to the identity-based encryption setting?

We demonstrate how to realize anamorphic channels in such a world, define identity-based anamorphic encryption (IBAE) and propose a construction based on the Boneh-Franklin IBE [BF01]. Moreover, we extend our results on the FO-transform from the PKAE to the IBAE setting. Interestingly, this allows us to obtain IBAE with strong security for the anamorphic channel for *standardized* IBE [ETS22].

Anamorphic signatures. Besides the dedicated study on encryption, recently Kutyłowski et al. [KPP⁺23a] investigated whether a confidential message transmission via anamorphic channels can still work in case the dictator forbids the use of encryption, and only authentication primitives such as signatures are available. They considered *symmetric* anamorphic channels and provided an affirmative answer to that question. After considering simpler constructions for PKAE, it seems natural to ask

Are there public-key anamorphic signatures (PKAS), and can we design such schemes as a simple combination of existing signature and PKE schemes?

Towards this goal, we introduce a general framework for PKAS and showcase constructions based on two known signature schemes, one being *standardized*.

1.1 Our Contributions

In the following, we briefly summarize our contributions:

- We introduce the concept of μ -message anamorphic extensions (PKAEE) to construct public-key anamorphic encryption schemes from simple CPA-secure PKE schemes that (or variants thereof) are used in practice. In particular, both the encryption of regular messages and anamorphic messages as well as their decryption should essentially resemble that of a *single* PKE scheme. This concept allows us to embed an anamorphic message in $\mu > 1$ ciphertexts of a PKE scheme. We demonstrate such extensions for the well-known ElGamal and the Dual-Regev schemes with $\mu = 2$, achieving both the *fully asymmetric* IND-CPA (or simply IND-CPA) and *strong* IND-CPA (sIND-CPA) security properties. In the fully asymmetric IND-CPA security notion, the adversary can select its challenge query corresponding to both normal and covert messages. The strong IND-CPA security refers to an adversary that additionally possesses the secret key of the PKE, hence, it can challenge only the covert messages.

We note that our Dual-Regev construction can be considered as the first fully asymmetric (s)IND-CPA PKAE based on lattices, given that existing constructions are either based on groups or SKE. We then turn towards the Fujisaki-Okamoto (FO) transform, often found in practical IND-CCA secure PKE schemes (such as Kyber or ML-KEM). Interestingly, although many previous works used IND-CCA secure PKEs to create anamorphic channels, these works

did restrict to IND-CPA security for the anamorphic message. We also consider security beyond IND-CPA and show that with our FO-approach, we can achieve strong IND-RCCA (sIND-RCCA) security where strong refers to the model that provides the secret key of the PKE scheme to the adversary.

- We initiate the study of identity-based anamorphic encryption (IBAE). We show that the Boneh-Franklin IBE (BF-IBE) admits a 2-message IBAE, satisfying fully asymmetric IND-CPA and strong IND-CPA security properties. Moreover, we show that the FO-approach for constructing an anamorphic channel can be equivalently applied to the IBE setting. This covers CCA secure variants of the BF and the Sakai-Kasahara IBEs, which are standardized [ETS22].
- We introduce the concept of public-key anamorphic signature extensions (PKASE). Our goal here is to combine well-known signature schemes and PKE schemes in a way that the embedding is simple. We introduce a generic framework that covers signature schemes which are canonical candidates for anamorphic extensions and, in particular, have at least one uniformly random component. The random components of a few signatures suffice to embed a pseudo-random ciphertext. We showcase our framework for Waters signatures and linear encryption as well as Boneh-Boyen-Sacham (BBS) signatures, being in the process of *standardization*, and FO-ElGamal encryption and achieve security beyond IND-CPA for the latter.

We provide an overview of our anamorphic encryption and signature schemes in Table 1.

PKAEE					
Our Work	PKE	$\mu = \#ct$	$\widehat{\mathcal{M}}$		Security
Sec. 5.1	ElGamal	2	restricted		IND-CPA+sIND-CPA
Sec. 5.2	DualRegev	2	restricted		IND-CPA+sIND-CPA
Sec. 5.3	FO-HashedElGamal	4	unrestricted		sIND-RCCA
IBAE					
Our Work	IBE	$\mu = \#ct$	$\widehat{\mathcal{M}}$		Security
Sec. 6.1	BF	2	restricted		IND-CPA+sIND-CPA
Sec. 6.2	FO-BF	4	unrestricted		sIND-RCCA
PKASE					
Our Work	PKE	Sig	$\mu = \#\sigma$	$\widehat{\mathcal{M}}$	Security
Sec. 7.1	LinEnc	Waters	3	unrestricted	IND-CPA
Sec. 7.2	ElGamal	BBS	4	unrestricted	IND-CPA
Sec. 7.2	FO-ElGamal	BBS	5	unrestricted	sIND-RCCA + IND-CCA

Table 1: Our anamorphic encryption and signature schemes. Here, we denote by $\#ct$ and $\#\sigma$, the total number of (regular) ciphertexts and signatures of the underlying PKE, IBE and Sig schemes needed to construct our PKAEE, IBAEE and PKASE respectively; by $\widehat{\mathcal{M}}$, the anamorphic message space where “restricted” means polynomial-sized and “unrestricted” means exponential-sized; by RCCA replayable CCA; by FO Fujisaki-Okamoto; by BF Boneh-Franklin; by LinEnc linear encryption [BBS04]; “Security” means the security of the anamorphic message.

Concurrent work. Concurrently and independently, Deo and Libert [DL25] constructed a PKAE scheme from the trapdoor-based Dual-Regev, where no double key is required to encrypt the covert message. Our technique is different from theirs as we obtain the lattice-based PKAE from μ -

message extension without using any additional lattice algorithm, whereas their work first utilizes a trapdoor sampling algorithm to obtain a version of the Dual-Regev scheme that recovers the encryption randomness using the trapdoor and then follows an encoding-decoding mechanism to embed the covert message into the randomness.

2 Technical Overview

We provide a technical overview of our contributions and how our approaches work, with details deferred to the respective sections.

2.1 μ -Message Public Key Anamorphic Encryption Extensions

We have already discussed that constructing PKAE is known to work when making an additional assumption such as when relying on PKE where the key generation algorithm samples various key pairs of the underlying CPA-secure PKE schemes and a *subset* of the secret keys is sufficient for decryption, which gives rise to a public key anamorphic channel. Nevertheless, only theoretically interesting schemes are known to provide such a feature. Moreover, for the schemes in [CGM24a], despite not being very close to schemes used in practice, the limitation to anamorphic triplets rather than anamorphic extensions seems inherent.

Alternatively, we could take inspiration from recent works [KPP⁺23a, KPP⁺23b] constructing secret key anamorphic encryption schemes (i.e., $\text{dk} = \text{tk}$) using the randomness-recovering properties of PKE schemes. We recall that for such a scheme the secret key can recover the random coins sampled during encryption. Let r be the randomness used to encrypt msg by $\text{PKE.Enc}(\text{pk}, \text{msg}; r) = \text{ct}$ then there is a randomness recovery algorithm that extracts r along with msg using sk . To create an anamorphic channel using this property, one needs a symmetric key encryption scheme SKE to encrypt an anamorphic message amsg and use $\text{SKE.Enc}(\tilde{\text{sk}}, \text{amsg})$ as the random coins r while computing ct . The secret key sk plays the role of the double key. If the SKE scheme produces pseudorandom ciphertexts then the dictator remains oblivious about the presence of such an anamorphic message inside ct having access to $(\text{pk}, \text{sk}, \text{ct})$. A naïve way to make this work in the case of PKAE is to replace the underlying SKE having pseudorandom ciphertexts with a PKE having the same property.

However, our approach towards PKAE is not to use any specific property of the PKE but to devise an approach that is capable of covering various well-known PKE schemes that (or variations thereof) are used in practice. Our design goal is to rely on a *single* PKE scheme both for the regular and anamorphic messages. It is however not clear a priori how to embed the anamorphic message amsg into a single ciphertext ct of the underlying PKE scheme such that decryption recovers both msg and amsg without relying on aforementioned assumptions or properties. To get around this apparent constraint, we embed an anamorphic message amsg within *multiple* ciphertexts of the underlying PKE. We formalize this as public-key anamorphic μ -message extension and consequently an anamorphic ciphertext consists of μ (not necessarily consecutively sent) ciphertexts of the underlying PKE. This can be seen as distributing a ciphertext of the PKE carrying amsg over μ ciphertexts containing regular messages $(m_1, \dots, m_\mu)$ generated with the *same* PKE. To formally capture this, we additionally introduce the notion of PKAE-compatibility, which requires to generate double-keys *on the fly*, and importantly *ciphertext extractibility*, demanding that the existence of an efficiently computable function F such that the anamorphic message amsg can

be computed by the decryption algorithm of the PKE on ciphertext $F(\text{act})$ computed from the anamorphic ciphertext act , i.e., $\text{PKE.Dec}(\text{tk}, F(\text{act})) = \text{amsg}$. Latter ensures that decryption of an anamorphic ciphertext can be performed using the decryption algorithm of the PKE. As a result, it precludes the possibility of detection by an adversarial dictator through side-channel analysis, which could otherwise reveal deviations from the standard decryption procedure when processing anamorphic ciphertexts on a recipient's device.

We note that Wang, Chen, Huang and Yung [WCHY23] proposed the notion of ℓ -receiver anamorphic encryption where an ℓ -bit amsg is embedded using a *symmetric* anamorphic channel across ℓ regular PKE ciphertexts such that the i th ciphertext encodes the i th bit of amsg by relying on pseudorandom PKEs. On the contrary, we encode a *single* amsg using an *asymmetric* anamorphic channel across μ (we achieve this with $\mu = 2$) regular ciphertexts using the *same* PKE for both the regular and anamorphic messages without relying on any specific property of PKE or additional assumptions such as pseudorandom PKE ciphertexts. Next, we demonstrate a public-key anamorphic 2-message extension for the ElGamal and the Dual-Regev encryption schemes.

PKAE based on ElGamal. We consider the ElGamal scheme with public key $\text{pk} = g^{\text{sk}}$, ciphertext $C = (g^t, g^{t \cdot \text{sk} + \text{msg}})$ encrypting a message msg using a random integer t . One could wonder whether it is possible to embed an ElGamal ciphertext $(g^\kappa, g^{\kappa \cdot \text{tk} + \text{amsg}})$ into a *single* ElGamal ciphertext $(r_1 = g^t, c_1 = g^{t \cdot \text{sk} + \text{msg}})$. Since t is uniformly random over $\mathbb{Z}_q$, it is not possible to extract something meaningful if we try to embed $g^{\kappa \cdot \text{tk} + \text{amsg}}$ into r_1 . On the other hand, both g^κ and r_1 are statistically close. Hence, we can directly set $t = \kappa$ for recovering the correct anamorphic message during decryption. Consequently, the only possible option left is to somehow encode $g^{\kappa \cdot \text{tk} + \text{amsg}}$ into c_1 with $t = \kappa$. Thus, we have the following equation $\kappa \cdot \text{tk} + \text{amsg} = \kappa \cdot \text{sk} + \text{msg}$. Given a pair of messages msg, amsg , this equality cannot hold with high probability when tk, sk are independently and uniformly sampled from $\mathbb{Z}_q$. As a result, we try to embed the ciphertext $(g^\kappa, g^{\kappa \cdot \text{tk} + \text{amsg}})$ into $\mu = 2$ ElGamal ciphertexts.

More specifically, we can set the double keys as $\text{dk} := (\text{dk}_1 = g^\alpha, \text{dk}_2 = g^{\alpha \cdot \text{sk}})$ and $\text{tk} := \alpha$. Embedding an anamorphic message amsg into two ciphertexts for messages msg_1 and msg_2 will result in $C_1 = (r_1, c_1) = (g^{\kappa_1}, \text{pk}^{\kappa_1} \cdot \text{msg}_1)$ and $C_2 = (r_2, c_2) = (\text{dk}_1^{\kappa_1} \cdot g^{\text{amsg}}, \text{dk}_2^{\kappa_1} \cdot \text{pk}^{\text{amsg}} \cdot \text{msg}_2)$. Decrypting the normal messages works as usual and the anamorphic message can be recovered by calling ElGamal decryption on (r_1, r_2) and finding $\log g^{\text{amsg}}$. We prove fully-asymmetric IND-CPA security and sIND-CPA security of the PKAE. To show the fully-asymmetric IND-CPA security, we first rely on the DDH assumption with the instance $(g^{\kappa_1}, g^{\text{tk}}, g^{\kappa_1 \cdot \text{tk}})$ to replace r_2 with a random element $r_2 = g^{\kappa_2}$ for some uniformly random integer κ_2 , hiding the anamorphic message amsg within the ciphertext (in the dictator's view, guaranteeing anamorphic security). At this stage, both C_1 and C_2 become two independent ElGamal ciphertexts encrypting msg_1 using κ_1 and msg_2 using κ_2 respectively. Therefore, the ElGamal encryption scheme under the public key pk hides $\text{msg}_1, \text{msg}_2$, ensuring fully asymmetric IND-CPA security of the PKAE. Further, in Section 5.1, we show sIND-CPA security assuming the DDH assumption where the adversary gets sk along with pk and dk .

PKAE based on Dual-Regev. Before sketching the idea behind our PKAE construction based on the Dual-Regev cryptosystem, we first recall the basic scheme: the key generation algorithm samples a vector $\mathbf{k} \leftarrow \{0, 1\}^m$, a matrix $\mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m}$, and returns $\text{pk} := (\mathbf{A}, \mathbf{b} = \mathbf{A} \cdot \mathbf{k})$ and $\text{sk} := \mathbf{k}$. To encrypt a message msg , the encryption algorithm chooses a vector $\mathbf{s} \leftarrow \mathbb{Z}_q^n$, a vector

$\mathbf{e} \leftarrow \chi^m$, and an integer $y \leftarrow \chi$, where χ is an LWE noise distribution, and computes ciphertext as $(\mathbf{r}^T, c) := (\mathbf{s}^T \cdot \mathbf{A} + \mathbf{e}^T, \mathbf{s}^T \cdot \mathbf{b} + y + \text{msg} \cdot \lfloor \frac{q}{2} \rfloor \bmod q)$. Decryption is simply checking whether $|c - \mathbf{r}^T \cdot \mathbf{k}| \leq \frac{q}{4}$. To design a PKAE associated with the Dual-Regev construction, we again replace the random choices $(\mathbf{s}_1, \mathbf{s}_2)$ made during encryption of two regular messages msg_1 and msg_2 with specially chosen values. Intuitively, one might expect to find a solution by taking an approach identical to our ElGamal-based PKAE. Nevertheless, such a straightforward adaptation does not work, which we now account in more detail. Recall that in our ElGamal-based PKAE, we use $\mu = 2$ regular ciphertexts to embed anamorphism by setting the randomness $\kappa_2 = \alpha\kappa_1 + \text{amsg}$, where κ_1 is uniformly random in $\mathbb{Z}_q$, such that amsg could be recovered by treating $(r_1 = g^{\kappa_1}, r_2 = g^{\kappa_2})$ as an ElGamal ciphertext and executing regular ElGamal decryption. This worked fine as an ElGamal ciphertext indeed contains two group elements. However, if we proceed in a similar manner with our Dual-Regev anamorphic construction, i.e., bias the randomness $\mathbf{s}_2$ in $\mathbf{r}_2^T = \mathbf{s}_2^T \cdot \mathbf{A} + \mathbf{e}_2^T$ (first component of second regular Dual-Regev ciphertext) such that amsg is encoded within, then we are not able to recover amsg since there is no way to extract $\mathbf{s}_2$ from $\mathbf{r}_2$. One could aim for a better approach by encoding amsg in $\mathbf{s}_2^T \cdot \mathbf{A} + \mathbf{e}_2^T$ and treating $(\mathbf{r}_1^T, \mathbf{r}_2^T)$ as a regular Dual-Regev ciphertext. But this again fails as a Dual-Regev ciphertext comprises of a vector $\mathbf{r}$ and a scalar c .

We need a slightly different strategy to embed the anamorphic message into $\mathbf{r}_2$. Let $\hat{\mathbf{b}}$ be the encryption double key dk such that $\hat{\mathbf{b}} = \mathbf{A}\hat{\mathbf{k}}$ where $\hat{\mathbf{k}} \leftarrow \{0, 1\}^m$ is the decryption double key tk . As in the case of ElGamal-based PKAE, the first ciphertext component encrypting amsg is the first ciphertext component of $C_1 = (\mathbf{r}_1^T, c_1) := (\mathbf{s}_1^T \cdot \mathbf{A} + \mathbf{e}_1^T, \mathbf{s}_1^T \cdot \mathbf{b} + y_1 + \text{msg}_1 \cdot \lfloor \frac{q}{2} \rfloor \bmod q)$. To recover the second ciphertext component $\mathbf{s}_1^T \cdot \hat{\mathbf{b}} + \hat{y} + \text{amsg} \cdot \lfloor \frac{q}{2} \rfloor$ encrypting amsg from $C_2 = (\mathbf{r}_2^T, c_2) := (\mathbf{s}_2^T \cdot \mathbf{A} + \mathbf{e}_2^T, \mathbf{s}_2^T \cdot \mathbf{b} + y_2 + \text{msg}_2 \cdot \lfloor \frac{q}{2} \rfloor \bmod q)$, our idea is to sample $\mathbf{s}_2$ such that the first entry of $\mathbf{r}_2^T$ is $\mathbf{s}_1^T \cdot \hat{\mathbf{b}} + \hat{y} + \text{amsg} \cdot \lfloor \frac{q}{2} \rfloor$. More precisely, $\mathbf{s}_2$ is sampled as follows:

$$\begin{aligned} x_i &\leftarrow \mathbb{Z}_q \text{ for } i \in [1, n-1], \\ x_n &= \left(- \sum_{i=1}^{n-1} x_i a_i + \mathbf{s}_1^T \cdot \hat{\mathbf{b}} + \hat{y} + \text{amsg} \cdot \lfloor \frac{q}{2} \rfloor - e_{2,1} \right) \cdot a_n^{-1} \bmod q, \\ \mathbf{s}_2^T &= [x_1, \dots, x_n] \end{aligned}$$

where $\mathbf{a} = (a_1, \dots, a_n)$ is the first column of $\mathbf{A}$ and $e_{2,1}$ is the first entry of $\mathbf{e}_2$. Therefore, the first entry of $\mathbf{r}_2^T$ becomes $\hat{c} = \mathbf{s}_1^T \cdot \hat{\mathbf{b}} + \hat{y} + \text{amsg} \cdot \lfloor \frac{q}{2} \rfloor \bmod q$. Thus, one can efficiently extract the regular Dual-Regev ciphertext $(\mathbf{r}_1^T, \hat{c})$ and recover amsg using $\text{tk} = \hat{\mathbf{k}}$. We prove fully asymmetric IND-CPA and strong IND-CPA security of the PKAE. We first show that under the DLWE assumption $\mathbf{s}_1^T \cdot \hat{\mathbf{b}} + \hat{y}$ is uniformly distributed over $\mathbb{Z}_q$, ensuring uniformity of x_n over $\mathbb{Z}_q$ in the adversary's view (even in the dictator's view as it does not have $\text{tk} = \hat{\mathbf{k}}$, guaranteeing anamorphic security). As a result, C_1, C_2 become independent regular Dual-Regev-ciphertexts encrypting $\text{msg}_1, \text{msg}_2$ respectively. In case of fully asymmetric IND-CPA security, the updated C_1, C_2 hide $\text{msg}_1, \text{msg}_2$ under the DLWE assumption.

Anamorphic clans. While the entire revolutionary theme of anamorphic cryptography is built on top of a totalitarian government regime, it is quite natural to imagine a setting where secret rebellious communities exist within. A prominent use-case would be an underground community liaising secretly to advocate freedom of citizens from the dictatorship rule. Additionally, the members of such communities must be subject to background checks to avoid infiltration by the government, and hence managed by a leader; we call such communities *anamorphic clans*.

Having introduced the concept of anamorphic clans, consider a scenario where the encryption double key dk of an anamorphic clan member is revealed to the dictator. A dictator could simply procure dk when getting distributed within the clan, or by corrupting its member(s). Intuitively, one might expect the dictator to forcefully demand the corresponding decryption double key tk . However, the dictator may have a stronger motive of silently surveilling the communication patterns to learn the communicating peers of the alleged clan member, and hence not demand tk immediately. It could also be the case that the dictator is unable to link the procured dk to a clan member and hence wishes to investigate who is anamorphically communicating with whom. This demands us to study a *stronger* security notion which is beyond the usual IND-CPA notion, where the adversary compromises both ask and dk ; we call it sIND-CPA security. Additionally, the dictator could also learn a list of anamorphic messages and corresponding anamorphic ciphertexts from a corrupted clan member. This motivates us to consider an *even stronger* security notion where we allow the adversary to access an oracle $aDec(tk, act)$ that decrypts anamorphic ciphertexts. In other words, we consider IND-CCA security for anamorphic messages.

PKAE extensions with security beyond IND-CPA. Another goal of this paper is to build PKAE for popular IND-CCA secure schemes using the same idea of a multi-message extension. Existing results for PKAE based on IND-CCA secure PKEs are based on theoretical constructions such as Koppula-Waters. Moreover, and interestingly, previous works though looking at anamorphisms for CCA secure schemes only guarantee IND-CPA security for the anamorphic messages and consequently no integrity protection at all. We study the use of the popular FO transformation [FO99, FO13] which is particularly relevant in the construction of post-quantum PKEs such as Kyber. We recall that FO encrypts a random message r with a weakly secure (e.g., IND-CPA secure PKE) using randomness derived from a hash function H on $H(r, msg)$ with msg being the message that one wants to encrypt. The random message r is then used as a basis to derive a key $G(r)$, with G being another hash function, for a symmetric encryption (SKE) scheme which then encrypts msg . The overall scheme then achieves IND-CCA security in the random oracle model. The basic idea of our construction is to use two FO-layers, an inner and outer layer of FO encryption. Basically, the inner-layer is used to encrypt the anamorphic message $amsg$ where the PKE scheme and the SKE scheme are required to have pseudorandom ciphertexts (in the IND\$ sense). Then we use a pseudo-random encoding (e.g., Elligator Squared [Tib14] in case of elliptic curves) to encode the inner-layer ciphertext into the message space of the PKE of the outer-layer FO encryption. Due to the property of the FO-transformation, i.e., randomness recoverability, we can thus use the randomness obtained from decrypting the outer-layer to decode this into a ciphertext of the inner-layer and decrypt. Interestingly, for such schemes, we achieve strong RCCA security for the anamorphic message.

Identity-based anamorphic encryption (IBAE). Deploying anamorphic clans using PKAE would require the anamorphic clan lead to generate the double key-pairs (dk, tk) , sending the double decryption keys to the members while registering double encryption keys for all the clan members. Clearly, such a solution would not be desirable in light of the overall complexity in generating and storing the double keys, and would additionally lead to scalability concerns. To overcome this, we could think of using identity-based encryption to establish the anamorphic channel, where the clan lead generates double master key-pairs $(dmpk, dmsk)$, derives and distributes double private keys dsk_{id} to clan members. Nevertheless, we are unsure how to realize such an anamorphic IBE

channel within the dictator-approved PKE scheme. Thus, we ask: *could we reformulate anamorphic encryption through the lens of identity-based encryption?* More specifically, could we design a covert anamorphic IBE channel within a regular IBE channel? This would require the dictatorship to authorize identity-based encryption, but why would that be convincing? However, as argued within, and demonstrated by our following discussion, identity-based encryption could be the most natural way to formulate anamorphic cryptography. To start, in an identity-based encryption setting, a dictator would independently sample IBE master keys (mpk, msk) and generate private keys sk_{id} associated with the public identity id of its citizens at any stage during the existence of the system. As an alternative approach, the dictator can also compel an IBE key generation authority to submit the master key pair. Securing the master keys will allow the dictator to perform silent surveillance within its regime as any message encrypted with respect to any id can be decrypted by the dictator without forcing citizens to submit private keys on demand. Finally, the dictator can derive private keys as convenient, and hence there is no need to store the users' private keys, contrary to the realization of an anamorphic clan in the PKAE settings.

We formalize the above intuition by introducing the notion of IBAE-compatibility which captures two properties. The first *on-the-fly double key generation* property captures the essence that a double master key pair $(\text{dmpk}, \text{dmsk})$ can be generated on-the-fly using only the knowledge of mpk , and can be deployed in two different dictatorial settings : (1) *authority-before-dictatorship* where the dictator coerces an existing IBE key generation authority to obtain the IBE master keys (mpk, msk) , and later this authority can freshly generate double master key pair $(\text{dmpk}, \text{dmsk})$ by independently executing the IBE setup algorithm and using its knowledge of mpk , eventually acting as the anamorphic clan leader, and (2) *authority-after-dictatorship* where the dictator itself generates the IBE master keys (mpk, msk) , and an anamorphic clan leader can independently sample double master key pair $(\text{dmpk}, \text{dmsk})$ using only mpk (which is made public by the dictator). The second *ciphertext extractability* property allows the anamorphic IBE decryption algorithm to be essentially the same as the regular IBE decryption algorithm, which additionally circumvents any possible dictatorial efforts of side-channel analysis on the recipient's device.

Therefore, IBE seems to be a premier candidate for an encryption scheme allowed by a dictator. We initiate the study of identity-based anamorphic encryption (IBAE) which allows embedding an IBE anamorphic channel inside an IBE. We present a concrete instantiation of anamorphic IBE based on the seminal Boneh-Franklin scheme [BF01] using our multi-message extension concept. Before discussing the intuition behind our IBAE constructions, we briefly recall the Boneh-Franklin BasicIdent scheme: the master public key is $\text{mpk} = (\mathbb{G}, \mathbb{G}_T, P, e, q, P_{\text{pub}} = P^{\text{msk}}, H_1, H_2)$, the private key for id is $\text{sk}_{\text{id}} = H_1(\text{id})^{\text{msk}}$, a ciphertext for id is $(r, c) = (P^\kappa, \text{msg} \oplus H_2(g_{\text{id}}^\kappa))$, where $\kappa \leftarrow \mathbb{Z}_q^*$, $g_{\text{id}} = e(H_1(\text{id}), P_{\text{pub}})$. The decryption algorithm simply computes $c \oplus H_2(e(\text{sk}_{\text{id}}, r))$ and extracts msg .

A 2-IBAE associated with the Boneh-Franklin (BF) IBE [BF01], works roughly as follows. Unlike the anamorphic extension of the ElGamal encryption scheme, where for two ciphertexts the first components (r_1, r_2) represent an ElGamal encryption of the anamorphic message, embedding amsg within two regular Boneh-Franklin ciphertexts is not that straightforward. In a BF ciphertext $C_2 = (r_2 = P^{\kappa_2}, c_2 = \text{msg} \oplus H_2(g_{\text{id}}^{\kappa_2}))$, the group element $g_{\text{id}}^{\kappa_2}$ is not explicitly available, where $g_{\text{id}} = e(H_1(\text{id}), P^{\text{amsk}})$. Rather, it only appears as an input to the hash function H_2 . Therefore, if we set κ_2 following the ElGamal encryption extension, then it seems impossible to recover the anamorphic message amsg due to the one-wayness of H_2 . At this point, the only solution appears to be encoding amsg outside the hash function H_2 . Consequently, we use rejection sampling such that amsg could be extracted from c_2 . More specifically, we sample κ_2 subject to $H_2(g_{\text{id}}^{\kappa_2}) =$

$\text{amsg} \oplus H_2(\hat{g}_{\text{id}}^{\kappa_1})$ where $\hat{g}_{\text{id}} = e(H_1(\text{id}), P^{\text{dmsk}})$. We then treat $(r_1 = P^{\kappa_1}, \hat{c} := \text{amsg} \oplus H_2(\hat{g}_{\text{id}}^{\kappa_1}))$ as a regular BF ciphertext and apply the regular decryption algorithm of BF using $\text{dsk}_{\text{id}} = H_1(\text{id})^{\text{dmsk}}$ to recover amsg . Using this technique, we can convey anamorphic messages of at most $O(\log \lambda)$ bits to ensure that encryption remains polynomial.

Let $\text{dmsk} = \alpha$ be the double master secret key. To show the anamorphic security, we rely on the DBDH assumption with the instance $(P^r, P^\alpha, P^{\kappa_1}, e(P, P)^t)$ where t can be either $r\alpha\kappa_1$ or random. Once Q_{id} is set as $H_1(\text{id}) = P^r$ (by viewing H_1 as a random oracle), we can write $\hat{g}_{\text{id}}^{\kappa_1} = e(P^r \cdot \text{dmsk}, P^{\kappa_1}) = e(P, P)^t$ which is uniformly distributed over $\mathbb{G}_T$ under the DBDH assumption. Since msk is known we can still simulate $g_{\text{id}}^{\kappa_1} = e(P^r \cdot \text{msk}, P^{\kappa_1})$ for C_1 . Finally, as H_2 is viewed as a random oracle, $\text{amsg} \oplus H_2(P^t)$ can be replaced by a random bit, ensuring that amsg is hidden from the dictator's view. We present the details of fully asymmetric IND-CPA and strong IND-CPA security proofs for the IBAE (which are proven depending on the IND-CPA security of the basic BF-IBE scheme) in Section 6.1.

Similar to PKAE, we then explore the use of the FO-transformation to create an anamorphic channel which achieves strong IND-RCCA security and allows for λ -bit anamorphic messages. Standardized Boneh-Franklin and Sakai-Kasahara are covered by our results.

2.2 μ -Message Public Key Anamorphic Signatures Extensions

Anamorphic signatures are a recent concept due to [KPP⁺23a] but, so far, they are not considered in the context of asymmetric anamorphic channels. Our aim is to investigate such public-key anamorphic signatures (PKAS) based on simple combinations of well-known signature and PKE schemes.

Similar to our PKAE approach, we define μ -message anamorphic extension of a signature scheme, i.e., we embed an anamorphic message into multiple signatures. Besides anamorphic security, i.e., the distributions of regular and anamorphic signatures must be indistinguishable, we also define an IND-CPA notion. In contrast to encryption, however, now we only have to care about amsg being hidden from someone knowing dk . We also consider a CCA variant of this notion as well as a strong RCCA notion where the adversary additionally receives the signing key ask .

Towards such constructions, our main observation is that there are well-known schemes where signatures contain some uniformly random components and the remaining components can be computed from the former in a “black-box way” (e.g., without knowing their discrete logarithms). This opens up the possibility to embed pseudo-random ciphertexts (from an IND-CPA secure PKE) into these random signature components. The number of random signature components and the number of ciphertext components, then controls how many signatures are required to transmit one anamorphic message. We put this intuition into a generic formal framework of anamorphism-friendly signatures and compliant encryption which we rigorously formalize. Depending on the security of the compliant PKE, we can then achieve either IND-CPA, sIND-RCCA and if the signature scheme satisfies strong unforgeability (i.e., SUF-CMA) also IND-CCA security for anamorphic messages.

We then show how to instantiate this framework based on Water's signatures [Wat05] together with linear encryption [BBS04] as well as BBS signatures [BBS04, TZ23], currently being standardized, together with (FO-)ElGamal encryption, latter with the help of the Elligator Squared encoding [Tib14].

To provide an intuition, for Water's signatures the hash is computed as $H(\text{msg}) = h_0 \prod_{i=1}^n h_i^{m_i}$, where the m_i is the i 'th bit of message msg . A signature is $(g^r, g^{\alpha\beta} H(\text{msg})^r)$ for random r , secret key $g^{\alpha\beta}$ and public key $(g^\alpha, g^\beta, h_0, \dots, h_n)$. Also recall that in linear encryption the public key

is $\mathbf{pk} = (u, v, h) \in \mathbb{G}^3$ and the secret key $(x, y) \in \mathbb{Z}_q$ such that $u^x = v^y = h$ holds. Encrypting a message $\mathbf{msg} \in \mathbb{G}$ amounts to choosing random κ_1, κ_2 and computing ciphertext $(c_1, c_2, c_3) = (u^{\kappa_1}, v^{\kappa_2}, h^{\kappa_1 + \kappa_2} \cdot \mathbf{msg})$ which looks pseudorandom under the decision linear (DLIN) assumption and decryption computes $\mathbf{msg} = c_3 \cdot (c_1^x \cdot c_2^y)^{-1}$. When making the assumption that every user generates its own parameters for the Water's hash and thus knows a_i such that $h_i = g^{a_i}$ for $i \in \{0, \dots, n\}$ and this is tolerated by the dictator², we can construct a simple PKAS. Using the knowledge of these discrete logarithms allows us to transmit an anamorphic message $\mathbf{amsg}$ by embedding one ciphertext component of a ciphertext (c_1, c_2, c_3) encrypting $\mathbf{amsg}$ into the first components of three Water's signatures.

Identification of anamorphic ciphertexts/signatures in a communication channel. An interesting question to consider is how to identify which set of μ ciphertexts (in a given list of ciphertexts) must be combined to anamorphically decrypt $\mathbf{amsg}$. One can typically assume that a receiver does not obtain a large number of ciphertexts/signatures at a certain point of time, and hence one can use a combinatorial approach that simply tries to decrypt any (μ -sized) subset. However, when considering a large number of ciphertexts/signatures, this brute-force approach becomes inefficient. An interesting way forward would be for a sender to share an ordering (which automatically fixes the ciphertexts to be combined during anamorphic decryption) with the receiver, and this ordering can be included as part the encryption double key $\mathbf{dk}$. The next step would be to rely on connection-oriented transport layer protocols, such as TCP, which allow setting up a reliable end-to-end connection and communicate data segments with unique sequencing. Finally, the segments are received in an ordered fashion at the destination. Additionally, due to its error-handling capability, TCP also allows retransmission of lost and out-of-order segments. Such lower layer protocols can ensure that ciphertexts are received in the correct order, and using the pre-shared ordering (part of $\mathbf{dk}$), the receiver can determine which of the μ sequences must be combined during anamorphic decryption. Hence, we believe that μ -message anamorphic ciphertexts/signatures could be easily deployed following some additional assumptions, however, formalizing the implementation specifications is out of the scope of this work.

3 Preliminaries

3.1 Notations

Let $\lambda \in \mathbb{N}$ denote the security parameter. We use PPT to denote a randomized algorithm running in time $a \cdot \lambda^c$ for some constants a and c . A function $f : \mathbb{N} \rightarrow \mathbb{R}^+$ is called negligible in λ , if it grows slower than λ^{-c} for every constant c . We use $\text{negl}(\lambda)$ to denote a generic negligible function. We denote the set of positive integers upto n as $[n] := \{1, \dots, n\}$. For a finite set S , we use $x \leftarrow S$ to denote the uniformly sampling of an element x from S . For a probabilistic algorithm $\mathcal{A}$, we let $y \leftarrow \mathcal{A}(x)$ denote the process of running $\mathcal{A}(x)$ with fresh randomness and assigning the result to y . We denote $\mathcal{D}_1 \approx_c \mathcal{D}_2$ to represent computational indistinguishability between two distributions $\mathcal{D}_1$ and $\mathcal{D}_2$. We represent empty strings using ϵ . We use lowercase bold letters to denote vectors (e.g. $\mathbf{k}$) and uppercase bold letters for matrices (e.g. $\mathbf{A}$). We strictly follow the convention of all vectors $\mathbf{k}$ being column vectors, and denote the corresponding row vector as $\mathbf{k}^T$.

²For the second construction based on BBS we do not require such an assumption

3.2 Computational Assumptions

Next, we recall the various hardness assumptions used in our constructions. We begin with the DDH and the DBDH assumptions, which are used in our ElGamal and Boneh-Franklin-based anamorphic encryption schemes.

Definition 1 (The DDH Assumption [Bon98]) *Let $\lambda \in \mathbb{N}$ be the security parameter and let q be a polynomial in λ . Let $\mathbb{G} = \langle g \rangle$ be a group of order $q > 2^\lambda$. Then for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that for all $\lambda \in \mathbb{N}$, the following holds:*

$$|\Pr[a, b \leftarrow \mathbb{Z}_q : \mathcal{A}(g^a, g^b, g^{ab}) = 1] - \Pr[a, b, c \leftarrow \mathbb{Z}_q : \mathcal{A}(g^a, g^b, g^c) = 1]| \leq \text{negl}(\lambda)$$

where the probability is taken over the coin tosses of $\mathcal{A}$ and random choices of a, b, c and g .

Definition 2 (The DBDH Assumption [BF01]) *Let $q \in \mathbb{N}$ be a large prime and let $(\mathbb{G}, \mathbb{G}_T, g, e, q)$ be a pairing group, where g is a generator of $\mathbb{G}$. Then for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that for all $\lambda \in \mathbb{N}$, the following holds:*

$$|\Pr[a, b, c \leftarrow \mathbb{Z}_q : \mathcal{A}(g^a, g^b, g^c, e(g, g)^{abc}) = 1] - \Pr[a, b, c, t \leftarrow \mathbb{Z}_q : \mathcal{A}(g^a, g^b, g^c, e(g, g)^t) = 1]| \leq \text{negl}(\lambda)$$

where the probability is taken over the coin tosses of $\mathcal{A}$, and random choices of a, b, c, t, g .

Next, we recall the Leftover Hash Lemma [ILL89] and the DLWE assumption [Reg09] used in our Dual-Regev based PKAE constructions.

Lemma 1 (Leftover Hash Lemma [ILL89]) *Let $\mathcal{H}$ be a universal hash function family consisting of function $h : \mathcal{X} \rightarrow \mathcal{Y}$. Let X, Z be random variables such that $\mathbb{H}_\infty(X|Z) \geq \log |\mathcal{Y}| + 2 \log(1/\epsilon)$ for some $\epsilon > 0$ and X is supported over $\mathcal{X}$. Let H, Y be uniformly random and independent over $\mathcal{H}, \mathcal{Y}$ respectively. Then $\mathbf{SD}((H, H(X), Z), (H, Y, Z)) \leq \epsilon$.*

Definition 3 (The DLWE Assumption [Reg09]) *Let $q \in \mathbb{N}$ be a large prime and let $n, m \in \mathbb{N}$. Let χ be a noise distribution over $\mathbb{Z}_q$. The (n, q, χ) -DLWE assumption of dimension m states that the following distributions are computationally indistinguishable:*

$$(\mathbf{A}, \mathbf{s}^T \cdot \mathbf{A} + \mathbf{e}^T) \stackrel{c}{\approx} (\mathbf{A}, \mathbf{u}) : \mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m}, \mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \chi^m, \mathbf{u} \leftarrow \mathbb{Z}_q^m$$

We now state the CDH problem used in our PKAS construction based on Water's signatures.

Definition 4 (The CDH Problem [DH21]) *Let $q \in \mathbb{N}$ be a large prime and let $\mathbb{G} = \langle g \rangle$ be a multiplicative cyclic group of order q . Let $a, b \leftarrow \mathbb{Z}_q$. Then the CDH problem is to compute g^{ab} .*

We now recall the DLIN assumption by Boneh, Boyen and Shacham [BBS04] used in our Waters signature based public key anamorphic signature.

Definition 5 (The DLIN Assumption [BBS04]) *Let $q \in \mathbb{N}$ be a large prime and let $\mathbb{G}$ be a multiplicative cyclic group of order q , where u, v, h are uniformly random generators of $\mathbb{G}$. Let a, b be uniformly random elements of $\mathbb{Z}_q$. Then for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that for all $\lambda \in \mathbb{N}$, the following holds:*

$$|\Pr[u, v, h \leftarrow \mathbb{G} : \mathcal{A}(u, v, h, u^a, v^b, h^{a+b}) = 1] - \Pr[u, v, h, \eta \leftarrow \mathbb{G} : \mathcal{A}(u, v, h, u^a, v^b, \eta) = 1]| \leq \text{negl}(\lambda)$$

where the probability is taken over the uniformly random choice of parameters to $\mathcal{A}$, and over the coin tosses of $\mathcal{A}$.

3.3 Secret Key Encryption

Definition 6 (Secret Key Encryption) We say that a triplet $\Pi_{\text{SKE}} = (\text{Gen}, \text{Enc}, \text{Dec})$ of PPT algorithms with a message space $\mathcal{M}$ is an IND-CPA secure secret key encryption (SKE) scheme if the following holds:

$\text{sk} \leftarrow \text{Gen}(1^\lambda)$: The key generation algorithm takes as input the security parameter 1^λ , and outputs a secret key sk .

$\text{ct} \leftarrow \text{Enc}(\text{sk}, \text{msg})$: The encryption algorithm takes as input the secret key sk , and a message $\text{msg} \in \mathcal{M}$, and outputs a ciphertext ct .

$\text{msg} \leftarrow \text{Dec}(\text{sk}, \text{ct})$: The decryption algorithm takes as input the secret key sk and the ciphertext ct , and outputs a message msg .

Furthermore, the algorithms must satisfy the following correctness and security properties.

Correctness: For every message $\text{msg} \in \mathcal{M}$, the following holds:

$$\Pr \left[\text{msg} = \text{Dec}(\text{sk}, \text{ct}) : \begin{array}{l} \text{sk} \leftarrow \text{Gen}(1^\lambda) \\ \text{ct} \leftarrow \text{Enc}(\text{sk}, \text{msg}) \end{array} \right] = 1$$

IND-CPA security: For every PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that for all $\lambda \in \mathbb{N}$, the following holds:

$$|\Pr[\text{GIND-CPA}_{\text{SKE}}^{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\text{GIND-CPA}_{\text{SKE}}^{\mathcal{A}}(\lambda, 1) = 1]| \leq \text{negl}(\lambda)$$

where the security game is defined as follows:

$\text{GIND-CPA}_{\text{SKE}}^{\mathcal{A}}(\lambda, \beta)$:

1. $\text{sk} \leftarrow \text{Gen}(1^\lambda)$
2. $(\text{msg}_0, \text{msg}_1) \leftarrow \mathcal{A}(1^\lambda)$
3. $\text{ct}_\beta = \text{Enc}(\text{sk}, \text{msg}_\beta)$
4. **return** $\beta' \leftarrow \mathcal{A}(\text{ct}_\beta)$

Additional IND-CPA security definition. We consider an additional security definition for SKE, called weak IND\$-CPA (wIND\$-CPA) security.

Definition 7 Let Π_{SKE} be an SKE with ciphertext space denoted by $\mathcal{C}$. We call Π_{SKE} wIND\$-CPA secure if for every PPT adversary $\mathcal{A}$ it holds that

$$|\Pr[\text{GwIND\$-CPA}_{\text{SKE}}^{\mathcal{A}}(\lambda, 1) = 1] - \Pr[\text{GwIND\$-CPA}_{\text{SKE}}^{\mathcal{A}}(\lambda, 0) = 1]| \leq \text{negl}(\lambda)$$

where the security game is defined as follows:

GwIND\$-CPA_{SKE}^A(λ, β) :

1. $\text{sk} \leftarrow \text{Gen}(1^\lambda)$
2. $\text{msg} \leftarrow \mathcal{A}(1^\lambda)$
3. $\text{ct}_0 \leftarrow \text{Enc}(\text{sk}, \text{msg})$
4. $\text{ct}_1 \leftarrow \mathcal{C}$
5. **return** $\beta' \leftarrow \mathcal{A}(\text{ct}_\beta)$

3.4 Public Key Encryption

We first recall the standard syntax of PKE which we require to have indistinguishable encryptions under the chosen plaintext attack (IND-CPA). We note that the scheme may also be defined with respect to a global setup $\text{gp} \leftarrow \text{Setup}(1^\lambda)$ generating parameters gp used by Gen .

Definition 8 (Public Key Encryption) *We say that a triplet $\Pi_{\text{PKE}} = (\text{Gen}, \text{Enc}, \text{Dec})$ of PPT algorithms with a message space $\mathcal{M}$ is an IND-CPA secure public key encryption (PKE) scheme if the following holds:*

$(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$: *The key generation algorithm takes as input the security parameter 1^λ , and outputs a public key pk and a secret key sk .*

$\text{ct} \leftarrow \text{Enc}(\text{pk}, \text{msg})$: *The encryption algorithm takes as input the public key pk , and a message $\text{msg} \in \mathcal{M}$, and outputs a ciphertext ct .*

$\text{msg} \leftarrow \text{Dec}(\text{sk}, \text{ct})$: *The decryption algorithm takes as input the secret key sk and a ciphertext ct , and outputs a message msg .*

Furthermore, the algorithms must satisfy the following correctness and security properties.

Correctness: *For every message $\text{msg} \in \mathcal{M}$, the following holds:*

$$\Pr \left[\text{msg} = \text{Dec}(\text{sk}, \text{ct}) : \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda) \\ \text{ct} \leftarrow \text{Enc}(\text{pk}, \text{msg}) \end{array} \right] = 1$$

IND-CPA security: *For every PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that for all $\lambda \in \mathbb{N}$, the following holds:*

$$|\Pr[\text{GIND-CPA}_{\text{PKE}}^{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\text{GIND-CPA}_{\text{PKE}}^{\mathcal{A}}(\lambda, 1) = 1]| \leq \text{negl}(\lambda)$$

where the security game is defined as follows:

GIND-CPA_{PKE}^A(λ, β) :

1. $(pk, sk) \leftarrow \text{Gen}(1^\lambda)$
2. $(msg_0, msg_1) \leftarrow \mathcal{A}(pk)$
3. $ct_\beta = \text{Enc}(pk, msg_\beta)$
4. **return** $\beta' \leftarrow \mathcal{A}(ct_\beta)$

Additional IND-CPA security definition. We consider an additional security definition for PKE, called IND\$-CPA security.

Definition 9 Let Π_{PKE} be a PKE with ciphertext space denoted by $\mathcal{C}$. We call PKE IND\$-CPA secure if for every PPT adversary $\mathcal{A}$ it holds that

$$|\Pr[\text{GIND\$-CPA}_{\text{PKE}}^{\mathcal{A}}(\lambda, 1) = 1] - \Pr[\text{GIND\$-CPA}_{\text{PKE}}^{\mathcal{A}}(\lambda, 0) = 1]| \leq \text{negl}(\lambda)$$

where the security game is defined as follows:

GIND\$-CPA_{PKE}^A(λ, β) :

1. $(pk, sk) \leftarrow \text{Gen}(1^\lambda)$
2. $msg \leftarrow \mathcal{A}(pk)$
3. $ct_0 \leftarrow \text{Enc}(pk, msg)$
4. $ct_1 \leftarrow \mathcal{C}$
5. **return** $\beta' \leftarrow \mathcal{A}(ct_\beta)$

3.5 Identity-Based Encryption

Definition 10 (Identity-Based Encryption) We say that a triplet $\Pi_{\text{IBE}} = (\text{Setup}, \text{Enc}, \text{Dec}, \text{KeyGen})$ of PPT algorithms with a message space $\mathcal{M}$, identity space $\mathcal{ID}$ is an IND-CPA secure identity-based encryption (PKE) scheme if the following holds:

$(mpk, msk) \leftarrow \text{Setup}(1^\lambda)$: The setup algorithm takes as input the security parameter 1^λ , and outputs a master public key mpk and a master secret key msk .

$sk_{id} \leftarrow \text{KeyGen}(msk, id)$: The key generation algorithm takes as input the master secret key msk and an identity $id \in \mathcal{ID}$, and outputs a secret key sk_{id} .

$ct \leftarrow \text{Enc}(mpk, id, msg)$: The encryption algorithm takes as input the master public key mpk , an identity $id \in \mathcal{ID}$ and a message $msg \in \mathcal{M}$, and outputs a ciphertext ct .

$msg \leftarrow \text{Dec}(sk_{id}, ct)$: The decryption algorithm takes as input a secret key sk_{id} and a ciphertext ct , and outputs a message msg .

Furthermore, the algorithms must satisfy the following correctness and security properties.

Correctness: For every message $\text{msg} \in \mathcal{M}$ and identity $\text{id} \in \mathcal{ID}$, the following holds:

$$\Pr \left[\begin{array}{l} (\text{mpk}, \text{msk}) \leftarrow \text{Setup}(1^\lambda) \\ \text{msg} = \text{Dec}(\text{sk}_{\text{id}}, \text{ct}) : \text{sk}_{\text{id}} \leftarrow \text{KeyGen}(\text{msk}, \text{id}) \\ \text{ct} \leftarrow \text{Enc}(\text{mpk}, \text{id}, \text{msg}) \end{array} \right] = 1$$

IND-CPA security: For every PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that for all $\lambda \in \mathbb{N}$, the following holds:

$$|\Pr[\text{GIND-CPA}_{\text{IBE}}^{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\text{GIND-CPA}_{\text{IBE}}^{\mathcal{A}}(\lambda, 1) = 1]| \leq \text{negl}(\lambda)$$

where the security game is defined as follows:

GIND-CPA_{IBE}^A(λ, β) :

1. $(\text{mpk}, \text{msk}) \leftarrow \text{Setup}(1^\lambda)$
 2. $(\text{id}^*, \text{msg}_0, \text{msg}_1) \leftarrow \mathcal{A}^{\text{OKeyGen}(\cdot)}(\text{mpk})$
 3. $\text{ct}_\beta = \text{Enc}(\text{mpk}, \text{id}^*, \text{msg}_\beta)$
 4. **return** $\beta' \leftarrow \mathcal{A}^{\text{OKeyGen}(\cdot)}(\text{ct}_\beta)$
- where $\text{OKeyGen}(\text{id}) = \text{KeyGen}(\text{msk}, \text{id})$
and id^* is not queried to $\text{OKeyGen}(\cdot)$.

Additional IND-CPA security definition. We consider an additional security definition for IBE, called IND\$-CPA security.

Definition 11 Let Π_{IBE} be an IBE with ciphertext space denoted by $\mathcal{C}$. We call IBE IND\$-CPA secure if for every PPT adversary $\mathcal{A}$ it holds that

$$|\Pr[\text{GIND\$-CPA}_{\text{IBE}}^{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\text{GIND\$-CPA}_{\text{IBE}}^{\mathcal{A}}(\lambda, 1) = 1]| \leq \text{negl}(\lambda)$$

where the security game is defined as follows:

GIND\\$-CPA_{IBE}^A(λ, β) :

1. $(\text{mpk}, \text{msk}) \leftarrow \text{Setup}(1^\lambda)$
 2. $(\text{id}^*, \text{msg}) \leftarrow \mathcal{A}^{\text{OKeyGen}(\cdot)}(\text{mpk})$
 3. $\text{ct}_0 = \text{Enc}(\text{mpk}, \text{id}^*, \text{msg})$
 4. $\text{ct}_1 \leftarrow \mathcal{C}$
 5. **return** $\beta' \leftarrow \mathcal{A}^{\text{OKeyGen}(\cdot)}(\text{ct}_\beta)$
- where $\text{OKeyGen}(\text{id}) = \text{KeyGen}(\text{msk}, \text{id})$
and id^* is not queried to $\text{OKeyGen}(\cdot)$.

3.6 Signature Scheme

We now recall the definition of a basic signature scheme and note that the scheme may also be defined with respect to a global setup with parameters gp .

Definition 12 (Signature Scheme) *We say that a triplet $\Pi_s = (\text{Gen}, \text{Sig}, \text{Ver})$ of PPT algorithms with a message space $\mathcal{M}$ is a signature scheme if the following holds:*

$(\text{vk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$: *The key generation algorithm Gen takes as input the security parameter 1^λ , and outputs a signing key sk and a verification key vk .*

$\text{sig} \leftarrow \text{Sig}(\text{sk}, \text{msg})$: *The signing algorithm Sig takes as input the signing key sk , and a message $\text{msg} \in \mathcal{M}$ and outputs a signature sig .*

$1 \leftarrow \text{Ver}(\text{vk}, \text{msg}, \text{sig})$: *The verification algorithm takes input the verification key vk , the message msg and the signature sig , and outputs 1.*

Furthermore, the algorithms must satisfy the following correctness and security properties.

Correctness: *For every message $\text{msg} \in \mathcal{M}$, the following holds:*

$$\Pr \left[\text{Ver}(\text{vk}, \text{msg}, \text{sig}) = 1 : \begin{array}{l} (\text{vk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda) \\ \text{sig} \leftarrow \text{Sig}(\text{sk}, \text{msg}) \end{array} \right] = 1$$

Security: *The signature scheme Π_{SS} is secure against chosen-message attacks if for every PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that for all $\lambda \in \mathbb{N}$, the following holds:*

$$|\Pr[\text{GSig-CMA}_{\Pi_{\text{SS}}}^{\mathcal{A}}(\lambda) = 1]| \leq \text{negl}(\lambda)$$

where the security game is defined as follows:

$\text{GSig-CMA}_{\Pi_{\text{SS}}}^{\mathcal{A}}(\lambda)$:

1. $(\text{vk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$
2. $(\text{msg}^*, \text{sig}^*) \leftarrow \mathcal{A}^{\text{OSig}(\text{sk}, \cdot)}(\text{vk})$
where $\text{OSig}(\text{sk}, \text{msg}) = (\text{msg}, \text{sig})$
3. **if** $\text{Ver}(\text{vk}, \text{msg}^*, \text{sig}^*) = 1$ **and**
 $(\text{msg}^*, \text{sig}^*)$ *has not returned by OSig*
return 1;
else return 0

4 Public Key Anamorphic Encryption

We now present the relevant notions for PKAE from the literature. We follow the current line of works on *receiver* anamorphic encryption introduced in [PPY22], and subsequently generalized in [BGH⁺24, CGM24a, KPP⁺23a]. Given our core motivation is to realize covert channels using an asymmetric variant, we follow the foundational work on PKAE in [PPY24, BGH⁺24].

At a high-level, a PKAE is a triplet $\Pi_{\text{PKAE}} = (\text{aGen}, \text{aEnc}, \text{aDec})$ associated with a regular PKE $\Pi_{\text{PKE}} = (\text{Gen}, \text{Enc}, \text{Dec})$ which operates in either of the two modes: *regular* and *anamorphic*. The regular mode is basically the ordinary encryption scheme Π_{PKE} where the receiver generates a key-pair (pk, sk) using Gen , and hands over the secret key sk to the dictator, who can then decrypt any ciphertext ct meant for the receiver. In the anamorphic mode, the receiver runs aGen and generates anamorphic key-pair (apk, ask) along with encryption and decryption *double* keys (dk, tk) . When a sender wants to send an anamorphic message amsg , she chooses an innocuous-looking message msg and runs aEnc , such that the output ciphertext act decrypts to amsg using tk on aDec , and msg using ask on Dec . Additionally, aDec might require to identify the existence of non-anamorphic messages. This is captured by a notion called *robustness* (cf. Section A) proposed in [BGH⁺24], which essentially requires aDec to return $\perp$ when decrypting a regular message msg encrypted with Enc , with overwhelming probability. *Anamorphic security* essentially requires that a PPT dictator should not be able to tell apart the output distributions of regular and anamorphic ciphertexts, along with the corresponding key pairs. IND-CPA *security* for PKAE is a security property with respect to the group of peers using the anamorphic channel. Here, an adversary given (apk, dk) must not be able to derive information about amsg from a given ciphertext. Below, we also define several (obvious) security notions beyond IND-CPA.

Definition 13 (Public Key Anamorphic Encryption [CGM24a] [PPY24]) *A public key anamorphic encryption (PKAE) scheme with anamorphic message space $\widehat{\mathcal{M}}$ associated with a public key encryption scheme $\Pi_{\text{PKE}} = (\text{Gen}, \text{Enc}, \text{Dec})$ with a regular message space $\mathcal{M}$ is a triplet $\Pi_{\text{PKAE}} = (\text{aGen}, \text{aEnc}, \text{aDec})$ of PPT algorithms defined as follows:*

$(\text{apk}, \text{ask}, \text{dk}, \text{tk}) \leftarrow \text{aGen}(1^\lambda)$: *The anamorphic key generation algorithm aGen takes as input the security parameter 1^λ , and outputs an anamorphic public key apk , an anamorphic secret key ask , an encryption double key dk and a decryption double key tk .*

$\text{act} \leftarrow \text{aEnc}(\text{apk}, \text{dk}, \text{msg}, \text{amsg})$: *The anamorphic encryption algorithm aEnc takes as input the anamorphic public key apk , the encryption double key dk , a regular message $\text{msg} \in \mathcal{M}$ and an anamorphic message $\text{amsg} \in \widehat{\mathcal{M}}$, and outputs an anamorphic ciphertext act .*

$\text{amsg} \leftarrow \text{aDec}(\text{tk}, \text{act})$: *The anamorphic decryption algorithm aDec takes as input the secret decryption double key tk and the anamorphic ciphertext act , and outputs an anamorphic message amsg or the symbol $\perp \notin \widehat{\mathcal{M}}$ indicating absence of anamorphic message.*

Furthermore, the algorithms must satisfy the following properties.

Correctness: *We call Π_{PKAE} correct if for every pair of messages $(\text{msg}, \text{amsg}) \in \mathcal{M} \times \widehat{\mathcal{M}}$, it holds that*

$$\Pr \left[\text{amsg} \neq \text{aDec}(\text{tk}, \text{act}) : \begin{array}{l} (\text{apk}, \text{ask}, \text{dk}, \text{tk}) \leftarrow \text{aGen}(1^\lambda) \\ \text{act} \leftarrow \text{aEnc}(\text{apk}, \text{dk}, \text{msg}, \text{amsg}) \end{array} \right] \leq \text{negl}(\lambda)$$

where the probability is taken over the random coins tosses of aGen and aEnc .

Anamorphic security: Π_{PKAE} satisfies anamorphic security if for every PPT dictator $\mathcal{D}$ it holds that

$$|\Pr[\text{GReal}_{\text{PKAE}}^{\mathcal{D}}(\lambda) = 1] - \Pr[\text{GAnam}_{\text{PKAE}}^{\mathcal{D}}(\lambda) = 1]| \leq \text{negl}(\lambda)$$

where the security games are defined as follows:

<u>$\text{GReal}_{\text{PKAE}}^{\mathcal{D}}(\lambda) :$</u>	<u>$\text{GAnam}_{\text{PKAE}}^{\mathcal{D}}(\lambda) :$</u>
1. $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$	1. $(\text{apk}, \text{ask}, \text{dk}, \text{tk}) \leftarrow \text{aGen}(1^\lambda)$
2. return $\mathcal{D}^{\text{OEnc}(\cdot, \cdot)}(\text{pk}, \text{sk})$ where	2. return $\mathcal{D}^{\text{OaEnc}(\cdot, \cdot)}(\text{apk}, \text{ask})$ where
$\text{OEnc}(\text{msg}, \text{amsg}) = \text{Enc}(\text{pk}, \text{msg})$	$\text{OaEnc}(\text{msg}, \text{amsg}) =$
	$\text{aEnc}(\text{apk}, \text{dk}, \text{msg}, \text{amsg})$

Fully asymmetric IND-CPA security: Π_{PKAE} satisfies fully asymmetric IND-CPA security if for every PPT adversary $\mathcal{A}$ there exists a negligible function $\text{negl}(\lambda)$ such that for all $\lambda \in \mathbb{N}$, the following holds:

$$|\Pr[\text{GIND-CPA}_{\text{PKAE}}^{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\text{GIND-CPA}_{\text{PKAE}}^{\mathcal{A}}(\lambda, 1) = 1]| \leq \text{negl}(\lambda)$$

where the security game is defined as follows:

<u>$\text{GIND-CPA}_{\text{PKAE}}^{\mathcal{A}}(\lambda, \beta) :$</u>
1. $(\text{apk}, \text{ask}, \text{dk}, \text{tk}) \leftarrow \text{aGen}(1^\lambda)$
2. $(\text{msg}_0, \text{msg}_1, \text{amsg}_0, \text{amsg}_1) \leftarrow \mathcal{A}(\text{apk}, \text{dk})$
3. $\text{act}_\beta \leftarrow \text{aEnc}(\text{apk}, \text{dk}, \text{msg}_\beta, \text{amsg}_\beta)$
4. return $\mathcal{A}(\text{act}_\beta)$

Additionally, we consider new but obvious security notions for Π_{PKAE} ³.

Strong IND-CPA security and beyond: Π_{PKAE} satisfies strong IND-CPA (sIND-CPA) security if for every PPT adversary $\mathcal{A}$ there exists a negligible function $\text{negl}(\lambda)$ such that for all $\lambda \in \mathbb{N}$, the following holds:

$$|\Pr[\text{GsIND-CPA}_{\text{PKAE}}^{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\text{GsIND-CPA}_{\text{PKAE}}^{\mathcal{A}}(\lambda, 1) = 1]| \leq \text{negl}(\lambda)$$

where the security game is defined as follows:

³Similar security notions have been concurrently and independently proposed by Choi, Collins, Liu and Zikas in [CCLZ25], however without any concrete instantiations.

GsIND-CPA_{PKAE}^A(λ, β) :

1. $(\text{apk}, \text{ask}, \text{dk}, \text{tk}) \leftarrow \text{aGen}(1^\lambda)$
2. $(\text{msg}, \text{amsg}_0, \text{amsg}_1) \leftarrow \mathcal{A}(\text{apk}, \text{ask}, \text{dk})$
3. $\text{act}_\beta \leftarrow \text{aEnc}(\text{apk}, \text{dk}, \text{msg}, \text{amsg}_\beta)$
4. **return** $\mathcal{A}(\text{act}_\beta)$

We call Π_{PKAE} strong IND-CCA secure if the advantage of $\mathcal{A}$ in GIND-CPA_{PKAE}^A is negligible when it is additionally given access to an $\text{aDec}(\text{tk}, \cdot)$ -oracle. To guarantee IND-CCA security with respect to the dictator who knows ask , we additionally provide ask to $\mathcal{A}$ as input and call the corresponding security notion strong IND-CCA secure (sIND-CCA). Since, unfortunately, our constructions do not achieve sIND-CCA security, we define a relaxation in the spirit of Replayable CCA (RCCA) security for PKE as introduced in [CKN03]: RCCA captures a security notion which is identical to CCA except for the added ability of the attacker to generate new ciphertexts that decrypt to the same plaintexts as previously seen ciphertexts. More precisely, our sIND-RCCA game coincides with the sIND-CCA game, except that the $\text{aDec}(\text{tk}, \cdot)$ -oracle rejects for a given act if $\text{aDec}(\text{tk}, \text{act})$ results in amsg_0 or amsg_1 .

sIND-RCCA security: Π_{PKAE} satisfies sIND-RCCA security if for every PPT adversary $\mathcal{A}$ there exists a negligible function $\text{negl}(\lambda)$ such that for all $\lambda \in \mathbb{N}$, the following holds:

$$|\Pr[\text{GsIND-RCCA}_{\text{PKAE}}^{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\text{GsIND-RCCA}_{\text{PKAE}}^{\mathcal{A}}(\lambda, 1) = 1]| \leq \text{negl}(\lambda)$$

where the security game is defined as follows:

GsIND-RCCA_{PKAE}^A(λ, β) :

1. $(\text{apk}, \text{ask}, \text{dk}, \text{tk}) \leftarrow \text{aGen}(1^\lambda)$
2. $(\text{msg}, \text{amsg}_0, \text{amsg}_1) \leftarrow \mathcal{A}^{\text{OaDec}(\cdot)}(\text{apk}, \text{ask}, \text{dk})$
3. $\text{act}_\beta \leftarrow \text{aEnc}(\text{apk}, \text{dk}, \text{msg}, \text{amsg}_\beta)$
4. **return** $\mathcal{A}^{\text{OaDec}(\cdot)}(\text{act}_\beta)$

where $\text{OaDec}(\text{act}) = \text{aDec}(\text{tk}, \text{act})$ and

$\text{OaDec}(\text{act})$ outputs $\perp$ if $\text{aDec}(\text{tk}, \text{act}) = \text{amsg}_0$ or amsg_1 .

5 μ -Message Public Key Anamorphic Encryption Extension

In this section, we present concrete realizations of PKAE based on the well-known ElGamal [ElG86] and the Dual-Regev [GPV08] cryptosystems as well as the Fujisaki-Okamoto transformation [FO99, FO13].

Recall our previous discussion on embedding amsg within μ ciphertexts of the underlying PKE, which we now formally regard through the following definition.

Definition 14 (μ -Message Public Key Anamorphic Encryption Extension) For $\mu \in \mathbb{N}$, a public key anamorphic encryption extension (μ -PKAEE) scheme with anamorphic message space $\widehat{\mathcal{M}}$ associated with a public key encryption scheme $\Pi_{\text{PKE}} = (\text{Gen}, \text{Enc}, \text{Dec})$ with a regular message space $\mathcal{M}$ is a triplet $\Pi_{\text{PKAE}} = (\text{aGen}, \text{aEnc}, \text{aDec})$ of PPT algorithms defined as follows:

$(\text{apk}, \text{ask}, \text{dk}, \text{tk}) \leftarrow \text{aGen}(1^\lambda)$: The anamorphic key generation algorithm aGen takes as input the security parameter 1^λ , and outputs the anamorphic public key apk , anamorphic secret key ask , an encryption double key dk and a decryption double key tk .

$\text{act} \leftarrow \text{aEnc}(\text{apk}, \text{dk}, \text{msg}, \text{amsg})$: The anamorphic encryption algorithm aEnc takes as input the anamorphic public key apk , the encryption double key dk , a regular message vector $\text{msg} \in \mathcal{M}^\mu$, and an anamorphic message $\text{amsg} \in \widehat{\mathcal{M}}$, and outputs an anamorphic ciphertext vector act .

$\text{amsg} \leftarrow \text{aDec}(\text{tk}, \text{act})$: The anamorphic decryption algorithm aDec takes as input the decryption double key tk and the anamorphic ciphertext act , and outputs an anamorphic message amsg or the symbol $\perp \notin \widehat{\mathcal{M}}$ indicating absence of an anamorphic message.

We now introduce the PKAE-compatible notion for a PKE. At a high-level, the notion encompasses two properties. The first property *on-the-fly double key generation* captures the essence of anamorphic extension since the double keys dk and tk can simply be derived from regular key-pairs already in use. More precisely, for key pairs (pk, sk) and (pk', sk') generated by PKE.Gen , we set sk' as tk , while dk is set as a function of the key pairs. The second property indicates that aDec is essentially the same as PKE.Dec under the existence of a publicly known poly-time function F . In particular, $F(\text{act})$ produces a valid ciphertext that decrypts to amsg using PKE.Dec . We call this property *ciphertext extractibility*.

Definition 15 (PKAE-Compatible) A public key encryption scheme $\Pi_{\text{PKE}} = (\text{Gen}, \text{Enc}, \text{Dec})$ is said to be PKAE-compatible for the triplet $\Pi_{\text{PKAE}} = (\text{aGen}, \text{aEnc}, \text{aDec})$ if the following holds:

- *On-the-fly double key generation:* A part of output (dk, tk) of $\text{PKAE.aGen}(1^\lambda)$ is the output of $\text{PKE.Gen}(1^\lambda)$. In particular, if $\text{PKAE.aGen}(1^\lambda) = (\text{ask}, \text{apk}, \text{dk}, \text{tk})$ and $\text{PKE.Gen}(1^\lambda) = (\text{pk}, \text{sk})$, $\text{PKE.Gen}(1^\lambda) = (\text{pk}', \text{sk}')$ then $\text{apk} = \text{pk}$, $\text{ask} = \text{sk}$, $\text{tk} = \text{sk}'$ and there exists an efficiently computable function G such that $\text{dk} = G(\text{pk}, \text{sk}, \text{pk}', \text{sk}')$.
- *Ciphertext extractibility:* There exist an efficiently computable function F such that $\text{PKE.Dec}(\text{tk}, F(\text{act})) = \text{amsg}$, where $\text{act} = \text{PKAE.aEnc}(\text{apk}, \text{dk}, \text{msg}, \text{amsg})$.

Throughout the rest of this section, we present PKAE constructions for the ElGamal and Dual-Regev encryption schemes which follow this notion. The recently proposed simple public key anamorphic construction based on CS-Lite [CGM24a], for instance, however, does not satisfy the PKAE-compatible notion.

5.1 Extension of the ElGamal Encryption Scheme

We now show a concrete PKAE construction based on the well-known ElGamal encryption scheme [EIG86]. The ElGamal PKE scheme is a triplet of PPT algorithms $\Pi_{\text{El}} = (\text{Gen}, \text{Enc}, \text{Dec})$ defined over a cyclic group $\mathbb{G} = \langle g \rangle$ of order q . The key generation algorithm El.Gen on input 1^λ , samples a

uniformly random x in $\mathbb{Z}_q$, and returns $(\text{sk} := x, \text{pk} := g^x)$. To encrypt a message $\text{msg} \in \mathcal{M}$, where $\mathcal{M}$ is the group $\mathbb{G}$ itself, the encryption algorithm El.Enc selects a uniformly random κ in $\mathbb{Z}_q$, and computes the ciphertext ct as $(\text{ct}_1, \text{ct}_2) = (g^\kappa, \text{pk}^\kappa \cdot \text{msg})$. The decryption algorithm El.Dec simply recovers msg by computing $\text{ct}_2 \cdot (\text{ct}_1)^{-\text{sk}}$. We can also consider ElGamal for $\mathcal{M}$ being a polynomially-bounded subspace of $\mathbb{Z}_q$, then msg can be recovered by computing the DL of $\text{ct}_2 \cdot (\text{ct}_1)^{-\text{sk}}$. It is well-known that ElGamal is IND-CPA secure under the DDH assumption in $\mathbb{G}$.

Recall from our earlier discussions (Section 2), that our goal is to embed amsg within μ ciphertexts. With the ElGamal encryption scheme, we achieve this goal using $\mu = 2$ ciphertexts. More specifically, we design an ElGamal 2-message public key anamorphic encryption extension Π_{aEl} such that it is indistinguishable from an ElGamal 2-message encryption.

ElGamal 2-Message Encryption Scheme. The ElGamal 2-message encryption scheme $\Pi_{2\text{El}} = (\text{Gen}, \text{Enc}, \text{Dec})$, associated with $\Pi_{\text{El}} = (\text{Gen}, \text{Enc}, \text{Dec})$ does the obvious: $\Pi_{2\text{El}}$ accepts a message vector consisting of two messages encrypts them separately using El.Enc and decrypts the corresponding ciphertext vector by applying El.Dec twice. Obviously, $\Pi_{2\text{El}}$ is also IND-CPA secure under DDH.

ElGamal 2-Message PKAE. We begin with an interesting observation that the ciphertext $\text{ct} = (\text{ct}_1, \text{ct}_2)$ output by 2El.Enc precisely contains two uniformly random group elements κ_1, κ_2 (first components of ct_1 and ct_2 respectively). If we could bias these random choices in a way that it covertly embeds amsg , then we would be able to recover amsg by treating κ_1 and κ_2 as regular ElGamal ciphertext components. Additionally, we need to ensure that the biased ciphertext components are not “much” different from a random output of the regular ElGamal encryption scheme, as otherwise the dictator would be able to tell apart regular and anamorphic instances. We now give more details.

The ElGamal 2-message public key anamorphic encryption extension $\Pi_{\text{aEl}} = (\text{aGen}, \text{aEnc}, \text{aDec})$ works as follows: the key generation algorithm aEl.aGen sets $(\text{apk}, \text{ask}) = (\text{pk}, \text{sk})$ as in El.Gen . In addition, for a uniformly random $\alpha \in \mathbb{Z}_q$, it sets the double keys as $\text{tk} = \alpha$ and $\text{dk} = (g^\alpha, g^{\alpha x})$. On input a regular message vector $\text{msg} = (\text{msg}_1, \text{msg}_2) \in \mathcal{M}^2$, and an anamorphic message $\text{amsg} \in \widehat{\mathcal{M}}$, where $\widehat{\mathcal{M}}$ is a polynomially bounded subspace of $\mathbb{Z}_q$, the encryption algorithm aEl.aEnc computes $\text{ct}_1 = (r_1, c_1)$ using a uniformly random κ on msg_1 , identical to 2El.Enc . However, for computing the second ciphertext ct_2 , instead of choosing a uniformly random element κ_2 in $\mathbb{Z}_q$, the algorithm implicitly sets $\kappa_2 = \alpha \cdot \kappa + \text{amsg}$ and obtains $r_2 := g^{\alpha\kappa + \text{amsg}}$ and $c_2 := g^{x(\alpha\kappa + \text{amsg})} \cdot \text{msg}_2$. The decryption algorithm aEl.aDec recovers amsg by simply returning the discrete logarithm of $r_2 \cdot (r_1)^{-\text{tk}}$, taking advantage of the polynomial message space $\widehat{\mathcal{M}}$. Recall that the dictator has no information on the key pair (tk, dk) , and hence is oblivious to the presence of amsg . We present our construction in Figure 1.

Correctness, Compatibility and Security Analysis. We provide proofs of correctness, compatibility, anamorphic security and (s)IND-CPA security of Π_{aEl} below.

Correctness: We now show the correctness of $\Pi_{\text{aEl}} = (\text{aGen}, \text{aEnc}, \text{aDec})$ based on the ElGamal PKE scheme. Let $\text{act} = (\text{act}_1, \text{act}_2)$ be the output of $\text{aEl.aEnc}(\text{apk}, \text{dk}, \text{msg}, \text{amsg})$, where $\text{act}_1 = (r_1, c_1)$ and $\text{act}_2 = (r_2, c_2)$ are generated using randomness κ and $\text{dk}_1^\kappa \cdot g^{\text{amsg}}$. The double encryption key $\text{dk} = (\text{dk}_1, \text{dk}_2) = (g^\alpha, g^{\alpha x})$. Therefore, we have $r_1 = g^\kappa$ and $r_2 = g^{\alpha\kappa + \text{amsg}}$. The anamorphic decryption algorithm knows $\text{tk} = \alpha$, the double decryption key corresponding to dk . Thus, it can compute $g^{\text{amsg}} = r_2 \cdot (r_1)^{-\text{tk}}$. Since $\widehat{\mathcal{M}}$ is polynomially bounded, the decryption algorithm recovers

<u>aGen(1^λ) :</u> 1. $(\text{sk} = x, \text{pk} = y) \leftarrow \text{El.Gen}(1^\lambda)$ 2. $\alpha \leftarrow \mathbb{Z}_q$ 3. $\text{ask} := \text{sk}, \text{apk} := \text{pk}$ 4. $\text{dk} := (g^\alpha, g^{\alpha x}), \text{tk} := \alpha$ 5. return $(\text{ask}, \text{apk}, \text{dk}, \text{tk})$ <u>aDec(tk, act) :</u> 1. $g^{\text{amsg}} := \text{EL.Dec}(\text{tk}, F(\text{act}))$ 2. return $\log g^{\text{amsg}}$	<u>aEnc($\text{apk}, \text{dk}, \text{msg}, \text{amsg}$) :</u> 1. Parse $\text{dk} = (\text{dk}_1, \text{dk}_2)$ 2. Parse $\text{msg} = (\text{msg}_1, \text{msg}_2)$ 3. $\kappa \leftarrow \mathbb{Z}_q$ 4. $\text{act}_1 := \text{El.Enc}(\text{apk}, \text{msg}_1; \kappa)$ $// \text{act}_1 = (r_1, c_1)$ $// r_1 = g^\kappa, c_1 := \text{apk}^\kappa \cdot \text{msg}_1$ 5. $\text{act}_2 := \text{El.Enc}(\text{apk}, \text{msg}_2; \kappa\alpha + \text{amsg})$ $// \text{act}_2 = (r_2, c_2)$ $// r_2 = \text{dk}_1^\kappa \cdot g^{\text{amsg}}$ $// c_2 = \text{dk}_2^\kappa \cdot \text{apk}^{\text{amsg}} \cdot \text{msg}_2$ 6. return $\text{act} := (\text{act}_1, \text{act}_2)$ <u>F(act) :</u> 1. Parse $\text{act} = (\text{act}_1, \text{act}_2)$ 2. Parse $\text{act}_1 = (r_1, c_1), \text{act}_2 = (r_2, c_2)$ 3. return (r_1, r_2)
---	---

Figure 1: 2-PKAE Π_{aEl}

$\text{amsg} \in \widehat{\mathcal{M}}$ by computing the discrete logarithm of g^{amsg} , which is a poly-time algorithm.

Lemma 2 *ElGamal is PKAE-compatible for Π_{aEl} described in Figure 1 associated with the ElGamal 2-message encryption scheme $\Pi_{2\text{El}}$.*

Proof. First, we observe that for a given key pair (pk, sk) of a regular ElGamal PKE scheme, we can set tk as a uniformly random element α in $\mathbb{Z}_q$, and $\text{dk} = (g^{\text{tk}}, \text{pk}^{\text{tk}})$. Thus, it is easy to see that the double key pair (dk, tk) can be efficiently computed using only apk . Hence, the PKAE satisfies the *on-the-fly key generation* property. Next, to see that Π_{aEl} satisfies the *ciphertext extractability* property is pretty straightforward. The function F defined in Figure 1 extracts the first components $r_1 = g^\kappa$ and $r_2 = g^{\alpha\kappa + \text{amsg}}$ from $\text{act}_1 = (r_1, c_1), \text{act}_2 = (r_2, c_2)$. Then, applying the regular ElGamal decryption algorithm EL.Dec with tk would simply recover amsg by computing $g^{\text{amsg}} = r_2 \cdot (r_1)^{-\text{tk}}$, which is poly-time as the anamorphic message space is poly-bounded. $\square$

Theorem 1 *Under the DDH assumption for $\mathbb{G}$, the ElGamal 2-message public key anamorphic encryption extension Π_{aEl} described in Figure 1 satisfies anamorphic, fully asymmetric IND-CPA security and sIND-CPA security.*

Proof. We prove the theorem in three parts.

Anamorphic security. We use a sequence of hybrid games $\text{Game } 0, \text{Game } (1, j)$ for $j \in [0, Q]$ to prove this theorem. In each hybrid game, we assume $\mathcal{D}$ is a PPT dictator and Q be the total number of queries made by $\mathcal{D}$ to the encryption oracle.

Game 0: This is the anamorphic $\text{GAnam}_{\text{aEl}}^{\mathcal{D}}(\lambda)$. The challenger generates $(\text{ask} = x, \text{apk} = g^x, \text{tk}, \text{dk}) \leftarrow \text{aEl.aGen}(1^\lambda)$, and sends the key pair (apk, ask) to $\mathcal{D}$. Then, $\mathcal{D}$ makes polynomial number of queries to the encryption oracle with a pair of messages $\{(\text{msg}_i, \text{amsg}_i)\}_{i \in [Q]}$. At the i -th query, $\mathcal{D}$ receives a ciphertext act_i computed as follows:

1. Parse $\text{msg}_i = (\text{msg}_{i_1}, \text{msg}_{i_2})$
2. $\alpha, \kappa_1 \leftarrow \mathbb{Z}_q, \kappa_2 := \alpha \cdot \kappa_1 + \text{amsg}_i$.
3. $\text{act}_{i_1} := \text{El.Enc}(\text{apk}, \text{msg}_{i_1}; \kappa_1) = (g^{\kappa_1}, g^{x\kappa_1} \cdot \text{msg}_{i_1})$
4. $\text{act}_{i_2} := \text{El.Enc}(\text{apk}, \text{msg}_{i_2}; \kappa_2) = (g^{\kappa_2}, g^{x\kappa_2} \cdot \text{msg}_{i_2})$
5. **return** $\text{act}_i := (\text{act}_{i_1}, \text{act}_{i_2})$

Game $(1, j)$ for $j \in [0, Q]$: We define $\text{Game } (1, 0)$ to be identical to $\text{Game } 0$, and $\text{Game } (1, j)$ is exactly the same as $\text{Game } (1, 0)$ except that the first j (anamorphic) ciphertexts $\{\text{act}_i\}_{i \in [1, j]}$ are computed as follows:

1. Parse $\text{msg}_i = (\text{msg}_{i_1}, \text{msg}_{i_2})$ where $i \leq j$
2. $\kappa_1, \kappa_2 \leftarrow \mathbb{Z}_q$
3. $\text{act}_{i_1} := \text{El.Enc}(\text{apk}, \text{msg}_{i_1}; \kappa_1) = (g^{\kappa_1}, g^{x\kappa_1} \cdot \text{msg}_{i_1})$
4. $\text{act}_{i_2} := \text{El.Enc}(\text{apk}, \text{msg}_{i_2}; \kappa_2) = (g^{\kappa_2}, g^{x\kappa_2} \cdot \text{msg}_{i_2})$
5. **return** $\text{act}_i := (\text{act}_{i_1}, \text{act}_{i_2})$

We show that $\text{Game } (1, j)$ is indistinguishable from $\text{Game } (1, j - 1)$ by the DDH assumption. The only difference between $\text{Game } (1, j)$ and $\text{Game } (1, j - 1)$ is in the computation of κ_2 . In particular, κ_2 is set as $(\alpha \cdot \kappa_1 + \text{amsg}_i)$ in $\text{Game } (1, j - 1)$, whereas κ_2 is sampled uniformly random from $\mathbb{Z}_q$ in $\text{Game } (1, j)$. Thus the challenger can simulate $\text{Game } (1, j)$ with $(g^\alpha, g^{\kappa_1}, g^t)$ when $t = \alpha \cdot \kappa_1$, and $\text{Game } (1, j)$, when t is uniformly random in $\mathbb{Z}_q$. Then by the DDH assumption in $\mathbb{G}$, the following holds:

$$\Pr[\alpha, \kappa_1 \leftarrow \mathbb{Z}_q : \mathcal{A}(g^\alpha, g^{\kappa_1}, g^{\alpha \cdot \kappa_1}) = 1] - \Pr[\alpha, \kappa_1, t \leftarrow \mathbb{Z}_q : \mathcal{A}(g^\alpha, g^{\kappa_1}, g^t) = 1] \leq \text{negl}(\lambda)$$

Since $t = \alpha \cdot \kappa_1$ is indistinguishable from uniformly random, $\kappa_2 = t + \text{amsg}_i$ is statistically close to t . In other words, the hybrids $\text{Game } (1, j)$ and $\text{Game } (1, j - 1)$ are indistinguishable.

Finally, we observe that $\text{Game } (1, Q)$ is exactly the same as $\text{GReal}_{\text{aEl}}^{\mathcal{D}}(\lambda)$. This completes the proof of anamorphic security.

Fully asymmetric IND-CPA security. We prove the security using hybrid games $\text{Game } 0, \dots, \text{Game } 3$. In each hybrid game, we assume $\mathcal{A}$ is a PPT adversary.

Game 0: This is the IND-CPA security experiment $\text{GIND-CPA}_{\text{PKAE}}^{\mathcal{A}}(\lambda, 0)$ defined as follows:

- 1: $x, \alpha \leftarrow \mathbb{Z}_q$
- 2: $\text{ask} := x, \text{apk} := g^x, \text{dk} = (g^\alpha, g^{\alpha x})$
- 3: $(\text{msg}_0, \text{msg}_1, \text{amsg}_0, \text{amsg}_1) \leftarrow \mathcal{A}(\text{apk}, \text{dk})$
- 4: Parse $\text{msg}_0 = (\text{msg}_{0,1}, \text{msg}_{0,2})$
- 5: Parse $\text{msg}_1 = (\text{msg}_{1,1}, \text{msg}_{1,2})$
- 6: Parse $\text{dk} = (\text{dk}_1, \text{dk}_2)$
- 7: $\kappa_1 \leftarrow \mathbb{Z}_q$
- 8: Set $\kappa_2 := \alpha \kappa_1 + \text{amsg}_0$
- 9: $\text{act}_1^* := \text{El.Enc}(\text{apk}, \text{msg}_{0,1}; \kappa_1) = (r_1 := g^{\kappa_1}, c_1 := \text{apk}^{\kappa_1} \cdot \text{msg}_{0,1})$
- 10: $\text{act}_2^* := \text{El.Enc}(\text{apk}, \text{msg}_{0,2}; \kappa_2)$
 $= (r_2 := \text{dk}_1^{\kappa_1} \cdot g^{\text{amsg}_0}, c_2 := \text{dk}_2^{\kappa_1} \cdot \text{apk}^{\text{amsg}_0} \cdot \text{msg}_{0,2})$
- 11: **return** $\beta' \leftarrow \mathcal{A}(\text{act}^* = (\text{act}_1^*, \text{act}_2^*))$

Game 1: This is the same as **Game 0** except that the challenger samples κ_2 uniformly at random from $\mathbb{Z}_q$ instead of setting it as $\kappa_2 = \alpha \kappa_1 + \text{amsg}_0$. The indistinguishability follows the DDH assumption. More formally, we design a simulator $\mathcal{B}$ that given a DDH instance $(g^{\kappa_1}, g^\alpha, g^t)$ in the group $\mathbb{G}$, interacts with $\mathcal{A}$ to solve whether $t = \alpha \kappa_1$ or t is uniformly random in $\mathbb{Z}_q$.

Simulator $\mathcal{B}(1^\lambda, g^{\kappa_1}, g^\alpha, g^t)$

- 1: $x \leftarrow \mathbb{Z}_q$
- 2: $\text{ask} := x, \text{apk} := g^x, \text{dk} = (g^\alpha, g^{\alpha x})$
- 3: $(\text{msg}_0, \text{msg}_1, \text{amsg}_0, \text{amsg}_1) \leftarrow \mathcal{A}(\text{apk}, \text{dk})$
- 4: Parse $\text{msg}_0 = (\text{msg}_{0,1}, \text{msg}_{0,2})$
- 5: Parse $\text{msg}_1 = (\text{msg}_{1,1}, \text{msg}_{1,2})$
- 6: Parse $\text{dk} = (\text{dk}_1, \text{dk}_2)$
- 7: $\text{act}_1^* := \text{El.Enc}(\text{apk}, \text{msg}_{0,1}; \kappa_1) = (r_1 := g^{\kappa_1}, c_1 := (g^{\kappa_1})^x \cdot \text{msg}_{0,1})$
- 8: $\text{act}_2^* := \text{El.Enc}(\text{apk}, \text{msg}_{0,2}; t + \text{amsg}_0)$
 $= (r_2 := g^{t+\text{amsg}_0}, c_2 := (g^t)^x \cdot \text{apk}^{\text{amsg}_0} \cdot \text{msg}_{0,2})$
- 9: **return** $\beta' \leftarrow \mathcal{A}(\text{act}^* = (\text{act}_1^*, \text{act}_2^*))$

If $t = \alpha \kappa_1$, then $\mathcal{B}$ simulates the experiment $\text{GsIND-CPA}_{\text{PKAE}}^{\mathcal{A}}(\lambda, 0)$. This is because even though x is chosen randomly during simulation, (apk, dk) is a valid pair, and hence act^* is a valid encryption of amsg_0 . If $t \leftarrow \mathbb{Z}_q$, then $\kappa_2 = t + \text{amsg}_0$ is uniformly distributed in $\mathbb{G}$, and hence $\mathcal{B}$ simulated **Game 1** in $\mathcal{A}$'s view. Thus, **Game 1** is indistinguishable from **Game 0** by the DDH assumption.

Game 2: This is the same as Game 1 except that the challenger encrypts $\mathbf{msg}_1$ instead of $\mathbf{msg}_0$:

- 1: $x, \alpha \leftarrow \mathbb{Z}_q$
- 2: $\text{ask} := x, \text{apk} := g^x, \text{dk} = (g^\alpha, g^{\alpha x})$
- 3: $(\mathbf{msg}_0, \mathbf{msg}_1, \text{amsg}_0, \text{amsg}_1) \leftarrow \mathcal{A}(\text{apk}, \text{dk})$
- 4: Parse $\mathbf{msg}_0 = (\text{msg}_{0,1}, \text{msg}_{0,2})$
- 5: Parse $\mathbf{msg}_1 = (\text{msg}_{1,1}, \text{msg}_{1,2})$
- 6: Parse $\text{dk} = (\text{dk}_1, \text{dk}_2)$
- 7: $\kappa_1, \kappa_2 \leftarrow \mathbb{Z}_q$
- 8: $\text{act}_1^* := \text{El.Enc}(\text{apk}, \text{msg}_{1,1}; \kappa_1) = (r_1 := g^{\kappa_1}, c_1 := \text{apk}^{\kappa_1} \cdot \text{msg}_{1,1})$
- 9: $\text{act}_2^* := \text{El.Enc}(\text{apk}, \text{msg}_{1,2}; \kappa_2) = (r_2 := g^{\kappa_2}, c_2 := \text{apk}^{\kappa_2} \cdot \text{msg}_{1,2})$
- 10: **return** $\beta' \leftarrow \mathcal{A}(\text{act}^* = (\text{act}_1^*, \text{act}_2^*))$

The indistinguishability between Game 1 and Game 2 follows the underlying ElGamal scheme, in particular from the DDH assumption.

Game 3: This is the same as Game 2 except that the challenger sets $\kappa_2 = \alpha\kappa_1 + \text{amsg}_1$ instead of sampling it uniformly at random from $\mathbb{Z}_q$.

- 1: $x, \alpha \leftarrow \mathbb{Z}_q$
- 2: $\text{ask} := x, \text{apk} := g^x, \text{dk} = (g^\alpha, g^{\alpha x})$
- 3: $(\mathbf{msg}_0, \mathbf{msg}_1, \text{amsg}_0, \text{amsg}_1) \leftarrow \mathcal{A}(\text{apk}, \text{dk})$
- 4: Parse $\mathbf{msg}_0 = (\text{msg}_{0,1}, \text{msg}_{0,2})$
- 5: Parse $\mathbf{msg}_1 = (\text{msg}_{1,1}, \text{msg}_{1,2})$
- 6: Parse $\text{dk} = (\text{dk}_1, \text{dk}_2)$
- 7: $\kappa_1 \leftarrow \mathbb{Z}_q$
- 8: Set $\kappa_2 := \alpha\kappa_1 + \text{amsg}_1$
- 9: $\text{act}_1^* := \text{El.Enc}(\text{apk}, \text{msg}_{1,1}; \kappa_1) = (r_1 := g^{\kappa_1}, c_1 := \text{apk}^{\kappa_1} \cdot \text{msg}_{1,1})$
- 10: $\text{act}_2^* := \text{El.Enc}(\text{apk}, \text{msg}_{1,2}; \kappa_2)$
 $= (r_2 := \text{dk}_1^{\kappa_1} \cdot g^{\text{amsg}_1}, c_2 := \text{dk}_2^{\kappa_1} \cdot \text{apk}^{\text{amsg}_1} \cdot \text{msg}_{1,2})$
- 11: **return** $\beta' \leftarrow \mathcal{A}(\text{act}^* = (\text{act}_1^*, \text{act}_2^*))$

The indistinguishability follows the DDH assumption as in Game 1.

Finally, we observe that Game 3 is exactly the same as $\text{GIND-CPA}_{\text{PKAE}}^A(\lambda, 1)$. This concludes the fully asymmetric security of Π_{aEl} .

sIND-CPA security. We prove the strong IND-CPA security by designing a simulator that plays the role of the challenger in the security game.

We start with a contradiction that there exists a PPT adversary $\mathcal{A}$ that breaks the sIND-CPA security of Π_{aEl} with a non-negligible probability. We then design a simulator $\mathcal{B}$ that given a DDH instance $(g^\kappa, g^\alpha, g^t)$ in the group $\mathbb{G}$, interacts with $\mathcal{A}$ to solve whether $t = \alpha\kappa$ or t is uniformly random in $\mathbb{Z}_q$.

$\mathcal{B}$ prepares the simulation by sampling a uniformly random x , and setting $\text{ask} = x$, $\text{apk} = g^x$, $\text{dk} = (g^\alpha, g^{\alpha x})$, and sends $(\text{apk}, \text{ask}, \text{dk})$ to $\mathcal{A}$. When $\mathcal{A}$ returns $(\text{msg}, \text{amsg}_0, \text{amsg}_1)$, $\mathcal{B}$ calculates challenge ciphertext $\text{act}^* = \text{aEl.aEnc}(\text{apk}, \text{dk}, \text{msg}, \text{amsg}_\beta)$ for a uniformly random bit $\beta \in \{0, 1\}$, and sends it to $\mathcal{A}$. Finally $\mathcal{A}$ outputs its guess β' , and $\mathcal{B}$ returns $(\beta == \beta')$. The simulation is formally specified as follows:

Simulator $\mathcal{B}(1^\lambda, g^\kappa, g^\alpha, g^t)$

- 1: $x \leftarrow \mathbb{Z}_q$
- 2: $\text{ask} := x, \text{apk} := g^x, \text{dk} = (g^\alpha, g^{\alpha x})$
- 3: $(\text{msg}, \text{amsg}_0, \text{amsg}_1) \leftarrow \mathcal{A}(\text{apk}, \text{ask}, \text{dk})$
- 4: $\beta \leftarrow \{0, 1\}$
- 5: Parse $\text{msg} = (\text{msg}_1, \text{msg}_2)$
- 6: Parse $\text{dk} = (\text{dk}_1, \text{dk}_2)$
- 7: $\text{act}_1^* := \text{El.Enc}(\text{apk}, \text{msg}_1; \kappa) = (r_1 := g^\kappa, c_1 := \text{apk}^\kappa \cdot \text{msg}_1)$
 $= (r_1 := g^\kappa, c_1 := (g^\kappa)^x \cdot \text{msg}_1)$
- 8: $\text{act}_2^* := \text{El.Enc}(\text{apk}, \text{msg}_2; \kappa\alpha + \text{amsg}) = (r_2 := \text{dk}_1^\kappa \cdot g^{\text{amsg}_\beta}, c_2 := \text{dk}_2^\kappa \cdot \text{apk}^{\text{amsg}_\beta} \cdot \text{msg}_2)$
 $= (r_2 := g^{t+\text{amsg}_\beta}, c_2 := (g^t)^x \cdot \text{apk}^{\text{amsg}_\beta} \cdot \text{msg}_2)$
- 9: $\beta' \leftarrow \mathcal{A}(\text{act}^* = (\text{act}_1^*, \text{act}_2^*))$
- 10: **return** $\beta == \beta'$

If $t = \alpha\kappa$, then the distribution of (apk, ask) and the challenge ciphertext act^* corresponds exactly to $\mathcal{A}$'s view as in the experiment $\text{GsIND-CPA}_{\text{PKAE}}^{\mathcal{A}}(\lambda, \beta)$. This is because even though x is chosen randomly during simulation, (apk, ask) is a valid pair, and hence act^* is a valid encryption of amsg_β . If $t \leftarrow \mathbb{Z}_q$, then with all but negligible probability act^* is neither an encryption of amsg_0 nor amsg_1 . In fact, $t + \text{amsg}_\beta$ is uniformly distributed in $\mathbb{G}$, and hence β is hidden from $\mathcal{A}$'s view. Therefore, if $\mathcal{A}$ breaks the sIND-CPA security of Π_{aEl} with a non-negligible probability then $\mathcal{B}$ can also break the DDH assumption with a non-negligible probability. $\square$

5.2 Extension of the Dual-Regev Encryption Scheme

We now present a PKAE construction based on the Dual-Regev cryptosystem by Gentry, Peikert and Vaikuntanathan [GPV08]. We focus on a simple version that encrypts single bits. The Dual-Regev encryption scheme is a triplet of PPT algorithms $\Pi_{\text{DReg}} = (\text{Gen}, \text{Enc}, \text{Dec})$ defined over a security parameter λ as follows: let $n = n(\lambda)$, $q = q(\lambda)$ and $m = m(\lambda)$ be polynomials in λ . Let $\mathbb{T} = \mathbb{R}/\mathbb{Z}$ denote the group of reals $[0, 1)$ with modulo 1 addition. Then for $\alpha \in \mathbb{R}^+$, let Ψ_α be the distribution on $\mathbb{T}$ of a normal variable with mean 0 and standard deviation $\alpha/\sqrt{2\pi}$. Following the notations of [GPV08], we set $m \geq 2n \log q$, $r = \omega(\sqrt{\log m})$, $q \geq 5r(m+1)$, $\alpha \leq 1/(r\sqrt{m+1} \cdot \omega(\sqrt{\log n}))$ and the LWE noise distribution $\chi = \Psi_\alpha$. The key generation algorithm DReg.Gen randomly samples vector $\mathbf{k} \in \{0, 1\}^m$ and a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$. It then returns $\text{sk} := \mathbf{k}$ and $\text{pk} := (\mathbf{A}, \mathbf{b} = \mathbf{A} \cdot \mathbf{k})$. On an input $\text{msg} \in \mathcal{M}$, where $\mathcal{M} = \{0, 1\}$, the encryption algorithm DReg.Enc selects a uniformly random vector $\mathbf{s} \in \mathbb{Z}_q^n$, a vector $\mathbf{e}^T$ sampled randomly from χ^m , and an integer y sampled from χ . It then computes the ciphertext as $(\mathbf{r}^T, c) := (\mathbf{s}^T \cdot \mathbf{A} + \mathbf{e}^T, \mathbf{s}^T \cdot \mathbf{b} + y + \text{msg} \cdot \lfloor \frac{q}{2} \rfloor \bmod q)$. The decryption algorithm DReg.Dec is probabilistic, and outputs 0 if $|c - \mathbf{r}^T \cdot \mathbf{k}| \leq \frac{q}{4}$, and 1 otherwise. Note that, $|c - \mathbf{r}^T \cdot \mathbf{k}|$ simply reduces to $\text{msg} \cdot \lfloor \frac{q}{2} \rfloor$ plus an error term $(y - \mathbf{e}^T \cdot \mathbf{k})$, whose norm is smaller than $\frac{q}{5}$ with an overwhelming probability in λ . Dual-Regev using parameters as described above is IND-CPA secure under the (n, q, χ) -DLWE assumption of dimension m . As in the case of ElGamal, we can similarly define Dual-Regev 2-message encryption scheme $\Pi_{2\text{DReg}} = (\text{Gen}, \text{Enc}, \text{Dec})$ where encryption takes two messages as input.

Dual-Regev 2-Message Encryption Scheme. We consider the Dual-Regev 2-message encryption scheme $\Pi_{2\text{DReg}} = (\text{Gen}, \text{Enc}, \text{Dec})$ which simply applies DReg.Enc on a message vector containing two messages and DReg.Dec on a ciphertext vector containing two DReg.Enc ciphertexts. Clearly, the IND-CPA security of $\Pi_{2\text{DReg}}$ is implied by that of Π_{DReg} .

Dual-Regev 2-Message PKAE. The Dual-Regev 2-message public key anamorphic encryption extension $\Pi_{\text{aDReg}} = (\text{aGen}, \text{aEnc}, \text{aDec})$ works as follows: the key generation aDReg.aGen sets $(\text{apk}, \text{ask}) = (\text{pk}, \text{sk})$ output by DReg.Gen . It then selects a secret vector $\hat{\mathbf{k}}$, and sets $\text{tk} := \hat{\mathbf{k}}$, $\text{dk} := (\mathbf{A}, \hat{\mathbf{b}} = \mathbf{A} \cdot \hat{\mathbf{k}})$. To encrypt a regular message vector $\text{msg} = (\text{msg}_1, \text{msg}_2) \in \mathcal{M}^2$, along with an anamorphic message $\text{amsg} \in \hat{\mathcal{M}}$, where $\mathcal{M} = \hat{\mathcal{M}} = \{0, 1\}$, the anamorphic encryption algorithm aDReg.aEnc samples a uniformly random $\mathbf{s}_1$ in $\mathbb{Z}_q^n$, $\mathbf{e}_1, \mathbf{e}_2$ from χ^m and $\hat{y}$ from χ . It then computes $\text{act}_1 = (\mathbf{r}_1^T, c_1)$ as $\text{DReg.Enc}(\text{apk}, \text{msg}_1; \mathbf{s}_1, \mathbf{e}_1^T)$. To encode amsg in $\mathbf{r}_2^T$ (first component of act_2), it first computes a non-zero solution to $\mathbf{a}_1^T \cdot \mathbf{x} = \mathbf{s}_1^T \cdot \hat{\mathbf{b}} + \hat{y} + \text{amsg} \cdot \lfloor \frac{q}{2} \rfloor - e_{2,1} \bmod q$, where $\mathbf{A} = [\mathbf{a}_1, \dots, \mathbf{a}_m]$ and $\mathbf{e}_2^T = [e_{2,1}, \dots, e_{2,m}]$, and sets $\mathbf{s}_2 = \mathbf{x}$. We know that such a solution exists (by the rank-nullity theorem). Then, $\text{act}_2 = (\mathbf{r}_2^T, c_2)$ is computed as $\text{DReg.Enc}(\text{apk}, \text{msg}_2; \mathbf{s}_2, \mathbf{e}_2^T)$. Finally, the anamorphic ciphertext act is $(\text{act}_1, \text{act}_2)$. Let $\text{proj} : \mathbb{Z}_q^m \rightarrow \mathbb{Z}_q$ be a projection onto $\mathbb{Z}_q$. To recover amsg , the anamorphic decryption algorithm aDReg.aDec extracts the first element of $\mathbf{r}_2^T$ as $\hat{c} = \text{proj}_1(\mathbf{r}_2^T)$, followed by computing DReg.Dec on $\hat{\mathbf{r}}^T = \mathbf{r}_1^T$ and $\hat{c}$, using tk . We note that amsg is hidden from the dictator's view as it has no information on the double keys. See Figure 2 for our construction.

Correctness, Compatibility and Security Analysis. We provide proofs of correctness, compatibility, anamorphic security and (s)IND-CPA security of Π_{aDReg} below.

<u>aGen(1^λ) :</u> 1. $(\text{sk} := \mathbf{k}, \text{pk} := (\mathbf{A}, \mathbf{b} = \mathbf{A} \cdot \mathbf{k})) \leftarrow \text{DReg.Gen}(1^\lambda)$ 2. $\hat{\mathbf{k}} \leftarrow \{0, 1\}^m$ 3. $\text{ask} := \text{sk}, \text{apk} := \text{pk}$ 4. $\text{tk} := \hat{\mathbf{k}}, \text{dk} := (\mathbf{A}, \hat{\mathbf{b}} = \mathbf{A} \cdot \hat{\mathbf{k}})$ 5. return $(\text{ask}, \text{apk}, \text{tk}, \text{dk})$ <u>aDec(tk, act) :</u> 1. $\text{amsg} := \text{DReg.Dec}(\text{tk}, F(\text{act}))$ 2. return amsg <u>$F(\text{act})$:</u> 1. Parse $\text{act} = (\text{act}_1, \text{act}_2)$ 2. Parse $\text{act}_1 = (\mathbf{r}_1^T, c_1)$, $\text{act}_2 = (\mathbf{r}_2^T, c_2)$ 3. $\mathbf{r}_2^T = [d_1, \dots, d_m]$ 4. $\hat{c} := \text{proj}_1(\mathbf{r}_2^T), \hat{\mathbf{r}}^T := \mathbf{r}_1^T$ 5. return $(\hat{\mathbf{r}}^T, \hat{c})$	<u>aEnc($\text{apk}, \text{dk}, \text{msg}, \text{amsg}$) :</u> 1. Parse $\text{msg} = (\text{msg}_1, \text{msg}_2)$ 2. $\mathbf{s}_1 \leftarrow \mathbb{Z}_q^n, \mathbf{e}_1, \mathbf{e}_2 \leftarrow \chi^m, \hat{y} \leftarrow \chi$ 3. $\mathbf{A} = [\mathbf{a}_1, \dots, \mathbf{a}_m], \mathbf{e}_2^T = [e_{2,1}, \dots, e_{2,m}]$ $\mathbf{a}_1^T = [a_{1,1}, \dots, a_{1,n}]$ 4. for $i \in \{1, \dots, n-1\}$: $x_i \leftarrow \mathbb{Z}_q$ 5. $x_n = \left(-\sum_{i=1}^{n-1} x_i a_{1,i} + \mathbf{s}_1^T \cdot \hat{\mathbf{b}} + \hat{y} + \right.$ $\left. \text{amsg} \cdot \lfloor \frac{q}{2} \rfloor - e_{2,1} \right) \cdot a_{1,n}^{-1} \bmod q$ 6. $\mathbf{x}^T = [x_1, \dots, x_n]$ 7. $\mathbf{s}_2 := \mathbf{x}$ 8. $\text{act}_1 := \text{DReg.Enc}(\text{apk}, \text{msg}_1; \mathbf{s}_1, \mathbf{e}_1^T)$ $// \text{act}_1 = (\mathbf{r}_1^T, c_1)$ $// \mathbf{r}_1^T = \mathbf{s}_1^T \cdot \mathbf{A} + \mathbf{e}_1^T$ $// c_1 = \mathbf{s}_1^T \cdot \mathbf{b} + y_1 + \text{msg}_1 \cdot \lfloor \frac{q}{2} \rfloor \bmod q$ 9. $\text{act}_2 := \text{DReg.Enc}(\text{apk}, \text{msg}_2; \mathbf{s}_2, \mathbf{e}_2^T)$ $// \text{act}_2 = (\mathbf{r}_2^T, c_2)$ $// \mathbf{r}_2^T = \mathbf{s}_2^T \cdot \mathbf{A} + \mathbf{e}_2^T$ $// c_2 = \mathbf{s}_2^T \cdot \mathbf{b} + y_2 + \text{msg}_2 \cdot \lfloor \frac{q}{2} \rfloor \bmod q$ 10. return $\text{act} := (\text{act}_1, \text{act}_2)$
--	--

Figure 2: 2-PKAEE Π_{aDReg}

Correctness: We now show the correctness of the $\Pi_{\text{aDReg}} = (\text{aGen}, \text{aEnc}, \text{aDec})$ based on the Dual-Regev PKE scheme. Let the encryption double key be $\text{dk} = (\text{dk}_1, \text{dk}_2) = (\mathbf{A}, \hat{\mathbf{b}} = \mathbf{A} \cdot \hat{\mathbf{k}})$. Let $\text{act} = (\text{act}_1, \text{act}_2)$ output by $\text{aDReg.aEnc}(\text{apk}, \text{dk}, \text{msg}, \text{amsg})$, where $\text{act}_1 = (\mathbf{r}_1^T, c_1)$ and $\text{act}_2 = (\mathbf{r}_2^T, c_2)$ generated using randomness $\mathbf{s}_1$ and $\mathbf{s}_2$ such that $\mathbf{r}_1^T = \mathbf{s}_1^T \cdot \text{dk}_1 + \mathbf{e}_1^T$ and $\mathbf{r}_2^T = \mathbf{s}_2^T \cdot \text{dk}_1 + \mathbf{e}_2^T$, where $\mathbf{s}_2$ is set as a solution to $\mathbf{a}_1^T \cdot \mathbf{x} = \mathbf{s}_1^T \cdot \text{dk}_2 + \hat{y} + \text{amsg} \cdot \lfloor \frac{q}{2} \rfloor - e_{2,1} \bmod q$, where $\text{dk}_1 = [\mathbf{a}_1, \dots, \mathbf{a}_m] \in \mathbb{Z}_q^{n \times m}$, $\mathbf{e}_2^T = [e_{2,1}, \dots, e_{2,m}] \in \chi^{1 \times m}$ and $\hat{y} \in \chi$. Then, observe that the first element of the vector $\mathbf{r}_2^T$

is $\mathbf{s}_1^T \cdot \mathbf{dk}_2 + \hat{y} + \mathbf{amsg} \cdot \lfloor \frac{q}{2} \rfloor \bmod q$. The anamorphic decryption algorithm knows $\mathbf{tk} = \hat{\mathbf{k}}$, the decryption double key corresponding to $\mathbf{dk}$. It calls the function $F(\mathbf{act})$, which extracts the first element of $\mathbf{r}_2^T$ using a projection function and assigns the value to $\hat{c}$. Thus it can compute $\mathbf{amsg}$ from $\hat{\mathbf{r}}^T = \mathbf{r}_1^T = \mathbf{s}_1^T \cdot \mathbf{dk}_1 + \mathbf{e}_1^T$ and $\hat{c} = \mathbf{s}_1^T \cdot \mathbf{dk}_2 + \hat{y} + \mathbf{amsg} \cdot \lfloor \frac{q}{2} \rfloor \bmod q$, by returning $\mathbf{amsg} = 0$ if $|\hat{c} - \hat{\mathbf{r}}^T \cdot \mathbf{tk}| \leq \frac{q}{4}$, and $\mathbf{amsg} = 1$ otherwise. Note that, $|\hat{c} - \hat{\mathbf{r}}^T \cdot \mathbf{tk}|$ simply reduces to $\mathbf{amsg} \cdot \lfloor \frac{q}{2} \rfloor$ plus an LWE noise $\hat{y} - \mathbf{e}_2^T \cdot \hat{\mathbf{k}} \bmod q$, whose norm is smaller than $\frac{q}{5}$, except with a negligible probability in λ .

Lemma 3 *Dual-Regev is PKAE-compatible for Π_{aDReg} described in Figure 2 associated with the Dual-Regev 2-message encryption scheme Π_{2DReg} .*

Proof. First, observe that $\text{aDReg.aGen}(1^\lambda)$ outputs $\mathbf{ask} = \mathbf{k}$, $\mathbf{apk} = (\mathbf{A}, \mathbf{b} = \mathbf{A} \cdot \mathbf{k})$, $\mathbf{tk} = \hat{\mathbf{k}}$ and $\mathbf{dk} = (\mathbf{A}, \hat{\mathbf{b}} = \mathbf{A} \cdot \hat{\mathbf{k}})$, where $\mathbf{k}$ and $\hat{\mathbf{k}}$ are sampled uniformly at random from $\{0, 1\}^m$. Therefore, the key pair $(\mathbf{apk}, \mathbf{ask})$ has identical distributions with $\{(\mathbf{pk}, \mathbf{sk}) : \mathbf{sk} = \hat{\mathbf{k}}, \mathbf{pk} = (\mathbf{A}, \hat{\mathbf{b}} = \mathbf{A} \cdot \hat{\mathbf{k}}) \text{ s.t. } \hat{\mathbf{k}} \leftarrow \{0, 1\}^m\}$, which is essentially the distribution of secret and public keys of a Dual-Regev PKE scheme. Additionally, it is easy to see that the double key pair $(\mathbf{dk}, \mathbf{tk})$ can be efficiently computed using only $\mathbf{apk}$. Hence, the PKAE satisfies *on-the-fly double key generation* property. Next, observe that the function F defined in Figure 2 takes as input $\mathbf{act} \leftarrow \text{aDReg.aEnc}(\mathbf{apk}, \mathbf{dk}, \mathbf{msg}, \mathbf{amsg})$, and parses it as $\mathbf{act}_1 = (\mathbf{r}_1^T, c_1)$ and $\mathbf{act}_2 = (\mathbf{r}_2^T, c_2)$. It then extracts $\mathbf{r}_1^T$ (call it $\hat{\mathbf{r}}^T$) and the first component of $\mathbf{r}_2^T$ (call it $\hat{c}$). Note that $(\hat{\mathbf{r}}^T, \hat{c})$ is a valid Dual-Regev ciphertext as $\hat{\mathbf{r}}^T = \mathbf{s}_1^T \cdot \mathbf{A} + \mathbf{e}_1^T$ and $\hat{c} = \mathbf{s}_1^T \cdot \hat{\mathbf{b}} + \hat{y} + \mathbf{amsg} \cdot \lfloor \frac{q}{2} \rfloor \bmod q$. Thus, $\mathbf{amsg}$ can be extracted by executing DReg.Dec using $\mathbf{tk} = \hat{\mathbf{k}}$, which implies that the PKAE satisfies the *ciphertext extractability* property. Hence we conclude that the Dual-Regev PKE is PKAE-compatible. $\square$

Theorem 2 *Let us consider $m \geq 2n \log q$, $r = \omega(\sqrt{\log m})$, $q \geq 5r(m+1)$, $\alpha \leq 1/(r\sqrt{m+1} \cdot \omega(\sqrt{\log n}))$ and $\chi = \Psi_\alpha$. Then, under the (n, q, χ) -DLWE assumption of dimension m , the Dual-Regev 2-message anamorphic encryption extension Π_{aDReg} described in Fig. 2 satisfies anamorphic, fully asymmetric IND-CPA security and sIND-CPA security.*

Proof. We prove the theorem in three parts.

Anamorphic security. We use a sequence of hybrid games $\text{Game } 0, \text{Game } (1, j)$ for $j \in [0, Q]$ to prove this theorem. In each hybrid game, we assume $\mathcal{D}$ is a PPT dictator and Q be the total number of queries made by the dictator to the encryption oracle.

Game 0: This is the anamorphic $\text{GAnam}_{\text{aDReg}}^{\mathcal{D}}(\lambda)$. The challenger generates $(\mathbf{apk} = (\mathbf{A}, \mathbf{b} = \mathbf{A} \cdot \mathbf{k}), \mathbf{ask} = \mathbf{k}, \mathbf{dk}, \mathbf{tk}) \leftarrow \text{aDReg.aGen}(1^\lambda)$, and sends the key pair $(\mathbf{apk}, \mathbf{ask})$ to $\mathcal{D}$. Then $\mathcal{D}$ makes polynomial number of queries to the encryption oracle with a pair of messages $\{(\mathbf{msg}_i, \mathbf{amsg}_i)\}_{i \in [Q]}$. At the i -th query, the dictator receives a ciphertext $\mathbf{act}_i$ computed as follows:

1. Parse $\mathbf{msg}_i = (\mathbf{msg}_{i_1}, \mathbf{msg}_{i_2})$
2. $\hat{\mathbf{k}} \leftarrow \{0, 1\}^m$, $\mathbf{s}_1 \leftarrow \mathbb{Z}_q^n$, $\mathbf{e}_1, \mathbf{e}_2 \leftarrow \chi^m$, $y_1, y_2, \hat{y} \leftarrow \chi$
3. $\mathbf{A} = [\mathbf{a}_1, \dots, \mathbf{a}_m] \in \mathbb{Z}_q^{n \times m}$, $\mathbf{e}_2^T = [e_{2,1}, \dots, e_{2,m}] \in \chi^{1 \times m}$
4. $\hat{\mathbf{b}} := \mathbf{A} \cdot \hat{\mathbf{k}}$
5. **for** $i \in \{1, \dots, n-1\}$, $x_i \leftarrow \mathbb{Z}_q$

6. **end for**
7. $x_n = (-\sum_{i=1}^{n-1} x_i a_{1,i} + \mathbf{s}_1^T \cdot \widehat{\mathbf{b}} + \widehat{y} + \text{amsg}_i \cdot \lfloor \frac{q}{2} \rfloor - e_{2,1}) \cdot a_{1,n}^{-1} \mod q$
8. $\mathbf{x}^T = [x_1, \dots, x_n]$
9. $\mathbf{s}_2 := \mathbf{x}$
10. $\text{act}_{i_1} := \text{DReg.Enc}(\text{apk}, \text{msg}_{i_1}; \mathbf{s}_1, \mathbf{e}_1^T)$
 $= (\mathbf{s}_1^T \cdot \mathbf{A} + \mathbf{e}_1^T, \mathbf{s}_1^T \cdot \mathbf{b} + y_1 + \text{msg}_{i_1} \cdot \lfloor \frac{q}{2} \rfloor \mod q)$
11. $\text{act}_{i_2} := \text{DReg.Enc}(\text{apk}, \text{msg}_{i_2}; \mathbf{s}_2, \mathbf{e}_2^T)$
 $= (\mathbf{s}_2^T \cdot \mathbf{A} + \mathbf{e}_2^T, \mathbf{s}_2^T \cdot \mathbf{b} + y_2 + \text{msg}_{i_2} \cdot \lfloor \frac{q}{2} \rfloor \mod q)$
12. **return** $\text{act}_i := (\text{act}_{i_1}, \text{act}_{i_2})$

Game (1, j) for $j \in [0, Q]$: We define **Game (1, 0)** to be identical to **Game 0** and **Game (1, j)** is exactly the same as **Game (1, 0)** except that the first j (anamorphic) ciphertexts $\{\text{act}_i\}_{i \in [1, j]}$ are computed as follows:

1. Parse $\text{msg}_i = (\text{msg}_{i_1}, \text{msg}_{i_2})$ where $i \leq j$
2. $\mathbf{s}_1, \mathbf{s}_2 \leftarrow \mathbb{Z}_q^n, \mathbf{e}_1, \mathbf{e}_2 \leftarrow \chi^m, y_1, y_2 \leftarrow \chi$
3. $\text{msg}_{i_1} \leftarrow \{0, 1\}, \text{msg}_{i_2} := \text{msg}_i \oplus \text{msg}_{i_1}$
4. $\text{act}_{i_1} := \text{DReg.Enc}(\text{apk}, \text{msg}_{i_1}; \mathbf{s}_1, \mathbf{e}_1^T)$
 $= (\mathbf{s}_1^T \cdot \mathbf{A} + \mathbf{e}_1^T, \mathbf{s}_1^T \cdot \mathbf{b} + y_1 + \text{msg}_{i_1} \cdot \lfloor \frac{q}{2} \rfloor \mod q)$
5. $\text{act}_{i_2} := \text{DReg.Enc}(\text{apk}, \text{msg}_{i_2}; \mathbf{s}_2, \mathbf{e}_2^T)$
 $= (\mathbf{s}_2^T \cdot \mathbf{A} + \mathbf{e}_2^T, \mathbf{s}_2^T \cdot \mathbf{b} + y_2 + \text{msg}_{i_2} \cdot \lfloor \frac{q}{2} \rfloor \mod q)$
6. **return** $\text{act}_i := (\text{act}_{i_1}, \text{act}_{i_2})$

We show that **Game (1, j)** is indistinguishable from **Game (1, j - 1)** by the DLWE assumption. The only difference between **Game (1, j)** and **Game (1, j - 1)** is in the computation of the vector $\mathbf{s}_2$. In **Game (1, j)**, $\mathbf{s}_2$ is sampled uniformly at random from $\mathbb{Z}_q^n$, whereas in **Game (1, j - 1)**, $\mathbf{s}_2$ is set as $\mathbf{x}$ where $\mathbf{x}^T = [x_1, \dots, x_n]$ such that the first $(n - 1)$ elements of the sequence are uniformly random values in $\mathbb{Z}_q$, and $x_n = (-\sum_{i=1}^{n-1} x_i a_{1,i} + \mathbf{s}_1^T \cdot \widehat{\mathbf{b}} + \widehat{y} + \text{amsg}_i \cdot \lfloor \frac{q}{2} \rfloor - e_{2,1}) \cdot a_{1,n}^{-1} \mod q$, where the a_i 's are uniformly random elements of $\mathbb{Z}_q^m$. First, we observe that by the leftover hash lemma (see Lemma 1) $(\mathbf{A}, \widehat{\mathbf{A}}\mathbf{k} = \widehat{\mathbf{b}})$ is indistinguishable from $(\mathbf{A}, \widehat{\mathbf{b}} \leftarrow \mathbb{Z}_q^n)$ in $\mathcal{D}$'s view (since $m = \Omega(n \log q)$ and $\widehat{\mathbf{k}}$ is not given to the dictator). Next, by the DLWE assumption, we have

$$\begin{aligned}
& ((\mathbf{A} || \widehat{\mathbf{b}}), \mathbf{s}_1^T \cdot (\mathbf{A} || \widehat{\mathbf{b}}) + (\mathbf{e}_1^T, \widehat{y})) \\
& \approx_c ((\mathbf{A} || \widehat{\mathbf{b}}), (\mathbf{t} \leftarrow \mathbb{Z}_q^m, u \leftarrow \mathbb{Z}_q) \mod q) \quad (\text{by DLWE}) \\
& \approx_c ((\mathbf{A} || \widehat{\mathbf{b}}), (\mathbf{s}_1^T \mathbf{A} + \mathbf{e}_1^T, u \leftarrow \mathbb{Z}_q) \mod q) \quad (\text{by DLWE}).
\end{aligned}$$

Observe that we can use $\mathbf{s}_1^T \mathbf{A} + \mathbf{e}_1^T$ and $\mathbf{k}$ to simulate act_{i_1} (by implicitly setting $y_1 = \mathbf{e}_1^T \cdot \mathbf{k} + y'_1 \in \chi$ for a uniformly sampled $y'_1 \leftarrow \chi$) and replace $(\mathbf{s}_1^T \cdot \widehat{\mathbf{b}} + \widehat{y})$ with a uniformly random value u . Thus, x_n appears uniformly random in $\mathbb{Z}_q$, which essentially hides amsg_β within act_j . In other words, the hybrids **Game (1, j)** and **Game (1, j - 1)** are indistinguishable.

Finally, we observe that **Game (1, Q)** is exactly the same as $\text{GReal}_{2\text{DReg}}^{\mathcal{D}}(\lambda)$. This completes the proof.

Fully asymmetric IND-CPA security. We prove the security using hybrid games $\text{Game } 0, \dots, \text{Game } 3$. In each hybrid game, we assume $\mathcal{A}$ is a PPT adversary.

Game 0: This is the IND-CPA security experiment $\text{GIND-CPA}_{\text{PKAE}}^A(\lambda, 0)$ defined as follows:

- 1: $\mathbf{k}, \hat{\mathbf{k}} \leftarrow \{0, 1\}^m$
- 2: $\text{ask} := \mathbf{k}, \text{apk} := (\mathbf{A}, \mathbf{b} = \mathbf{A} \cdot \mathbf{k}), \text{dk} = (\mathbf{A}, \hat{\mathbf{b}} = \mathbf{A}\hat{\mathbf{k}})$
- 3: $(\text{msg}_0, \text{msg}_1, \text{ams}_0, \text{ams}_1) \leftarrow \mathcal{A}(\text{apk}, \text{dk})$
- 4: Parse $\text{msg}_0 = (\text{msg}_{0,1}, \text{msg}_{0,2})$
- 5: Parse $\text{msg}_1 = (\text{msg}_{1,1}, \text{msg}_{1,2})$
- 6: Parse $\text{dk} = (\text{dk}_1, \text{dk}_2)$
- 7: $\mathbf{s}_1 \leftarrow \mathbb{Z}_q^n, \mathbf{e}_2 \leftarrow \chi^m, \hat{y}, y_1, y_2 \leftarrow \chi$
- 8: $\text{dk}_1 = [\mathbf{a}_1, \dots, \mathbf{a}_m] \in \mathbb{Z}_q^{n \times m}, \mathbf{e}_2^T = [e_{2,1}, \dots, e_{2,m}] \in \chi^{1 \times m}$
- 9: **for** $i \in \{1, \dots, n-1\}, x_i \leftarrow \mathbb{Z}_q$
- 10: **end for**
- 11: $x_n = (-\sum_{i=1}^{n-1} x_i a_{1,i} + \mathbf{s}_1^T \cdot \text{dk}_2 + \hat{y} + \text{ams}_0 \cdot \lfloor \frac{q}{2} \rfloor - e_{2,1}) \cdot a_{1,n}^{-1} \pmod q$
- 12: $\mathbf{x}^T = [x_1, \dots, x_n]$
- 13: $\mathbf{s}_2 := \mathbf{x}$
- 14: $\text{act}_1^* := \text{DReg.Enc}(\text{apk}, \text{msg}_{0,1}; \mathbf{s}_1, \mathbf{e}_1^T, y_1) = (\mathbf{r}_1^T, c_1)$
 $// \mathbf{r}_1^T := \mathbf{s}_1^T \cdot \text{dk}_1 + \mathbf{e}_1^T, c_1 := \mathbf{s}_1^T \cdot \mathbf{b} + y_1 + \text{msg}_{0,1} \cdot \lfloor \frac{q}{2} \rfloor \pmod q$
- 15: $\text{act}_2^* := \text{DReg.Enc}(\text{pk}, \text{msg}_{0,2}; \mathbf{s}_2, \mathbf{e}_2^T, y_2) = (\mathbf{r}_2^T, c_2)$
 $// \mathbf{r}_2^T := \mathbf{s}_2^T \cdot \text{dk}_1 + \mathbf{e}_2^T, c_2 := \mathbf{s}_2^T \cdot \mathbf{b} + y_2 + \text{msg}_{0,2} \cdot \lfloor \frac{q}{2} \rfloor \pmod q$
- 16: **return** $\beta' \leftarrow \mathcal{A}(\text{act}^* = (\text{act}_1^*, \text{act}_2^*))$

Game 1: This is the same as Game 0 except that the challenger samples x_n uniformly at random from $\mathbb{Z}_q$ instead of setting it as $(\sum_{i=1}^{n-1} x_i a_{1,i} + \mathbf{s}_1^T \cdot \text{dk}_2 + \hat{y} + \text{ams}_0 \cdot \lfloor \frac{q}{2} \rfloor - e_{2,1}) \cdot a_{1,n}^{-1} \pmod q$. The indistinguishability follows the leftover hash lemma and DLWE assumption. First, we observe that by the leftover hash lemma $(\mathbf{A}, \mathbf{A}\hat{\mathbf{k}} = \hat{\mathbf{b}})$ is indistinguishable from $(\mathbf{A}, \hat{\mathbf{b}} \leftarrow \mathbb{Z}_q^n)$ in the $\mathcal{A}$'s view (since $m = \Omega(n \log q)$). More formally, we design a simulator $\mathcal{B}$ that breaks the following indistinguishability supported by the DLWE assumption,

$$((\mathbf{A}||\hat{\mathbf{b}}), \mathbf{s}_1^T \cdot (\mathbf{A}||\hat{\mathbf{b}}) + (\mathbf{e}_1^T, \hat{y})) \approx_c ((\mathbf{A}||\hat{\mathbf{b}}), (\mathbf{s}_1^T \mathbf{A} + \mathbf{e}_1^T, u \leftarrow \mathbb{Z}_q)) \pmod q.$$

Simulator $\mathcal{B}(1^\lambda, \mathbf{A}, \hat{\mathbf{b}}, \mathbf{s}_1^T \mathbf{A} + \mathbf{e}_1^T, u)$

- 1: $\mathbf{k} \leftarrow \{0, 1\}^m$
- 2: $\text{ask} := \mathbf{k}, \text{apk} := (\mathbf{A}, \mathbf{b} = \mathbf{A} \cdot \mathbf{k}), \text{dk} = (\mathbf{A}, \hat{\mathbf{b}})$
- 3: $(\text{msg}_0, \text{msg}_1, \text{ams}_0, \text{ams}_1) \leftarrow \mathcal{A}(\text{apk}, \text{dk})$
- 4: Parse $\text{msg}_0 = (\text{msg}_{0,1}, \text{msg}_{0,2})$
- 5: Parse $\text{msg}_1 = (\text{msg}_{1,1}, \text{msg}_{1,2})$
- 6: Parse $\text{dk} = (\text{dk}_1, \text{dk}_2)$

7: $\mathbf{e}_2 \leftarrow \chi^m, y'_1, y_2 \leftarrow \chi$
 8: $\mathbf{dk}_1 = [\mathbf{a}_1, \dots, \mathbf{a}_m] \in \mathbb{Z}_q^{n \times m}, \mathbf{e}_2^T = [e_{2,1}, \dots, e_{2,m}] \in \chi^{1 \times m}$
 9: **for** $i \in \{1, \dots, n-1\}$, $x_i \leftarrow \mathbb{Z}_q$
 10: **end for**
 11: $x_n = (-\sum_{i=1}^{n-1} x_i a_{1,i} + u + \mathbf{a}\mathbf{msg}_0 \cdot \lfloor \frac{q}{2} \rfloor - e_{2,1}) \cdot a_{1,n}^{-1} \bmod q$
 12: $\mathbf{x}^T = [x_1, \dots, x_n]$
 13: $\mathbf{s}_2 := \mathbf{x}$
 14: $\mathbf{act}_1^* := \text{DReg.Enc}(\mathbf{apk}, \mathbf{msg}_{0,1}; \mathbf{s}_1, \mathbf{e}_1^T, y_1 = \mathbf{e}_1^T \cdot \mathbf{k} + y'_1) = (\mathbf{r}_1^T, c_1)$
 // $\mathbf{r}_1^T := \mathbf{s}_1^T \cdot \mathbf{A} + \mathbf{e}_1^T, c_1 := (\mathbf{s}_1^T \mathbf{A} + \mathbf{e}_1^T) \mathbf{k} + \mathbf{msg}_{0,1} \cdot \lfloor \frac{q}{2} \rfloor \bmod q$
 15: $\mathbf{act}_2^* := \text{DReg.Enc}(\mathbf{pk}, \mathbf{msg}_{0,2}; \mathbf{s}_2, \mathbf{e}_2^T, y_2) = (\mathbf{r}_2^T, c_2)$
 // $\mathbf{r}_2^T := \mathbf{s}_2^T \cdot \mathbf{dk}_1 + \mathbf{e}_2^T, c_2 := \mathbf{s}_2^T \cdot \mathbf{b} + y_2 + \mathbf{msg}_{0,2} \cdot \lfloor \frac{q}{2} \rfloor \bmod q$
 16: **return** $\beta' \leftarrow \mathcal{A}(\mathbf{act}^* = (\mathbf{act}_1^*, \mathbf{act}_2^*))$

When $u = \mathbf{s}_1^T \cdot \widehat{\mathbf{b}} + \widehat{y}$, then the distribution of $(\mathbf{apk}, \mathbf{dk})$ and the challenge ciphertext $\mathbf{act}^*$ corresponds to $\mathcal{A}$'s view are the same as in the experiment $\text{GIND-CPA}_{\text{PKAE}}^A(\lambda, 0)$. This follows from the anamorphic key-pair being perfectly valid, which makes $\mathbf{act}_1^*$ a valid encryption of $\mathbf{msg}_{0,1}$ under $\mathbf{apk} = (\mathbf{A}, \mathbf{b})$ with $y_1 = \mathbf{e}_1^T \cdot \mathbf{k} + y'_1$. Moreover, x_n is distributed as in the original experiment. On the other hand, if u is uniformly random in $\mathbb{Z}_q$, then (in line 11) $u + \mathbf{a}\mathbf{msg}_0 \cdot \lfloor \frac{q}{2} \rfloor$ becomes uniformly random over $\mathbb{Z}_q$ (under the DLWE assumption). In other words, x_n is uniformly distributed over $\mathbb{Z}_q$ and $\mathcal{B}$ simulates Game 1. Thus, Game 1 is indistinguishable from Game 0 by the leftover hash lemma and the DLWE assumption.

Game 2: This is the same as Game 1 except that the challenger encrypts $\mathbf{msg}_1$ instead of $\mathbf{msg}_0$:

1: $\mathbf{k}, \widehat{\mathbf{k}} \leftarrow \{0, 1\}^m$
 2: $\mathbf{ask} := \mathbf{k}, \mathbf{apk} := (\mathbf{A}, \mathbf{b} = \mathbf{A} \cdot \mathbf{k}), \mathbf{dk} = (\mathbf{A}, \widehat{\mathbf{b}} = \mathbf{A}\widehat{\mathbf{k}})$
 3: $(\mathbf{msg}_0, \mathbf{msg}_1, \mathbf{a}\mathbf{msg}_0, \mathbf{a}\mathbf{msg}_1) \leftarrow \mathcal{A}(\mathbf{apk}, \mathbf{dk})$
 4: Parse $\mathbf{msg}_0 = (\mathbf{msg}_{0,1}, \mathbf{msg}_{0,2})$
 5: Parse $\mathbf{msg}_1 = (\mathbf{msg}_{1,1}, \mathbf{msg}_{1,2})$
 6: Parse $\mathbf{dk} = (\mathbf{dk}_1, \mathbf{dk}_2)$
 7: $\mathbf{s}_1, \mathbf{s}_2 \leftarrow \mathbb{Z}_q^n, \mathbf{e}_1, \mathbf{e}_2 \leftarrow \chi^m, \widehat{y}, y_1, y_2 \leftarrow \chi$,
 8: $\mathbf{act}_1^* := \text{DReg.Enc}(\mathbf{apk}, \mathbf{msg}_{1,1}; \mathbf{s}_1, \mathbf{e}_1^T, y_1) = (\mathbf{r}_1^T, c_1)$
 // $\mathbf{r}_1^T := \mathbf{s}_1^T \cdot \mathbf{dk}_1 + \mathbf{e}_1^T, c_1 := \mathbf{s}_1^T \cdot \mathbf{b} + y_1 + \mathbf{msg}_{1,1} \cdot \lfloor \frac{q}{2} \rfloor \bmod q$
 9: $\mathbf{act}_2^* := \text{DReg.Enc}(\mathbf{pk}, \mathbf{msg}_{1,2}; \mathbf{s}_2, \mathbf{e}_2^T, y_2) = (\mathbf{r}_2^T, c_2)$
 // $\mathbf{r}_2^T := \mathbf{s}_2^T \cdot \mathbf{dk}_1 + \mathbf{e}_2^T, c_2 := \mathbf{s}_2^T \cdot \mathbf{b} + y_2 + \mathbf{msg}_{1,2} \cdot \lfloor \frac{q}{2} \rfloor \bmod q$
 10: **return** $\beta' \leftarrow \mathcal{A}(\mathbf{act}^* = (\mathbf{act}_1^*, \mathbf{act}_2^*))$

The indistinguishability between Game 1 and Game 2 follows the underlying Dual-Regev encryption scheme, in particular from the leftover hash lemma and DLWE assumption.

Game 3: This is the same as Game 2 except that the challenger sets $x_n = (-\sum_{i=1}^{n-1} x_i a_{1,i} + \mathbf{s}_1^T \cdot \mathbf{dk}_2 + \hat{y} + \mathbf{ams}_{g_1} \cdot \lfloor \frac{q}{2} \rfloor - e_{2,1}) \cdot a_{1,n}^{-1} \bmod q$ instead of sampling it uniformly at random from $\mathbb{Z}_q$.

- 1: $\mathbf{k}, \hat{\mathbf{k}} \leftarrow \{0, 1\}^m$
- 2: $\text{ask} := \mathbf{k}, \text{apk} := (\mathbf{A}, \mathbf{b} = \mathbf{A} \cdot \mathbf{k}), \mathbf{dk} = (\mathbf{A}, \hat{\mathbf{b}} = \mathbf{A}\hat{\mathbf{k}})$
- 3: $(\mathbf{msg}_0, \mathbf{msg}_1, \mathbf{ams}_{g_0}, \mathbf{ams}_{g_1}) \leftarrow \mathcal{A}(\text{apk}, \mathbf{dk})$
- 4: Parse $\mathbf{msg}_0 = (\mathbf{msg}_{0,1}, \mathbf{msg}_{0,2})$
- 5: Parse $\mathbf{msg}_1 = (\mathbf{msg}_{1,1}, \mathbf{msg}_{1,2})$
- 6: Parse $\mathbf{dk} = (\mathbf{dk}_1, \mathbf{dk}_2)$
- 7: $\mathbf{s}_1 \leftarrow \mathbb{Z}_q^n, \mathbf{e}_1, \mathbf{e}_2 \leftarrow \chi^m, \hat{y}, y_1, y_2 \leftarrow \chi$
- 8: $\mathbf{dk}_1 = [\mathbf{a}_1, \dots, \mathbf{a}_m] \in \mathbb{Z}_q^{n \times m}, \mathbf{e}_2^T = [e_{2,1}, \dots, e_{2,m}] \in \chi^{1 \times m}$
- 9: **for** $i \in \{1, \dots, n-1\}, x_i \leftarrow \mathbb{Z}_q$
- 10: **end for**
- 11: $x_n = (\sum_{i=1}^{n-1} x_i a_{1,i} + \mathbf{s}_1^T \cdot \mathbf{dk}_2 + \hat{y} + \mathbf{ams}_{g_1} \cdot \lfloor \frac{q}{2} \rfloor - e_{2,1}) \cdot a_{1,n}^{-1} \bmod q$
- 12: $\mathbf{x}^T = [x_1, \dots, x_n]$
- 13: $\mathbf{s}_2 := \mathbf{x}$
- 14: $\text{act}_1^* := \text{DReg.Enc}(\text{apk}, \mathbf{msg}_{1,1}; \mathbf{s}_1, \mathbf{e}_1^T, y_1) = (\mathbf{r}_1^T, c_1)$
 $// \mathbf{r}_1^T := \mathbf{s}_1^T \cdot \mathbf{dk}_1 + \mathbf{e}_1^T, c_1 := \mathbf{s}_1^T \cdot \mathbf{b} + y_1 + \mathbf{msg}_{1,1} \cdot \lfloor \frac{q}{2} \rfloor \bmod q$
- 15: $\text{act}_2^* := \text{DReg.Enc}(\text{pk}, \mathbf{msg}_{1,2}; \mathbf{s}_2, \mathbf{e}_2^T, y_2) = (\mathbf{r}_2^T, c_2)$
 $// \mathbf{r}_2^T := \mathbf{s}_2^T \cdot \mathbf{dk}_1 + \mathbf{e}_2^T, c_2 := \mathbf{s}_2^T \cdot \mathbf{b} + y_2 + \mathbf{msg}_{1,2} \cdot \lfloor \frac{q}{2} \rfloor \bmod q$
- 16: **return** $\beta' \leftarrow \mathcal{A}(\text{act}^* = (\text{act}_1^*, \text{act}_2^*))$

The indistinguishability follows the DLWE assumption as in Game 1.

Finally, we observe that Game 3 is exactly the same as $\text{GIND-CPA}_{\text{PKAE}}^A(\lambda, 1)$. This concludes the fully asymmetric security of Π_{aDReg} .

sIND-CPA security. We prove the strong IND-CPA security by showing that an adversary's view in the sIND-CPA game is computationally indistinguishable from uniformly random. For the sake of contradiction, suppose there exists a PPT adversary $\mathcal{A}$ that breaks down sIND-CPA security of Π_{aDReg} with a non-negligible probability σ . Then we design a simulator $\mathcal{B}$ that breaks the DLWE assumption.

First, we observe that by the leftover hash lemma $(\mathbf{A}, \mathbf{A} \cdot \hat{\mathbf{k}} = \hat{\mathbf{b}})$ is indistinguishable from $(\mathbf{A}, \hat{\mathbf{b}} \leftarrow \mathbb{Z}_q^n)$ in the $\mathcal{A}$'s view (since $m = \Omega(n \log q)$ and $\hat{\mathbf{k}}$ is not known to the adversary). Next, by the DLWE assumption, we have

$$((\mathbf{A} \parallel \hat{\mathbf{b}}), \mathbf{s}_1^T \cdot (\mathbf{A} \parallel \hat{\mathbf{b}}) + (\mathbf{e}_1^T, \hat{y})) \approx_c ((\mathbf{A} \parallel \hat{\mathbf{b}}), (\mathbf{s}_1^T \mathbf{A} + \mathbf{e}_1^T, u \leftarrow \mathbb{Z}_q)) \bmod q$$

We specify the simulation as follows:

Simulator $\mathcal{B}(1^\lambda, \mathbf{A}, \hat{\mathbf{b}}, \mathbf{s}_1^T \mathbf{A} + \mathbf{e}_1^T, u)$


```

1:  $\mathbf{k} \leftarrow \{0, 1\}^m$ 
2:  $\text{ask} := \mathbf{k}, \text{apk} := (\mathbf{A}, \mathbf{b} = \mathbf{A} \cdot \mathbf{k}), \text{dk} = (\mathbf{A}, \widehat{\mathbf{b}})$ 
3:  $(\mathbf{msg}, \text{amsg}_0, \text{amsg}_1) \leftarrow \mathcal{A}(\text{apk}, \text{ask}, \text{dk})$ 
4:  $\beta \leftarrow \{0, 1\}$ 
5: Parse  $\text{dk} = (\text{dk}_1, \text{dk}_2)$ 
6: Parse  $\mathbf{msg} = (\text{msg}_1, \text{msg}_2)$ 
7:  $\mathbf{s}_1 \leftarrow \mathbb{Z}_q^n, \mathbf{e}_1, \mathbf{e}_2 \leftarrow \chi^m, y'_1, y_2 \leftarrow \chi$ 
8:  $\text{dk}_1 = [\mathbf{a}_1, \dots, \mathbf{a}_m] \in \mathbb{Z}_q^{n \times m}, \mathbf{e}_2^T = [e_{2,1}, \dots, e_{2,m}] \in \chi^{1 \times m}$ 
9: for  $i \in \{1, \dots, n-1\}, x_i \leftarrow \mathbb{Z}_q$ 
10: end for
11:  $x_n = (-\sum_{i=1}^{n-1} x_i a_{1,i} + u + \text{amsg}_\beta \cdot \lfloor \frac{q}{2} \rfloor - e_{2,1}) \cdot a_{1,n}^{-1} \pmod q$ 
12:  $\mathbf{x}^T = [x_1, \dots, x_n]$ 
13:  $\mathbf{s}_2 := \mathbf{x}$ 
14:  $\text{act}_1^* := \text{DReg.Enc}(\text{apk}, \text{msg}_1; \mathbf{s}_1, \mathbf{e}_1^T, y_1 = \mathbf{e}_1^T \cdot \mathbf{k} + y'_1) = (\mathbf{r}_1^T, c_1)$ 
//  $\mathbf{r}_1^T := \mathbf{s}_1^T \cdot \mathbf{A} + \mathbf{e}_1^T, c_1 := (\mathbf{s}_1^T \mathbf{A} + \mathbf{e}_1^T) \mathbf{k} + y'_1 + \text{msg}_1 \cdot \lfloor \frac{q}{2} \rfloor \pmod q$ 
15:  $\text{act}_2^* := \text{DReg.Enc}(\text{pk}, \text{msg}_2; \mathbf{s}_2, \mathbf{e}_2^T, y_2) = (\mathbf{r}_2^T, c_2)$ 
//  $\mathbf{r}_2^T := \mathbf{s}_2^T \cdot \mathbf{A} + \mathbf{e}_2^T, c_2 := \mathbf{s}_2^T \cdot \mathbf{b} + y_2 + \text{msg}_2 \cdot \lfloor \frac{q}{2} \rfloor \pmod q$ 
16:  $\beta' \leftarrow \mathcal{A}(\mathbf{act}^* = (\text{act}_1^*, \text{act}_2^*))$ 
17: return  $\beta == \beta'$ 

```

When $u = \mathbf{s}_1^T \cdot \widehat{\mathbf{b}} + \widehat{y}$, then the distribution of $(\text{ask}, \text{apk}, \text{dk})$ and the challenge ciphertext $\mathbf{act}^*$ corresponds to $\mathcal{A}$'s view are the same as in the experiment $\text{GslIND-CPA}_{\text{PKAE}}^{\mathcal{A}}(\lambda, \beta)$. This follows from the anamorphic key-pair being perfectly valid, which makes $\mathbf{act}^*$ a valid encryption of amsg_β under $\text{dk} = (\mathbf{A}, \widehat{\mathbf{b}})$ with $y_1 = \mathbf{e}_1^T \cdot \mathbf{k} + y'_1$ where y'_1 is randomly sampled from the LWE error distribution. On the other hand, if u is uniformly random in $\mathbb{Z}_q^n$, then (in line 11) $u + \text{amsg}_\beta \cdot \lfloor \frac{q}{2} \rfloor$ is statistically close to uniform. Therefore, the challenge ciphertext $\mathbf{act}^*$ hide amsg_β . Thus, $\mathcal{A}$ has no information on the encrypted anamorphic message. Therefore, if $\mathcal{A}$ breaks the sIND-CPA security of Π_{aDReg} with a non-negligible probability then $\mathcal{B}$ can also break the DLWE assumption with a non-negligible probability. $\square$

5.3 Extension of FO-Based PKE

The well-known Fujisaki-Okamoto (FO) transformation [FO99, FO13] allows to turn a weakly secure PKE and SKE into a PKE that is IND-CCA secure in the Random Oracle Model. Let $\Pi_{\text{PKE}} = (\text{Gen}, \text{Enc}, \text{Dec})$ and $\Pi_{\text{SKE}} = (\text{Gen}, \text{Enc}, \text{Dec})$ be the underlying encryption schemes. Then the FO transformation converts them into a scheme $\Pi_{\text{FO}} = (\text{Gen}, \text{Enc}, \text{Dec})$ as follows: $\text{FO.Gen}(1^\lambda)$ simply executes $(\text{pk}, \text{sk}) \leftarrow \text{PKE.Gen}(1^\lambda)$. $\text{FO.Enc}(\text{pk}, \text{msg})$ returns $c := (c_1, c_2) := (\text{PKE.Enc}(\text{pk}, r; H(r, \text{msg})), \text{SKE.Enc}(G(r), \text{msg}))$, where $r \leftarrow \mathcal{M}_{\text{PKE}}$ is randomly chosen from the message space of PKE, and H and G are hash functions (modeled as random oracles) mapping to the space of random coins $\mathcal{R}_{\text{PKE}}$ of PKE and the key space $\mathcal{K}_{\text{SKE}}$ of SKE, respectively. To decrypt, $\text{FO.Dec}(\text{sk}, c)$ computes $r := \text{PKE.Dec}(\text{sk}, c_1)$ and returns $\text{SKE.Dec}(G(r), c_2)$ if $c_1 = \text{PKE.Enc}(\text{pk}, r; H(r, \text{msg}))$.

The idea to turn Π_{FO} into an anamorphic public key encryption scheme is simple: The FO transformation provides us with the recoverable randomness $r \leftarrow \mathcal{M}_{\text{PKE}}$ which we can use to embed the components of a Π_{FO} ciphertext for amsg . For this purpose, the ciphertext components need to be pseudo-random. More precisely, we require IND\$-CPA security for Π_{PKE} and wIND\$-CPA security for Π_{SKE} , both saying that its hard for an adversary to distinguish the encryption of a chosen message from a random ciphertext. In the wIND\$-CPA game, no encryption oracle is given. For both definitions, please refer to Section 3. Besides ciphertexts being pseudo-random, they also need to be compatible with $\mathcal{M}_{\text{PKE}}$. To ensure the latter, we require the existence of a pair of PPT algorithms (Encode, Decode) that allows us to encode ciphertexts from $\mathcal{C}_{\text{FO}} = \mathcal{C}_{\text{PKE}} \times \mathcal{C}_{\text{SKE}}$ to plaintext vectors from $\mathcal{M}_{\text{PKE}}^\mu$ for some small $\mu \in \mathbb{N}$ (cf. Definition 16). The resulting μ -message anamorphic extension Π_{FO} is given in Definition 3.

Definition 16 (Pseudo-Random Encodings) *Let $\text{Setup}(1^\lambda)$ be a PPT algorithm that outputs (polynomial-size descriptions of) two sets $\mathbb{X}$ and $\mathbb{Y}$ which both allow for efficient uniform sampling. Let (Encode, Decode) be a pair of PPT algorithms such that Encode takes an element $x \in \mathbb{X}$ as input and outputs an element $y \in \mathbb{Y}$ or $\perp$ and Decode takes an element $y \in \mathbb{Y}$ as input and outputs an element $x \in \mathbb{X}$ or $\perp$. We call this pair a pseudo-random $(\mathbb{X}, \mathbb{Y})$ -encoding associated with Setup, if it satisfies the following properties:*

1. *Correctness: For any $(\mathbb{X}, \mathbb{Y}) \leftarrow \text{Setup}(1^\lambda)$, it holds that*

$$\Pr [x \neq x' : x \leftarrow \mathbb{X}, y \leftarrow \text{Encode}(x), x' \leftarrow \text{Decode}(y)] \leq \text{negl}(\lambda)$$

2. *Pseudo-randomness: For any $(\mathbb{X}, \mathbb{Y}) \leftarrow \text{Setup}(1^\lambda)$, and any PPT algorithm $\mathcal{A}$ holds that*

$$\Pr \left[b = b' : \begin{array}{l} x \leftarrow \mathbb{X}, y_0 \leftarrow \text{Encode}(x), y_1 \leftarrow \mathbb{Y}, \\ b \leftarrow \{0, 1\}, b' \leftarrow \mathcal{A}(\mathbb{X}, \mathbb{Y}, y_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda)$$

Figure 3 presents our FO-based PKAE construction. It is not hard to see that Π_{FO} satisfies anamorphic security as the embedded ciphertexts components containing amsg are indistinguishable from a regular $r \leftarrow \mathcal{M}_{\text{PKE}}$. Moreover, the IND-CCA security of Π_{FO} for encrypting amsg (i.e., the “inner” layer encryption) implies sIND-RCCA for the overall construction Π_{FOAE} . Roughly speaking, that means that an adversary (including the dictator) is not able to maul amsg but it may tamper with the “inner” and “outer” layer ciphertexts. Note that Π_{FOAE} does not satisfy IND-CCA security as aDec does not check the integrity for the “outer” layer ciphertexts act_j . Hence, if the underlying Π_{PKE} is homomorphic, components of act_j could be re-randomized.

<u>aGen(1^λ) :</u> 1. $(\text{ask}, \text{apk}) \leftarrow \text{FO.Gen}(1^\lambda)$ 2. $(\text{tk}', \text{dk}) \leftarrow \text{FO.Gen}(1^\lambda)$ 3. $\text{tk} := (\text{ask}, \text{tk}')$ 4. return $(\text{ask}, \text{apk}, \text{dk}, \text{tk})$ <u>aDec(tk, act) :</u> 1. for $j \in \{1, \dots, \mu\}$: Parse $\text{act}_j = (c_1, c_2)$ $r_j := \text{PKE.Dec}(\text{ask}, c_1)$ if $r_j = \perp$ return $\perp$ 2. $c := \text{Decode}(r_1, \dots, r_\mu)$ 3. if $c = \perp$ return $\perp$ 4. return $\text{FO.Dec}(\text{tk}', c)$	<u>aEnc($\text{apk}, \text{dk}, \text{msg}, \text{amsg}$) :</u> 1. Parse $\text{msg} = (\text{msg}_1, \dots, \text{msg}_\mu)$ 2. $c \leftarrow \text{FO.Enc}(\text{dk}, \text{amsg})$ 3. $(r_1, \dots, r_\mu) \leftarrow \text{Encode}(c)$ // In the negligible event that // $\text{Encode}(c) = \perp$, we return to 2. 4. for $j \in \{1, \dots, \mu\}$: $\text{act}_j := \text{FO.Enc}(\text{apk}, \text{msg}_j; r_j)$ 5. return $\text{act} := (\text{act}_1, \dots, \text{act}_\mu)$
---	--

Figure 3: μ -PKAEE Π_{FOAE}

Theorem 3 Let Π_{FO} be an IND-CCA secure FO-based PKE where the underlying PKE Π_{PKE} is IND $\mathcal{S}$ -CPA secure and has an exponentially large message space $\mathcal{M}_{\text{PKE}}$ (i.e., $|\mathcal{M}_{\text{PKE}}| \geq 2^{\text{poly}(\lambda)}$ for non-constant poly), and the underlying SKE Π_{SKE} is wIND $\mathcal{S}$ -CPA secure. Furthermore, let $(\text{Encode}, \text{Decode})$ be a pseudo-random $(\mathcal{C}_{\text{FO}}, \mathcal{M}_{\text{PKE}}^\mu)$ -encoding, where $\mathcal{C}_{\text{FO}}$ denotes the ciphertext space of Π_{FO} . Then Π_{FOAE} as defined in Figure 3 is a μ -message anamorphic extension satisfying anamorphic security and sIND-RCCA security in the ROM.

Proof. We prove this theorem in two parts.

Anamorphic security. In the following sequence of hybrid games let $\mathcal{D}$ be a PPT dictator and Q its total number of oracle queries.

Game 0: This is the real game $\text{GReal}_{\text{FO}}^{\mathcal{D}}(\lambda)$ in the anamorphic security definition, where the challenger generates $(\text{pk}, \text{sk}) \leftarrow \text{PKE.Gen}(1^\lambda)$ and handles encryption queries $\text{OEnc}(\text{msg}, \text{amsg})$ of the dictator by computing $(\text{FO.Enc}(\text{pk}, \text{msg}_i))$ for $1 \leq i \leq \mu$. The adversary also has access to the random oracles H and G .

Game 1: In this game the challenger generates the key pair by computing $(\text{apk}, \text{ask}, \text{dk}, \text{tk}) \leftarrow \text{FOAE.aGen}(1^\lambda)$ and provides (apk, ask) as input to $\mathcal{D}$.

Game $(2, j)$ for $j \in [0, Q]$: In this series of games we slightly modify Game 1. We define Game $(2, 0)$ to be identical to Game 1. In Game $(2, j)$, the challenger handles the first j encryption oracle

calls as follows: In such a call, the oracle first randomly chooses an element c from $\mathcal{C}_{\text{FO}}$ and applies $\text{Encode}(c)$ to obtain $(r_1, \dots, r_\mu)$. Then the latter values are used as encryption randomness: $(\text{FO.Enc}(\text{apk}, \text{msg}_i; r_i))$ for $1 \leq i \leq \mu$. The remaining oracle calls are left untouched in comparison to Game (2, 0). Note that in Game (2, Q) we have modified all encryption computations in this way.

Game (3, j) for $j \in [0, Q]$: In this series of games, we modify Game (2, Q). We define Game (3, 0) to be identical to Game (2, Q). In Game (3, j), the challenger handles the first j encryption oracle calls as follows: In such a call, instead of choosing c fully at random from $\mathcal{C}_{\text{FO}}$, we replace the components representing the ciphertext of PKE by an actual encryption of a random message r' . That means, we compute $c = (c_{\text{PKE}}, c_{\text{SKE}}) \in \mathcal{C}_{\text{FO}} = \mathcal{C}_{\text{PKE}} \times \mathcal{C}_{\text{SKE}}$ as $c = (\text{PKE.Enc}(\text{dk}, r'), c_{\text{SKE}})$, where $r' \leftarrow \mathcal{M}_{\text{PKE}}$ and $c_{\text{SKE}} \leftarrow \mathcal{C}_{\text{SKE}}$.

Game (4, j) for $j \in [0, Q]$: In this series of games, we modify Game (3, Q). We define Game (4, 0) to be identical to Game (3, Q). In Game (4, j), the challenger handles the first j encryption oracle calls as follows: In such a call, we now also replace the components of c representing the ciphertext part of SKE. That means, we compute c as $c = (\text{PKE.Enc}(\text{dk}, r'), \text{SKE.Enc}(k, \text{amsg}))$, where $r' \leftarrow \mathcal{M}_{\text{PKE}}$ and $k \leftarrow \text{SKE.Gen}(1^\lambda)$.

Game 5: In this game, in all oracle calls when computing c , the encryption randomness of PKE and the key k of SKE is generated using the random oracles H and G as in the FO construction, i.e., $c = (\text{PKE.Enc}(\text{dk}, r'; H(r', \text{amsg})), \text{SKE.Enc}(G(r'), \text{amsg}))$, $r' \leftarrow \mathcal{M}_{\text{PKE}}$.

Game 6: This is the anamorphic game $\text{GAnam}_{\text{FOAE}}^{\mathcal{D}}(\lambda)$, where the challenger generates $(\text{apk}, \text{ask}, \text{dk}, \text{tk}) \leftarrow \text{FOAE.aGen}(1^\lambda)$ and handles encryption queries $\text{OaEnc}(\text{msg}, \text{amsg})$ of the dictator by computing $\text{FOAE.aEnc}(\text{apk}, \text{dk}, \text{msg}, \text{amsg})$.

Game 0 and Game 1 are identical as (ask, apk) are generated in aGen by FO.Gen . Game (2, 0) is indistinguishable from Game (2, Q), as a distinguisher $\mathcal{D}$ for Game (2, j) and Game (2, $j + 1$) (for random $j \in \{0, \dots, Q - 1\}$) can be turned in to an adversary against the pseudo-randomness of the encoding. Similarly, Game (3, 0) is indistinguishable from Game (3, Q), as from a distinguisher $\mathcal{D}$ for Game (3, j) and Game (3, $j + 1$) (for random j) an IND\$-CPA adversary against Π_{PKE} could be constructed. By a similar hybrid argument, Game (4, 0) is indistinguishable from Game (4, Q), as a distinguisher $\mathcal{D}$ for Game (4, j) and Game (4, $j + 1$) (for random j) leads to an wIND\$-CPA adversary against Π_{SKE} . Next, Game (4, Q) and Game 5 are computationally distinguishable: The random oracle call $H(r', \text{amsg})$, for *uniformly chosen* r' , will not result in a freshly chosen value if (r', amsg) has been input to H before. If there are Q_H queries to H in total, this could happen with probability at most $\frac{Q_H}{|\mathcal{M}_{\text{PKE}}|}$ for each call to OEnc and thus with probability $\frac{Q \cdot Q_H}{|\mathcal{M}_{\text{PKE}}|}$ for all calls. Note that our assumption is that the message space is of exponential size. A similar argument holds for random oracle calls $G(r')$. Finally, Game 5 and Game 6 are identical.

sIND-RCCA Security. This follows immediately from the IND-CCA security of Π_{FO} . The reduction algorithm $\mathcal{B}$ is given dk as input from its IND-CCA challenger and has access to a $\text{Dec}(\text{tk}', \cdot)$ oracle. It can generate $(\text{apk}, \text{ask}) \leftarrow \text{PKE.Gen}(1^\lambda)$ itself, and thus provide $(\text{apk}, \text{dk}, \text{ask})$ as input to the sIND-RCCA adversary $\mathcal{A}$.

Upon receiving $(\text{msg}, \text{amsg}_0, \text{amsg}_1)$ from $\mathcal{A}$, $\mathcal{B}$ provides $(\text{amsg}_0, \text{amsg}_1)$ to its own IND-CCA challenger to obtain c^* . The ciphertext c^* is then transformed into a challenge ciphertext vector act^* as specified in aEnc in Figure 3.

To simulate the $\text{aDec}(\text{tk}, \cdot)$ oracle, where $\text{tk} = (\text{ask}, \text{tk}')$, $\mathcal{B}$ uses the self-chosen ask to decrypt the public key components of act in order to obtain $(r_1, \dots, r_\mu)$ (cf. aDec in Figure 3). Then it decodes them to obtain c . If $c = c^*$, $\mathcal{B}$ rejects, i.e., it sends $\perp$ as oracle response to $\mathcal{A}$. Otherwise it queries its own $\text{Dec}(\text{tk}', \cdot)$ to decrypt c . If this decryption yields amsg_1 or amsg_2 , $\mathcal{B}$ sends $\perp$ as oracle response to $\mathcal{A}$. Note that this simulation of an sIND-RCCA aDec oracle is perfect: The only thing which is done differently by the simulation is that it checks if $c = c^*$ and returns $\perp$ if this equation holds. However, by correctness of Π_{FO} it follows that such a c would have been decrypted to amsg_1 or amsg_2 and thus the corresponding act would have also been rejected by the real aDec oracle. Finally $\mathcal{B}$ outputs whatever $\mathcal{A}$ outputs. $\square$

An exemplary instantiation: FO of Hashed ElGamal and XOR. Let us consider an instantiation of Π_{FO} based on hashed ElGamal as PKE and XOR for the SKE part. Hashed ElGamal satisfies $\text{IND\$-CPA}$ security under DDH in the ROM and XORing a message with a random key of the same length satisfies $\text{wIND\$-CPA}$ security. In this case, $\text{FO.Enc}(\text{pk}, \text{msg})$ would produce a ciphertext $c = (g^{H(r, \text{msg})}, H'(\text{pk}^{H(r, \text{msg})}) \oplus r, G(r) \oplus \text{msg})$, where $H : \{0, 1\}^* \times \{0, 1\}^* \rightarrow \mathbb{Z}_{|\mathbb{G}|}$, $H' : \mathbb{G} \rightarrow \{0, 1\}^{\ell_1}$, $G : \{0, 1\}^* \rightarrow \{0, 1\}^{\ell_2}$, and $\mathbb{G}$ is the group over which ElGamal is defined. To obtain Π_{FOAE} , we also need to instantiate an appropriate pseudo-random encoding. For this, certain parameters need to be fixed: Let us assume $\mathbb{G} = \mathbb{E}(\mathbb{F}_p)$ is an elliptic curve group of prime order q , where p is a λ -bit prime. Furthermore, let us set $\ell_1 = \ell_2 = \lambda$ (but also different parameter settings like $\ell_1 = \frac{\lambda}{c_1}$ and $\frac{\lambda}{c_1 c_2}$ for $c_1, c_2 \in \mathbb{N}$ s.t. $\ell_1, \ell_2 \in \mathbb{N}$ are possible). In this case, we have to encode random elements of $\mathcal{C}_{\text{FO}} = \mathbb{E}(\mathbb{F}_p) \times \{0, 1\}^\lambda \times \{0, 1\}^\lambda$ as random elements of $\mathcal{M}_{\text{PKE}}^\mu = (\{0, 1\}^\lambda)^\mu$ for some $\mu \in \mathbb{N}$. Obviously, the only difficulty here is encoding the first component, i.e., a random point of $\mathbb{E}(\mathbb{F}_p)$ by one or more random λ -bit strings. For this purpose, we can deploy Elligator Squared [Tib14]. This is a suitable encoding for essentially arbitrary elliptic curve groups $\mathbb{E}(\mathbb{F}_p)$ (also pairing-friendly) to $\mathbb{F}_p^2$. Encodings of random curve points can be efficiently computed and are statistically indistinguishable from uniformly random elements of $\mathbb{F}_p^2$. Moreover, [Tib14] describes how encodings can be turned into random bit strings. For instance, if p is very close to 2^λ (which is often the case for standardized curves), and we use the basic λ -bit representation for elements of $\mathbb{F}_p$ as integers from $\{0, \dots, p-1\}$, then we can simply interpret the output of the encoding as (statistically close to) uniform bitstrings from $\{0, 1\}^\lambda \times \{0, 1\}^\lambda$. In this way, we can obtain a pseudo-random encoding of $\mathbb{E}(\mathbb{F}_p) \times \{0, 1\}^\lambda \times \{0, 1\}^\lambda$ elements as $(\{0, 1\}^\lambda)^4$ elements.

6 Identity-Based Anamorphic Encryption Extension

In this section, we formally regard our new concept of reformulating public key anamorphic encryption through the lens of IBE. We further show how to realize this notion using the Boneh-Franklin IBE in Section 6.1. Section 6.2 shows how standardized IBE schemes based on the FO transformation can be turned into anamorphic schemes.

At a high-level, an IBAE is a quadruplet of PPT algorithms $\Pi_{\text{IBAE}} = (\text{aSetup}, \text{aKeyGen}, \text{aEnc}, \text{aDec})$ associated with a regular IBE $\Pi_{\text{IBE}} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ which could be deployed in either of the two modes: *regular* and *anamorphic*. In the regular mode, a dictator computes the master key-pairs (mpk, msk) using the Setup algorithm, and publishes the master public key mpk . It then derives user-specific private keys sk_{id} based on the public identity id of users, using the master secret key msk on the KeyGen algorithm. A message msg can be encrypted with respect to id of a receiver using mpk on the Enc algorithm, which can eventually be decrypted using the respective

private key sk_{id} on Dec algorithm. As one can clearly see the dictator along with the receiver can recover msg since it generates the user-specific private keys. In the anamorphic mode, while the dictator generates the anamorphic master key-pairs $(\text{ampk}, \text{amsk})$ and derives the anamorphic private keys ask_{id} , the anamorphic clan lead generates the double master key-pairs $(\text{dmpk}, \text{dmsk})$ and the double private key dsk_{id} of a user with identity id who is also an anamorphic clan member. To encrypt a regular message msg along with an anamorphic message amsk with respect to id , sender runs aEnc such that the output ciphertext decrypts to amsk using dsk_{id} on aDec and msg using ask_{id} on Dec. *Anamorphic security* essentially requires that the dictator should not be able to tell apart the output distributions of regular and anamorphic ciphertexts, along with the corresponding private key pairs. We now proceed more formally.

Definition 17 (μ -Message Identity-Based Anamorphic Encryption Extension) For $\mu \in \mathbb{N}$, a μ -message anamorphic identity-based encryption extension (μ -IBAAE) with an anamorphic message space $\widehat{\mathcal{M}}$ associated with an identity-based encryption scheme $\Pi_{\text{IBE}} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ with a regular message space $\mathcal{M}$ is a quadruplet of PPT algorithms $\Pi_{\text{IBAE}} = (\text{aSetup}, \text{aKeyGen}, \text{aEnc}, \text{aDec})$ defined as follows:

$(\text{ampk}, \text{amsk}, \text{dmpk}, \text{dmsk}) \leftarrow \text{aSetup}(1^\lambda)$: The anamorphic key setup algorithm aSetup takes as input the security parameter 1^λ , and outputs an anamorphic master public key ampk , an anamorphic master secret key amsk , a double master public key dmpk and a double master secret key dmsk .

$\text{dsk}_{\text{id}} \leftarrow \text{aKeyGen}(\text{dmsk}, \text{id})$: The anamorphic key generation algorithm aKeyGen takes as input the double master secret key dmsk and an arbitrary $\text{id} \in \mathcal{ID}$, and outputs a double private key dsk_{id} associated with id .

$\text{act} \leftarrow \text{aEnc}(\text{ampk}, \text{dmpk}, \text{id}, \text{msg}, \text{amsk})$: The anamorphic encryption algorithm aEnc takes as input the anamorphic master public key ampk , double master public key dmpk , an identity id , a regular message vector $\text{msg} \in \mathcal{M}^\mu$, and an anamorphic message $\text{amsk} \in \widehat{\mathcal{M}}$, and outputs an anamorphic ciphertext vector act .

$\text{amsk} \leftarrow \text{aDec}(\text{ask}_{\text{id}}, \text{dsk}_{\text{id}}, \text{act})$: The anamorphic decryption algorithm aDec takes as input an anamorphic private key ask_{id} , a double private key dsk_{id} and the anamorphic ciphertext vector act , and outputs an anamorphic message amsk or the symbol $\perp$.

Correctness: For every pair of messages $(\text{msg}, \text{amsk}) \in \mathcal{M}^\mu \times \widehat{\mathcal{M}}$, and for every identity $\text{id} \in \mathcal{ID}$, the following holds:

$$\Pr \left[\begin{array}{l} (\text{ampk}, \text{amsk}, \text{dmpk}, \text{dmsk}) \leftarrow \text{aSetup}(1^\lambda) \\ \text{ask}_{\text{id}} \leftarrow \text{KeyGen}(\text{amsk}, \text{id}) \\ \text{dsk}_{\text{id}} \leftarrow \text{aKeyGen}(\text{dmsk}, \text{id}) \\ \text{act} \leftarrow \text{aEnc}(\text{ampk}, \text{dmpk}, \text{id}, \text{msg}, \text{amsk}) \end{array} : \text{amsk} = \text{aDec}(\text{ask}_{\text{id}}, \text{dsk}_{\text{id}}, \text{act}) \right] = 1$$

where probability is taken over the random coin tosses of aSetup and aEnc .

Anamorphic security: For every PPT dictator $\mathcal{D}$ it holds that

$$|\Pr[\text{GReal}_{\text{IBE}}^{\mathcal{D}}(\lambda) = 1] - \Pr[\text{GANam}_{\text{IBAE}}^{\mathcal{D}}(\lambda) = 1]| \leq \text{negl}(\lambda)$$

where the security games are defined as follows:

<u>$\text{GReal}_{\text{IBE}}^{\mathcal{D}}(\lambda) :$</u>	<u>$\text{GAnam}_{\text{IBAE}}^{\mathcal{D}}(\lambda) :$</u>
1. $(\text{mpk}, \text{msk}) \leftarrow \text{Setup}(1^\lambda)$	1. $(\text{ampk}, \text{amsk}, \text{dmpk}, \text{dmsk}) \leftarrow \text{aSetup}(1^\lambda)$
2. return $\mathcal{D}^{\text{OEnc}(\cdot, \cdot, \cdot)}(\text{mpk}, \text{msk})$	2. return $\mathcal{D}^{\text{OaEnc}(\cdot, \cdot, \cdot)}(\text{ampk}, \text{amsk})$
where $\text{OEnc}(\text{id}, \text{msg}, \text{amsk}) = \text{ct}$,	where $\text{OaEnc}(\text{id}, \text{msg}, \text{amsk}) =$
$\text{ct}_i = \text{Enc}(\text{mpk}, \text{id}, \text{msg}_i)$,	$\text{aEnc}(\text{ampk}, \text{dmpk}, \text{id}, \text{msg}, \text{amsk})$
$\text{msg} = \{\text{msg}_i\}_{i \in [\mu]}$, $\text{ct} = \{\text{ct}_i\}_{i \in [\mu]}$	

Fully asymmetric IND-CPA security: An IBAE satisfies fully asymmetric IND-CPA security if for every PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that for all $\lambda \in \mathbb{N}$, the following holds:

$$|\Pr[\text{GIND-CPA}_{\text{IBAE}}^{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\text{GIND-CPA}_{\text{IBAE}}^{\mathcal{A}}(\lambda, 1) = 1]| \leq \text{negl}(\lambda)$$

where the security game is defined as follows:

<u>$\text{GIND-CPA}_{\text{IBAE}}^{\mathcal{A}}(1^\lambda, \beta) :$</u>
1. $(\text{ampk}, \text{amsk}, \text{dmpk}, \text{dmsk}) \leftarrow \text{aSetup}(1^\lambda)$
2. $(\text{id}^*, \text{msg}_0, \text{msg}_1, \text{amsk}_0, \text{amsk}_1) \leftarrow \mathcal{A}^{\text{OKG}(\cdot)}(\text{ampk}, \text{dmpk})$
3. $\text{act}_\beta = \text{aEnc}(\text{ampk}, \text{dmpk}, \text{id}^*, \text{msg}_\beta, \text{amsk}_\beta)$
4. return $\beta' \leftarrow \mathcal{A}^{\text{OKG}(\cdot)}(\text{act}_\beta)$
where $\text{OKG}(\text{id}) = (\text{KeyGen}(\text{amsk}, \text{id}), \text{aKeyGen}(\text{dmsk}, \text{id}))$
and id^* is not queried to $\text{OKG}(\cdot)$.

We also consider additional security definitions for IBAE.

Strong IND-CPA security and beyond: An IBAE satisfies strong IND-CPA security if for every PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that for all $\lambda \in \mathbb{N}$, the following holds:

$$|\Pr[\text{GsIND-CPA}_{\text{IBAE}}^{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\text{GsIND-CPA}_{\text{IBAE}}^{\mathcal{A}}(\lambda, 1) = 1]| \leq \text{negl}(\lambda)$$

where the security game is defined as follows:

GsIND-CPA_{IBAE}^A(1^λ, β) :

1. (ampk, amsk, dmpk, dmsk) $\leftarrow$ aSetup(1^λ)
 2. (id*, msg, amsg₀, amsg₁) $\leftarrow$ $\mathcal{A}^{\text{OaKeyGen}(\cdot)}$ (ampk, amsk, dmpk)
 3. **act**_β = aEnc(ampk, dmpk, id*, msg, amsg_β)
 4. **return** β' $\leftarrow$ $\mathcal{A}^{\text{OaKeyGen}(\cdot)}$ (**act**_β)
- where OaKeyGen(id) = aKeyGen(dmsk, id)
- and id* is not queried to OaKeyGen(·).

We call Π_{IBAE} fully asymmetric IND-CCA secure if the advantage of $\mathcal{A}$ in GIND-CPA_{IBAE}^A is negligible when it is also given access to an oracle ODec(·, ·) that on input (id, act) returns (Dec(amsk, id, act), aDec(dmsk, id, act)), which decrypts every given act except for the challenge ciphertext **act**_β with the challenge identity id*. To also guarantee IND-CCA security with respect to the dictator who knows ask, we additionally provide ask to $\mathcal{A}$ as input and call the corresponding security notion strong IND-CCA secure (sIND-CCA) where ODec(·, ·) is replaced with the oracle aDec(dmsk, ·, ·). Since, unfortunately, our constructions do not achieve sIND-CCA security, we define a relaxation in the spirit of Replayable CCA (RCCA) security for IBE similar to the case of PKE: RCCA captures a security notion which is identical to CCA except for the added ability of the attacker to generate new ciphertexts that decrypt to the same plaintexts as previously seen ciphertexts. More precisely, our sIND-RCCA game coincides with the sIND-CCA game, except that the aDec(dmsk, ·, ·)-oracle rejects for a given (id, act) if aDec(dmsk, id, act) results in amsg₀ or amsg₁.

sIND-RCCA security: An IBAE satisfies sIND-RCCA security if for every PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that for all $\lambda \in \mathbb{N}$, the following holds:

$$|\Pr[\text{GsIND-RCCA}_{\text{IBAE}}^A(\lambda, 0) = 1] - \Pr[\text{GsIND-RCCA}_{\text{IBAE}}^A(\lambda, 1) = 1]| \leq \text{negl}(\lambda)$$

where the security game is defined as follows:

GsIND-RCCA_{IBAE}^A(1^λ, β) :

1. (ampk, amsk, dmpk, dmsk) $\leftarrow$ aSetup(1^λ)
 2. (id*, msg, amsg₀, amsg₁) $\leftarrow$ $\mathcal{A}^{\text{OaKeyGen}(\cdot), \text{OaDec}(\cdot, \cdot)}$ (ampk, amsk, dmpk)
 3. **act**_β = aEnc(ampk, dmpk, id*, msg_β, amsg_β)
 4. **return** β' $\leftarrow$ $\mathcal{A}^{\text{OaKeyGen}(\cdot), \text{OaDec}(\cdot, \cdot)}$ (**act**_β)
- where OaKeyGen(id) = aKeyGen(dmsk, id), OaDec(id, act) = aDec(dmsk, id, act)
- and id* is not queried to OaKeyGen(·)
- and (id*, **act**_β) is not queried to OaDec(·, ·)
- and OaDec(id, act) outputs $\perp$ if aDec(dmsk, id, act) = amsg₀ or amsg₁.

We now formally regard the IBAE-compatible notion for an IBE. Building upon our previous insight, the notion captures *two* properties: *on-the-fly double key generation* property simply points out that the double master key pairs $(\text{dmpk}, \text{dmsk})$ can simply be derived from the regular IBE master key pairs (mpk, msk) , while the *ciphertext extractability* property invariably indicates that IBAE.aDec is essentially the same as IBE.Dec under the existence of an efficiently computable function F . More explicitly, $F(\text{ask}_{\text{id}}, \text{act})$ produces a valid ciphertext that extracts to amsg using dsk_{id} on IBE.Dec .

Definition 18 (IBAE-Compatible) *An identity-based encryption scheme $\Pi_{\text{IBE}} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ is said to be IBAE-compatible for the quadruplet $\Pi_{\text{IBAE}} = (\text{aSetup}, \text{aKeyGen}, \text{aEnc}, \text{aDec})$ if the following holds:*

- *On-the-fly double key generation:* A part of output $(\text{dmpk}, \text{dmsk})$ of $\text{IBAE.aSetup}(1^\lambda)$ is the output of $\text{IBE.Setup}(1^\lambda)$. In particular, if $\text{IBAE.aSetup}(1^\lambda) = (\text{ampk}, \text{amsk}, \text{dmpk}, \text{dmsk})$ and $\text{IBE.Setup}(1^\lambda) = (\text{mpk}, \text{msk})$, $\text{IBE.Setup}(1^\lambda) = (\text{mpk}', \text{msk}')$ then $\text{ampk} = \text{mpk}$, $\text{amsk} = \text{msk}$, $\text{dmsk} = \text{msk}'$ and there exists an efficiently computable function G such that $\text{dmpk} = G(\text{mpk}, \text{msk}, \text{mpk}', \text{msk}')$ ⁴.
- *Ciphertext extractability:* There exists an efficiently computable function F such that $\text{IBE.Dec}(\text{dsk}_{\text{id}}, F(\text{ask}_{\text{id}}, \text{act})) = \text{amsg}$, where $\text{act} = \text{IBAE.aEnc}(\text{ampk}, \text{dmpk}, \text{id}, \text{msg}, \text{amsg})$ for an $\text{id} \in \mathcal{ID}$.

Remark 1 The on-the-fly double key generation property guarantees that the double master secret key dmsk can be independently sampled via the IBE.Setup algorithm, without requiring knowledge of the original key pair $(\text{ampk}, \text{amsk})$. Moreover, the corresponding double master public key dmpk can be efficiently derived from two independently generated key pairs: the original $(\text{mpk} = \text{ampk}, \text{msk} = \text{amsk})$ and a freshly sampled $(\text{mpk}', \text{msk}' = \text{dmsk})$. This property supports two operational paradigms:

1. **Authority-before-dictatorship:** Here, the dictator initially obtains the key pair (mpk, msk) by coercing the key generation authority of an IBE system. Subsequently, the original authority must be able to generate a valid double master public key $\text{dmpk} = G(\text{mpk}, \text{msk}, \text{mpk}', \text{msk}')$ by independently executing IBE.Setup and becomes an anamorphic clan leader.
2. **Authority-after-dictatorship:** In this setting—preferred and further motivated in Section 2—the dictator independently generates the key pair (mpk, msk) , and any party can later assume the role of the anamorphic clan leader by independently sampling $(\text{msk}', \text{mpk}')$ through IBE.Setup and computing the joint public key as $\text{dmpk} = G(\text{mpk}, \perp, \text{mpk}', \text{msk}' = \text{dmsk})$.

Our construction of IBAEE, instantiated from the Boneh-Franklin **BasicIdent** scheme (Section 6.1), conforms to the second paradigm. In this case, the double master public key dmpk can be deterministically computed using the original authority's public key ampk and the independently generated secret key dmsk , aligning with the on-the-fly generation requirement.

⁴In $\text{dmpk} = G(\text{mpk}, \text{msk}, \text{mpk}', \text{msk}')$, the msk could be $\perp$ in certain scenarios (see Remark 1).

6.1 Extension of the Boneh-Franklin BasicIdent Scheme

We now present an IBAE construction based on the seminal Boneh-Franklin basic IBE scheme [BF01]. We focus on a simple version that encrypts single bits. The BasicIdent identity-based encryption scheme [BF01] is a quadruplet of PPT algorithms $\Pi_{\text{BF}} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ defined over a security parameter $\lambda \in \mathbb{N}$ as follows: let prime $q > 2^\lambda$ be a polynomial in λ . The setup algorithm **Setup** on input 1^λ samples groups $\mathbb{G}$ and $\mathbb{G}_T$ of order q and a bilinear map $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$, along with an arbitrary group generator $P \in \mathbb{G}$. Define $H_1 : \{0, 1\}^* \rightarrow \mathbb{G}^*$, $H_2 : \mathbb{G}_T \rightarrow \{0, 1\}$. Then for a uniformly random s in $\mathbb{Z}_q^*$, compute $P_{\text{pub}} := P^s$. The algorithm finally returns a master secret key $\text{msk} := s$ and a master public key $\text{mpk} := (\mathbb{G}, \mathbb{G}_T, P, e, q, P_{\text{pub}}, H_1, H_2)$. The key generation algorithm **KeyGen** on input $\text{id} \in \{0, 1\}^*$ and msk , computes $Q_{\text{id}} := H_1(\text{id})$, and returns a private key $\text{sk}_{\text{id}} := Q_{\text{id}}^s$ corresponding to id . To encrypt a message $\text{msg} \in \mathcal{M}$ towards a public identity id , where $\mathcal{M} = \{0, 1\}$, the encryption algorithm **Enc** samples a uniformly random κ in $\mathbb{Z}_q^*$ and returns ciphertext ct as $(\text{ct}_1, \text{ct}_2) = (P^\kappa, \text{msg} \oplus H_2(g_{\text{id}}^\kappa))$, where $g_{\text{id}} = e(Q_{\text{id}}, P_{\text{pub}})$. The decryption algorithm **Dec** simply recovers msg by computing $\text{ct}_2 \oplus H_2(e(\text{sk}_{\text{id}}, \text{ct}_1))$. The BasicIdent scheme is known to be IND-CPA secure under the DBDH assumption for $\mathbb{G}$. Following our prior formalization, we can similarly define the Boneh-Franklin 2-message IBE scheme $\Pi_{2\text{BF}} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ which encrypts two messages at once.

Boneh-Franklin 2-Message IBAE. To design a 2-IBAE associated with $\Pi_{2\text{BF}}$, we again bias the random choices made during encryption. Nevertheless, we note that the adaptation is not much identical to our PKAE constructions, and involves some subtleties. Elaborating on this, in our anamorphic extension of the ElGamal encryption scheme, we embed amsg within two regular ciphertexts $(\text{ct}_1, \text{ct}_2)$ by sampling their first components $r_1 = g^{\kappa_1}$ and $r_2 = g^{\kappa_2}$ such that $\kappa_2 := \alpha \cdot \kappa_1 + \text{amsg}$, where κ_1 is a uniformly random element of $\mathbb{Z}_q$. We then recover amsg by treating (r_1, r_2) as an ElGamal ciphertext, which works because r_2 is explicitly available in ct_2 . However, embedding amsg within two regular Boneh-Franklin ciphertexts is not that straightforward. Observe that, in Boneh-Franklin ciphertext $\text{ct}_2 = (r_2 = P^{\kappa_2}, c_2 = \text{msg} \oplus H_2(g_{\text{id}}^{\kappa_2}))$, the group element $g_{\text{id}}^{\kappa_2}$ is not explicitly available. Rather, it only appears as an input to the hash function H_2 , which is modeled as a random oracle in the security proof. Therefore, if we set κ_2 following the ElGamal encryption extension, then it seems impossible to recover the covert amsg due to the one-wayness of H_2 . At this point, the only solution appears to be encoding amsg outside the hash function H_2 . Then, a natural idea would be to apply rejection sampling technique such that amsg could be extracted from c_2 at the time of decryption. More specifically, we sample κ_2 subject to $H_2(g_{\text{id}}^{\kappa_2}) = \text{amsg} \oplus H_2(\hat{g}_{\text{id}}^{\kappa_1})$; call it $\hat{c}$. We then treat $(r_1 = P^{\kappa_1}, \hat{c} = \text{amsg} \oplus H_2(\hat{g}_{\text{id}}^{\kappa_1}))$ as a Boneh-Franklin ciphertext and apply the regular decryption algorithm **BF.Dec** to recover amsg . While we present a 2-IBAE construction that supports anamorphic messages of single bits for the simplicity of exposition, nevertheless we remark that one could extend our construction to convey at most $O(\log \lambda)$ bit-long anamorphic messages to ensure that encryption remains polynomial.

We now present more details. The Boneh-Franklin 2-IBAE extension $\Pi_{\text{aBF}} = (\text{aSetup}, \text{aKeyGen}, \text{aEnc}, \text{aDec})$ works as follows: the anamorphic key generation algorithm **aBF.aSetup** initially sets $(\text{ampk}, \text{amsk}) = (\text{mpk}, \text{msk})$ as in the regular **BF.Setup**, following which it samples a uniformly random $\alpha \in \mathbb{Z}_q^*$, and sets the double master secret key $\text{dmsk} := \alpha$ and the double master public key $\text{dmpk} := P^\alpha$. The anamorphic key generation algorithm **aBF.aKeyGen** takes as input dmsk and the user identity id and generates the double private key dsk_{id} associated with id . On input a regular message vector $\text{msg} = (\text{msg}_1, \text{msg}_2) \in \mathcal{M}^2$, and an anamorphic message $\text{amsg} \in \widehat{\mathcal{M}}$, where $\mathcal{M} = \widehat{\mathcal{M}} = \{0, 1\}$, the anamorphic encryption algorithm **aBF.aEnc** computes $\text{act}_1 = (r_1, c_1)$

using a uniformly random κ_1 on msg_1 as $\text{BF.Enc}(\text{ampk}, \text{msg}_1, \text{id}; \kappa_1)$. However, for encoding amsg in c_2 (second component of act_2), the algorithm chooses κ_2 subject to the constraint $H_2(g_{\text{id}}^{\kappa_2}) = \text{amsg} \oplus H_2(e(Q_{\text{id}}, \text{ampk})^{\kappa_1})$ and computes $\text{act}_2 = (r_2, c_2)$ as $\text{BF.Enc}(\text{ampk}, \text{msg}_2, \text{id}; \kappa_2)$. Finally, the anamorphic ciphertext vector is $\mathbf{act} = (\text{act}_1, \text{act}_2)$. To recover amsg , the anamorphic decryption algorithm aBF.aDec first extracts $H_2(g_{\text{id}}^{\kappa_2})$ by computing $H_2(e(\text{ask}_{\text{id}}, r_2))$. Note that $H_2(g_{\text{id}}^{\kappa_2})$ is set to be equal to $\text{amsg} \oplus H_2(e(Q_{\text{id}}, \text{ampk})^{\kappa_1})$, call it $\hat{c}$. Then it computes BF.Dec on $\hat{r} = r_1$ and $\hat{c}$. Recall that the dictator has no information on the double private key dsk_{id} associated with id , and hence is oblivious to the presence of amsg . We present our construction in Figure 4.

Correctness, Compatibility and Security Analysis. We provide proofs of correctness, compatibility, anamorphic security and (s)IND-CPA security of Π_{aBF} below.

Correctness: We now show the correctness of the $\Pi_{\text{aBF}} = (\text{aSetup}, \text{aKeyGen}, \text{aEnc}, \text{aDec})$ scheme based on the Boneh-Franklin IBE scheme. Let $\mathbf{act} = (\text{act}_1, \text{act}_2)$ be the output of $\text{aEnc}(\text{ampk}, \text{dmpk}, \text{id}, \text{msg}, \text{amsg})$, where $\text{act}_1 = (r_1, c_1)$ and $\text{act}_2 = (r_2, c_2)$. Here, $r_1 = P^{\kappa_1}$ and $r_2 = P^{\kappa_2}$ are generated using a uniformly random κ_1 in $\mathbb{Z}_q^*$, while κ_2 is such that $H_2(g_{\text{id}}^{\kappa_2}) = \text{amsg} \oplus H_2(\hat{g}_{\text{id}}^{\kappa_1})$, where $\hat{g}_{\text{id}} := e(Q_{\text{id}}, \text{dmpk} = P^\alpha)$ and $g_{\text{id}} := e(Q_{\text{id}}, P_{\text{pub}} = P^s)$. The anamorphic decryption algorithm aBF.aDec knows both the anamorphic private key $\text{ask}_{\text{id}} = Q_{\text{id}}^s$ and the double private key $\text{dsk}_{\text{id}} = Q_{\text{id}}^\alpha$. It first calls $F(\text{ask}_{\text{id}}, \mathbf{act})$, which extracts $H_2(g_{\text{id}}^{\kappa_2})$ by computing $H_2(e(\text{ask}_{\text{id}}, r_2))$. Note that $e(\text{ask}_{\text{id}}, r_2) = e(Q_{\text{id}}^s, P^{\kappa_2}) = e(Q_{\text{id}}, P^s)^{\kappa_2} = (g_{\text{id}}^{\kappa_2})$. Finally, amsg is recovered by executing BF.Dec on $\hat{r} = r_1$ and $\hat{c} = H_2(g_{\text{id}}^{\kappa_2}) = \text{amsg} \oplus H_2(\hat{g}_{\text{id}}^{\kappa_1})$. To see why it works, observe that $\text{BF.Dec}(\text{dsk}_{\text{id}}, F(\text{ask}_{\text{id}}, \mathbf{act}))$ computes $\hat{c} \oplus H_2(e(\text{dsk}_{\text{id}}, \hat{r})) = \text{amsg} \oplus H_2(\hat{g}_{\text{id}}^{\kappa_1}) \oplus H_2(e(Q_{\text{id}}^\alpha, P^{\kappa_1})) = \text{amsg} \oplus H_2(\hat{g}_{\text{id}}^{\kappa_1}) \oplus H_2(e(Q_{\text{id}}, P^\alpha)^{\kappa_1}) = \text{amsg}$.

Lemma 4 *Boneh-Franklin is IBAE-compatible for Π_{aBF} described in Figure 4 associated with the Boneh-Franklin 2-message encryption scheme $\Pi_{2\text{BF}}$.*

Proof. First, observe that $\text{aBF.aSetup}(1^\lambda)$ outputs $\text{amsk} = s$, $\text{ampk} = (\mathbb{G}, \mathbb{G}_T, P, e, q, P_{\text{pub}} = P^s, H_1, H_2)$, $\text{dmsk} = \alpha$ and $\text{dmpk} = P^\alpha$, where s and α are uniformly random in $\mathbb{Z}_q^*$. Therefore, the key pair $(\text{ampk}, \text{amsk})$ has identical distributions with $\{(\text{mpk}, \text{msk}) : \text{msk} = \tilde{s}, \text{mpk} = (\mathbb{G}', \mathbb{G}'_T, P', e', q', P'_{\text{pub}} = P'^s, H'_1, H'_2)\}$ such that $\tilde{s} \leftarrow \mathbb{Z}_q^*$, which is essentially the distribution of master secret and master public keys of a Boneh-Franklin IBE scheme. Additionally, it is easy to see that the double key pair $(\text{dmpk}, \text{dmsk})$ can be efficiently obtained from the knowledge of only ampk . Hence, the IBAE satisfies *on-the-fly double key generation* property. Next, observe that the function F defined in Figure 4 takes as input $\mathbf{act} \leftarrow \text{aBF.aEnc}(\text{ampk}, \text{dmpk}, \text{id}, \text{msg}, \text{amsg})$, and parses it as $\text{act}_1 = (r_1, c_1)$ and $\text{act}_2 = (r_2, c_2)$. It then extracts r_1 (call it $\hat{r}$) and the component $H_2(g_{\text{id}}^{\kappa_2}) = \text{amsg} \oplus H_2(\hat{g}_{\text{id}}^{\kappa_1})$ (call it $\hat{c}$). Note that $(\hat{r}, \hat{c})$ is a valid Boneh-Franklin ciphertext as $\hat{r} = r_1$ and $\hat{c} = \text{amsg} \oplus H_2(\hat{g}_{\text{id}}^{\kappa_1})$. Thus, amsg can be extracted by executing BF.Dec using $\text{dsk}_{\text{id}} = Q_{\text{id}}^\alpha$, which implies that the IBAE satisfies the *ciphertext extractability* property. Hence we conclude that the Boneh-Franklin IBE is IBAE-compatible. $\square$

Theorem 4 *Under the DBDH assumption for $(\mathbb{G}, \mathbb{G}_T, P, e, q)$, the Boneh-Franklin 2-message identity-based anamorphic encryption extension Π_{aBF} described in Figure 4 satisfies anamorphic, fully asymmetric IND-CPA and sIND-CPA security.*

Proof. We prove the theorem in three parts.

<u>aSetup(1^λ) :</u> 1. $(\text{mpk} := (\mathbb{G}, \mathbb{G}_T, P, e, q, P_{\text{pub}}, H_1, H_2), \text{msk} := s) \leftarrow \text{BF.Setup}(1^\lambda)$ 2. $\alpha \leftarrow \mathbb{Z}_q^*$ 3. $\text{ampk} := \text{mpk}, \text{amsk} := \text{msk}$ 4. $\text{dmpk} := P^\alpha, \text{dmsk} := \alpha$ 5. return $(\text{ampk}, \text{amsk}, \text{dmpk}, \text{dmsk})$ <u>aKeyGen(dmsk, id) :</u> 1. $Q_{\text{id}} := H_1(\text{id})$ 2. $\text{dsk}_{\text{id}} := Q_{\text{id}}^\alpha$ 3. return dsk_{id} <u>aDec($\text{ask}_{\text{id}}, \text{dsk}_{\text{id}}, \text{act}$) :</u> 1. $\text{amsg} := \text{BF.Dec}(\text{dsk}_{\text{id}}, F(\text{ask}_{\text{id}}, \text{act}))$ 2. return amsg	<u>aEnc($\text{ampk}, \text{dmpk}, \text{id}, \text{msg}, \text{amsg}$) :</u> 1. Parse $\text{msg} = (\text{msg}_1, \text{msg}_2)$ 2. $Q_{\text{id}} := H_1(\text{id})$ 3. $\hat{g}_{\text{id}} := e(Q_{\text{id}}, \text{dmpk}), g_{\text{id}} := e(Q_{\text{id}}, P_{\text{pub}})$ 4. $\kappa_1, \kappa_2 \leftarrow \mathbb{Z}_q^*$ s.t. $H_2(g_{\text{id}}^{\kappa_2}) = \text{amsg} \oplus H_2(\hat{g}_{\text{id}}^{\kappa_1})$ 5. $\text{act}_1 := \text{BF.Enc}(\text{ampk}, \text{msg}_1, \text{id}; \kappa_1)$ $// \text{act}_1 = (r_1, c_1)$ $// r_1 = P^{\kappa_1}, c_1 = \text{msg}_1 \oplus H_2(g_{\text{id}}^{\kappa_1})$ 6. $\text{act}_2 := \text{BF.Enc}(\text{ampk}, \text{msg}_2, \text{id}; \kappa_2)$ $// \text{act}_2 = (r_2, c_2)$ $// r_2 = P^{\kappa_2}, c_2 = \text{msg}_2 \oplus H_2(g_{\text{id}}^{\kappa_2})$ 7. return $\text{act} := (\text{act}_1, \text{act}_2)$ <u>$F(\text{ask}_{\text{id}}, \text{act})$:</u> 1. Parse $\text{act} = (\text{act}_1, \text{act}_2)$ 2. Parse $\text{act}_1 = (\mathbf{r}_1, c_1),$ $\text{act}_2 = (\mathbf{r}_2, c_2)$ 3. $H_2(e(\text{ask}_{\text{id}}, r_2)) = H_2(g_{\text{id}}^{\kappa_2}) = \text{amsg} \oplus H_2(\hat{g}_{\text{id}}^{\kappa_1})$ $// \text{ask}_{\text{id}} \leftarrow \text{BF.KeyGen}(\text{amsk}, \text{id})$ 4. $\hat{r} := r_1, \hat{c} := \text{amsg} \oplus H_2(\hat{g}_{\text{id}}^{\kappa_1})$ 5. return $(\hat{r}, \hat{c})$
---	--

Figure 4: 2-IBAE Π_{aBF}

Anamorphic security. We use a sequence of hybrid games **Game 0**, **Game** $(1, j)$ for $j \in [0, Q]$ to prove this theorem. In each hybrid game, we assume $\mathcal{D}$ is a PPT dictator and Q be the total number of queries made by $\mathcal{D}$ to the encryption oracle.

Game 0: This is the anamorphic $\text{GAnam}_{2\text{BF}}^{\mathcal{D}}(\lambda)$. The challenger generates $(\text{dmsk} = \alpha, \text{dmpk} =$

$P^\alpha) \leftarrow \text{aBF.aSetup}(1^\lambda)$, and sends the key pair $(\text{ampk}, \text{amsk})$ to $\mathcal{D}$. Then, $\mathcal{D}$ makes polynomial number of queries to the encryption oracle with a tuple $\{(\text{id}_i, \text{msg}_i, \text{amsg}_i)\}_{i \in [Q]}$. At the i -th query, $\mathcal{D}$ receives a ciphertext act_i computed as follows:

1. Parse $\text{msg}_i = (\text{msg}_{i_1}, \text{msg}_{i_2})$
2. $Q_{\text{id}_i} := H_1(\text{id}_i)$
3. $\hat{g}_{\text{id}_i} := e(Q_{\text{id}_i}, \text{dmpk}), g_{\text{id}_i} := e(Q_{\text{id}_i}, P_{\text{pub}})$
4. $\kappa_1, \kappa_2 \leftarrow \mathbb{Z}_q^*$ s.t. $H_2(g_{\text{id}_i}^{\kappa_2}) = \text{amsg}_i \oplus H_2(\hat{g}_{\text{id}_i}^{\kappa_1})$
5. $\text{act}_1 := \text{BF.Enc}(\text{ampk}, \text{msg}_{i_1}, \text{id}_i; \kappa_1)$
6. $\text{act}_2 := \text{BF.Enc}(\text{ampk}, \text{msg}_{i_2}, \text{id}_i; \kappa_2)$
7. **return** $\text{act}_i := (\text{act}_{i_1}, \text{act}_{i_2})$

Game $(1, j)$ for $j \in [0, Q]$: We define Game $(1, 0)$ to be identical to Game 0, and Game $(1, j)$ is exactly the same as Game $(1, 0)$ except that the first j (anamorphic) ciphertexts $\{\text{act}_i\}_{i \in [1, j]}$ are computed as follows:

1. Parse $\text{msg}_i = (\text{msg}_{i_1}, \text{msg}_{i_2})$
2. $\kappa_1, \kappa_2 \leftarrow \mathbb{Z}_q^*$.
3. $\text{act}_{i_1} := \text{BF.Enc}(\text{ampk}, \text{msg}_{i_1}, \text{id}_i; \kappa_1) = (r_1 = P^{\kappa_1}, c_1 = \text{msg}_{i_1} \oplus H_2(g_{\text{id}_i}^{\kappa_1}))$
4. $\text{act}_{i_2} := \text{BF.Enc}(\text{ampk}, \text{msg}_{i_2}, \text{id}_i; \kappa_2) = (r_2 = P^{\kappa_2}, c_2 = \text{msg}_{i_2} \oplus H_2(g_{\text{id}_i}^{\kappa_2}))$
5. **return** $\text{act}_i := (\text{act}_{i_1}, \text{act}_{i_2})$

We show that Game $(1, j)$ is indistinguishable from Game $(1, j - 1)$ by the DBDH assumption. The only difference between Game $(1, j)$ and Game $(1, j - 1)$ is in the computation of κ_1, κ_2 . In particular, κ_1, κ_2 are sampled from $\mathbb{Z}_q^*$ such that $H_2(g_{\text{id}_i}^{\kappa_2}) = \text{amsg}_i \oplus H_2(\hat{g}_{\text{id}_i}^{\kappa_1})$ in Game $(1, j - 1)$, whereas κ_1, κ_2 are sampled uniformly random from $\mathbb{Z}_q^*$ in Game $(1, j)$. We first show that $\hat{g}_{\text{id}_i}^{\kappa_1}$ is indistinguishable from a uniformly random element in $\mathbb{G}_T$ in $\mathcal{D}$'s view. Since H_1 is modeled as ROM, we can set $Q_{\text{id}_i} = P^r$ for some $r \leftarrow \mathbb{Z}_q$. Without loss of generality, we assume that κ_1 is first sampled uniformly at random from $\mathbb{Z}_q$ and then κ_2 is picked from $\mathbb{Z}_q^*$ such that $H_2(g_{\text{id}_i}^{\kappa_2}) = \text{amsg}_i \oplus H_2(\hat{g}_{\text{id}_i}^{\kappa_1})$ holds. Thus the challenger can simulate Game $(1, j)$ with $(P^r, P^\alpha, P^{\kappa_1}, e(P, P)^t)$ when $t = r\alpha\kappa_1$, and Game $(1, j)$, when t is uniformly random in $\mathbb{Z}_q$, where it sets $\hat{g}_{\text{id}_i}^{\kappa_1} = e(P, P)^t$. Therefore, by the DBDH assumption the following holds:

$$\begin{aligned} & |\Pr[r, \alpha, \kappa_1 \leftarrow \mathbb{Z}_q : \mathcal{A}(P^r, P^\alpha, P^{\kappa_1}, e(P, P)^{r \cdot \alpha \cdot \kappa_1}) = 1] - \\ & \Pr[r, \alpha, \kappa_1, t \leftarrow \mathbb{Z}_q : \mathcal{A}(P^r, P^\alpha, P^{\kappa_1}, e(P, P)^t) = 1]| \leq \text{negl}(\lambda) \end{aligned}$$

Since $t = r \cdot \alpha \cdot \kappa_1$ is indistinguishable from uniformly random and H_2 is modeled as ROM, $H_2(\hat{g}_{\text{id}_i}^{\kappa_1}) \oplus \text{amsg}_i$ is statistically close to $H_2(\hat{g}_{\text{id}_i}^{\kappa_1})$. Furthermore, at this stage, $g_{\text{id}_i}^{\kappa_2}$ and $\hat{g}_{\text{id}_i}^{\kappa_1}$ are independently distributed where $\kappa_1, \kappa_2 \leftarrow \mathbb{Z}_q^*$ and hence the hybrids Game $(1, j)$ and Game $(1, j - 1)$ are indistinguishable.

Finally, we observe that Game $(1, Q)$ is exactly the same as $\text{GReal}_{\text{BF}}^{\mathcal{D}}(\lambda)$. This completes the proof.

Fully asymmetric IND-CPA security. We prove the security using hybrid games $\text{Game } 0, \dots, \text{Game } 3$. In each hybrid, we assume $\mathcal{A}$ is a PPT adversary.

Game 0: This is the IND-CPA security experiment $\text{GIND-CPA}_{\text{PKAE}}^A(\lambda, 0)$ defined as follows:

- 1: $(\text{mpk} := (\mathbb{G}, \mathbb{G}_T, P, e, q, P_{\text{pub}}, H_1, H_2), \text{msk} := s) \leftarrow \text{BF.Setup}(1^\lambda)$
- 2: $\alpha \leftarrow \mathbb{Z}_q^*$
- 3: $\text{ampk} := \text{mpk}, \text{amsk} := \text{msk}$
- 4: $\text{dmpk} := P^\alpha, \text{dmsk} := \alpha, \mathcal{S} = \emptyset, \mathcal{K} = \emptyset$
- 5: Key queries: $\mathcal{A}(\text{ampk}, \text{dmpk})$ sends id
 $\mathcal{A}$ gets $(H_1(\text{id})^{\text{amsk}}, H_1(\text{id})^{\text{dmsk}})$
 $\mathcal{S} \leftarrow \mathcal{S} \cup \{\text{id}\}, \mathcal{K} \leftarrow \mathcal{K} \cup \{(H_1(\text{id})^{\text{amsk}}, H_1(\text{id})^{\text{dmsk}})\}$
- 6: $(\text{id}^*, \text{msg}_0, \text{msg}_1, \text{amsg}_0, \text{amsg}_1) \leftarrow \mathcal{A}(\text{ampk}, \text{dmpk}, \mathcal{S}, \mathcal{K})$
- 7: Parse $\text{msg}_0 = (\text{msg}_{0,1}, \text{msg}_{0,2})$
- 8: Parse $\text{msg}_1 = (\text{msg}_{1,1}, \text{msg}_{1,2})$
- 9: $Q_{\text{id}^*} := H_1(\text{id}^*)$
- 10: $\hat{g}_{\text{id}^*} := e(Q_{\text{id}^*}, \text{dmpk}), g_{\text{id}^*} := e(Q_{\text{id}^*}, P_{\text{pub}})$
- 11: $\kappa_1, \kappa_2 \leftarrow \mathbb{Z}_q^*$ s.t. $H_2(g_{\text{id}^*}^{\kappa_2}) = \text{amsg}_0 \oplus H_2(\hat{g}_{\text{id}^*}^{\kappa_1})$
- 12: $\text{act}_1 = (r_1, c_1) := \text{BF.Enc}(\text{ampk}, \text{msg}_{0,1}, \text{id}; \kappa_1)$
 $// r_1 = P^{\kappa_1}, c_1 = \text{msg}_{0,1} \oplus H_2(g_{\text{id}^*}^{\kappa_1})$
- 13: $\text{act}_2 = (r_2, c_2) := \text{BF.Enc}(\text{ampk}, \text{msg}_{0,2}, \text{id}; \kappa_2)$
 $// r_2 = P^{\kappa_2}, c_2 = \text{msg}_{0,2} \oplus H_2(g_{\text{id}^*}^{\kappa_2})$
- 14: Repeat step 5
- 15: **return** $\beta' \leftarrow \mathcal{A}(\text{act}^* = (\text{act}_1^*, \text{act}_2^*))$ if $\text{id}^* \notin \mathcal{S}$

Game 1: This is the same as Game 0 except that the challenger replaces $\text{amsg}_0 \oplus H_2(\hat{g}_{\text{id}^*}^{\kappa_1})$ with a uniformly random bit c^* . In particular, the challenger samples $\kappa_2 \leftarrow \mathbb{Z}_q^*, c^* \leftarrow \{0, 1\}$ and sets $H_2(g_{\text{id}^*}^{\kappa_2}) = c^*$. The indistinguishability between Game 0 and Game 1 follows from the IND\$-CPA security of the Boneh-Franklin BasicIdent scheme⁵ under the public key dmpk .

We show that if there is an adversary $\mathcal{A}$ that breaks the indistinguishability between the two games, then there exists another adversary $\mathcal{B}$ which breaks the IND\$-CPA security of the Boneh-Franklin BasicIdent scheme. More specifically, $\mathcal{B}$, having received $\text{mpk} = (\mathbb{G}, \mathbb{G}_T, P, e, q, P_{\text{pub}}, H_1, H_2)$ from its challenger, simulates the adversary $\mathcal{A}$ as follows:

Simulator $\mathcal{B}(1^\lambda, \text{mpk})$

- 1: $s \leftarrow \mathbb{Z}_q^*$
- 2: $\text{ampk} := (\mathbb{G}, \mathbb{G}_T, P, e, q, P_{\text{pub}} = P^s, H_1, H_2), \text{amsk} := s$

⁵It is easy to see that Boneh-Franklin BasicIdent scheme satisfies the IND\$-CPA security under the BDH problem [BF01], which ensures that the challenge ciphertext can be sampled uniformly at random from the ciphertext space $\mathbb{G} \times \{0, 1\}$.

- 3: $\text{dmpk} := P^\alpha, \mathcal{S} = \emptyset, \mathcal{K} = \emptyset$
- 4: Key queries: $\mathcal{A}(\text{ampk}, \text{dmpk})$ sends id
 - $\mathcal{B}$ sends id to its challenger and gets dsk_{id}
 - $\mathcal{A}$ gets $(H_1(\text{id})^{\text{amsk}}, \text{dsk}_{\text{id}})$
 - $\mathcal{S} \leftarrow \mathcal{S} \cup \{\text{id}\}, \mathcal{K} \leftarrow \mathcal{K} \cup \{(H_1(\text{id})^{\text{amsk}}, \text{dsk}_{\text{id}})\}$
- 5: $(\text{id}^*, \mathbf{msg}_0, \mathbf{msg}_1, \text{amsg}_0, \text{amsg}_1) \leftarrow \mathcal{A}(\text{ampk}, \text{dmpk}, \mathcal{S}, \mathcal{K})$
- 6: Parse $\mathbf{msg}_0 = (\text{msg}_{0,1}, \text{msg}_{0,2})$
- 7: Parse $\mathbf{msg}_1 = (\text{msg}_{1,1}, \text{msg}_{1,2})$
- 8: $\mathcal{B}$ sends $(\text{id}^*, \text{amsg}_0, \text{amsg}_1)$ to its challenger and gets (r^*, c^*)
- 9: $Q_{\text{id}^*} := H_1(\text{id}^*)$
- 10: $g_{\text{id}^*} := e(Q_{\text{id}^*}, P_{\text{pub}}) = e(Q_{\text{id}^*}, P)^{\text{amsk}}$
- 11: $\kappa_2 \leftarrow \mathbb{Z}_q^*$ s.t. $H_2(g_{\text{id}^*}^{\kappa_2}) = c^*$
- 12: $\text{act}_1 := (r_1, c_1)$ where $r_1 := r^*, c_1 := \text{msg}_{0,1} \oplus H_2(e(Q_{\text{id}^*}, r^*)^{\text{amsk}})$
- 13: $\text{act}_2 = (r_2, c_2) := \text{BF.Enc}(\text{ampk}, \text{msg}_{0,2}, \text{id}; \kappa_2)$
- // $r_2 = P^{\kappa_2}, c_2 = \text{msg}_{0,2} \oplus H_2(g_{\text{id}^*}^{\kappa_2})$
- 14: Repeat step 5
- 15: $\mathcal{B}$ simulates all the hash queries of $\mathcal{A}$ using its own challenger
- 16: **return** $\beta' \leftarrow \mathcal{A}(\text{act}^* = (\text{act}_1^*, \text{act}_2^*))$ if $\text{id}^* \notin \mathcal{S}$

It can be observed that $\mathcal{B}$ perfectly simulates $\mathcal{A}$. Firstly, if (r^*, c^*) takes the form $(r^* = P^{\kappa_1}, c^* = \text{amsg}_0 \oplus H_2(\hat{g}_{\text{id}^*}^{\kappa_1}))$, i.e. the BasicIdent scheme encrypts amsg_0 using mpk (or dmpk in the view of $\mathcal{A}$), then $\mathcal{B}$ simulates Game 0 for $\mathcal{A}$. The only difference is in step 12, where act_1 is computed as $\text{act}_1 = (r_1, c_1)$. However, we observe that

$$r_1 = r^* = P^{\kappa_1}, c_1 = \text{msg}_{0,1} \oplus H_2(e(Q_{\text{id}^*}, r^*)^{\text{amsk}}) = \text{msg}_{0,1} \oplus H_2(g_{\text{id}^*}^{\kappa_1})$$

which is a valid ciphertext encrypting $\text{msg}_{0,1}$ under ampk . On the other hand, if the BasicIdent scheme picks a random ciphertext (r^*, c^*) in the step 8, then $\mathcal{B}$ samples $\kappa_2 \leftarrow \mathbb{Z}_q^*$ such that $H_2(g_{\text{id}^*}^{\kappa_2}) = c^*$ in step 11. In this case, $\mathcal{B}$ simulates Game 1 for $\mathcal{A}$. Hence, the indistinguishability between Game 0 and Game 1 follows from the IND\$-CPA security of the Boneh-Franklin BasicIdent scheme under the public key mpk .

Game 2: This is the same as Game 1 except that the challenger encrypts $\mathbf{msg}_1$ instead of $\mathbf{msg}_0$:

- 1: $(\text{mpk} := (\mathbb{G}, \mathbb{G}_T, P, e, q, P_{\text{pub}}, H_1, H_2), \text{msk} := s) \leftarrow \text{BF.Setup}(1^\lambda)$
- 2: $\alpha \leftarrow \mathbb{Z}_q^*$
- 3: $\text{ampk} := \text{mpk}, \text{amsk} := \text{msk}$
- 4: $\text{dmpk} := P^\alpha, \text{dmsk} := \alpha, \mathcal{S} = \emptyset, \mathcal{K} = \emptyset$
- 5: Key queries: $\mathcal{A}(\text{ampk}, \text{dmpk})$ sends id
 - $\mathcal{A}$ gets $(H_1(\text{id})^{\text{amsk}}, H_1(\text{id})^{\text{dmsk}})$
 - $\mathcal{S} \leftarrow \mathcal{S} \cup \{\text{id}\}, \mathcal{K} \leftarrow \mathcal{K} \cup \{(H_1(\text{id})^{\text{amsk}}, H_1(\text{id})^{\text{dmsk}})\}$

6: $(\text{id}^*, \mathbf{msg}_0, \mathbf{msg}_1, \text{amsg}_0, \text{amsg}_1) \leftarrow \mathcal{A}(\text{ampk}, \text{dmpk}, \mathcal{S}, \mathcal{K})$
 7: Parse $\mathbf{msg}_0 = (\text{msg}_{0,1}, \text{msg}_{0,2})$
 8: Parse $\mathbf{msg}_1 = (\text{msg}_{1,1}, \text{msg}_{1,2})$
 9: $Q_{\text{id}^*} := H_1(\text{id}^*)$
 10: $\hat{g}_{\text{id}^*} := e(Q_{\text{id}^*}, \text{dmpk}), g_{\text{id}^*} := e(Q_{\text{id}^*}, P_{\text{pub}})$
 11: $\kappa_1, \kappa_2 \leftarrow \mathbb{Z}_q^*, c^* \leftarrow \{0, 1\}$ s.t. $H_2(g_{\text{id}^*}^{\kappa_2}) = c^*$
 12: $\text{act}_1 = (r_1, c_1) := \text{BF.Enc}(\text{ampk}, \text{msg}_{1,1}, \text{id}; \kappa_1)$
 // $r_1 = P^{\kappa_1}, c_1 = \text{msg}_{1,1} \oplus H_2(g_{\text{id}^*}^{\kappa_1})$
 13: $\text{act}_2 = (r_2, c_2) := \text{BF.Enc}(\text{ampk}, \text{msg}_{1,2}, \text{id}; \kappa_2)$
 // $r_2 = P^{\kappa_2}, c_2 = \text{msg}_{1,2} \oplus H_2(g_{\text{id}^*}^{\kappa_2})$
 14: Repeat step 5
 15: **return** $\beta' \leftarrow \mathcal{A}(\mathbf{act}^* = (\text{act}_1^*, \text{act}_2^*))$ if $\text{id}^* \notin \mathcal{S}$

The indistinguishability between Game 1 and Game 2 follows the IND-CPA security of the Boneh-Franklin BasicIdent scheme, in particular from the BDH assumption.

Game 3: This is the same as Game 2 except that the challenger sets $c^* = \text{amsg}_1 \oplus H_2(\hat{g}_{\text{id}^*}^{\kappa_1})$ instead of sampling it uniformly at random from $\{0, 1\}$:

1: $(\text{mpk} := (\mathbb{G}, \mathbb{G}_T, P, e, q, P_{\text{pub}}, H_1, H_2), \text{msk} := s) \leftarrow \text{BF.Setup}(1^\lambda)$
 2: $\alpha \leftarrow \mathbb{Z}_q^*$
 3: $\text{ampk} := \text{mpk}, \text{amsk} := \text{msk}$
 4: $\text{dmpk} := P^\alpha, \text{dmsk} := \alpha, \mathcal{S} = \emptyset, \mathcal{K} = \emptyset$
 5: Key queries: $\mathcal{A}(\text{ampk}, \text{dmpk})$ sends id
 $\mathcal{A}$ gets $(H_1(\text{id})^{\text{amsk}}, H_1(\text{id})^{\text{dmsk}})$
 $\mathcal{S} \leftarrow \mathcal{S} \cup \{\text{id}\}, \mathcal{K} \leftarrow \mathcal{K} \cup \{(H_1(\text{id})^{\text{amsk}}, H_1(\text{id})^{\text{dmsk}})\}$
 6: $(\text{id}^*, \mathbf{msg}_0, \mathbf{msg}_1, \text{amsg}_0, \text{amsg}_1) \leftarrow \mathcal{A}(\text{ampk}, \text{dmpk}, \mathcal{S}, \mathcal{K})$
 7: Parse $\mathbf{msg}_0 = (\text{msg}_{0,1}, \text{msg}_{0,2})$
 8: Parse $\mathbf{msg}_1 = (\text{msg}_{1,1}, \text{msg}_{1,2})$
 9: $Q_{\text{id}^*} := H_1(\text{id}^*)$
 10: $\hat{g}_{\text{id}^*} := e(Q_{\text{id}^*}, \text{dmpk}), g_{\text{id}^*} := e(Q_{\text{id}^*}, P_{\text{pub}})$
 11: $\kappa_1, \kappa_2 \leftarrow \mathbb{Z}_q^*$ s.t. $H_2(g_{\text{id}^*}^{\kappa_2}) = \text{amsg}_1 \oplus H_2(\hat{g}_{\text{id}^*}^{\kappa_1})$
 12: $\text{act}_1 = (r_1, c_1) := \text{BF.Enc}(\text{ampk}, \text{msg}_{1,1}, \text{id}; \kappa_1)$
 // $r_1 = P^{\kappa_1}, c_1 = \text{msg}_{1,1} \oplus H_2(g_{\text{id}^*}^{\kappa_1})$
 13: $\text{act}_2 = (r_2, c_2) := \text{BF.Enc}(\text{ampk}, \text{msg}_{1,2}, \text{id}; \kappa_2)$
 // $r_2 = P^{\kappa_2}, c_2 = \text{msg}_{1,2} \oplus H_2(g_{\text{id}^*}^{\kappa_2})$
 14: Repeat step 5
 15: **return** $\beta' \leftarrow \mathcal{A}(\mathbf{act}^* = (\text{act}_1^*, \text{act}_2^*))$ if $\text{id}^* \notin \mathcal{S}$

The indistinguishability follows the IND\$-CPA security of the Boneh-Franklin BasicIdent scheme under the public key dmpk similar to Game 1.

Finally, we observe that Game 3 is exactly the same as $\text{GIND-CPA}_{\text{IBAE}}^A(\lambda, 1)$. This concludes the fully asymmetric security of Π_{aBF} .

sIND-CPA security. We prove the strong IND-CPA security by designing a simulator that plays the role of the challenger in the IND-CPA security game of the Boneh-Franklin **BasicIdent** scheme against **ampk**, which is IND-CPA secure under the BDH problem. Note that the anamorphic master public key **ampk** and the master public key of the **BasicIdent** scheme are identically distributed since the Boneh-Franklin **BasicIdent** scheme is IBAE-compatible by Lemma 4. In other words, if an adversary breaks the sIND-CPA security of Π_{IBAE} then it can be used to break the IND-CPA security of Boneh-Franklin **BasicIdent** scheme.

We start with a contradiction that there exists a PPT adversary $\mathcal{A}$ that breaks the sIND-CPA security of Π_{aBF} with a non-negligible probability. We then design another adversary $\mathcal{B}$ that breaks IND-CPA security of the **BasicIdent** scheme. The challenger of $\mathcal{B}$ runs the setup of the **BasicIdent** scheme and returns $\text{mpk} = (\mathbb{G}, \mathbb{G}_T, P, e, q, P^\alpha, H_1, H_2)$ to $\mathcal{B}$. Then, $\mathcal{B}$ utilizes $\mathcal{A}$ to break the IND-CPA security of the **BasicIdent** scheme as follows:

Simulator $\mathcal{B}(1^\lambda, \text{mpk})$

- 1: $s \leftarrow \mathbb{Z}_q^*$
- 2: $\text{ampk} := (\mathbb{G}, \mathbb{G}_T, P, e, q, P_{\text{pub}} = P^s, H_1, H_2)$, $\text{amsk} := s$
- 3: $\text{dmpk} := P^\alpha, \mathcal{S} = \emptyset, \mathcal{K} = \emptyset$
- 4: Key queries: $\mathcal{A}(\text{ampk}, \text{amsk}, \text{dmpk})$ sends id
 - $\mathcal{B}$ sends id to its challenger and gets dsk_{id}
 - $\mathcal{A}$ gets $(H_1(\text{id})^{\text{amsk}}, \text{dsk}_{\text{id}})$
 - $\mathcal{S} \leftarrow \mathcal{S} \cup \{\text{id}\}, \mathcal{K} \leftarrow \mathcal{K} \cup \{(H_1(\text{id})^{\text{amsk}}, \text{dsk}_{\text{id}})\}$
- 5: $(\text{id}^*, \text{msg}, \text{amsg}_0, \text{amsg}_1) \leftarrow \mathcal{A}(\text{ampk}, \text{amsk}, \text{dmpk}, \mathcal{S}, \mathcal{K})$
- 6: Parse $\text{msg} = (\text{msg}_1, \text{msg}_2)$
- 7: $\mathcal{B}$ sends $(\text{id}^*, \text{amsg}_0, \text{amsg}_1)$ to its challenger and gets (r^*, c^*)
- 8: $Q_{\text{id}^*} := H_1(\text{id}^*)$
- 9: $g_{\text{id}^*} := e(Q_{\text{id}^*}, P_{\text{pub}}) = e(Q_{\text{id}^*}, P)^{\text{amsk}}$
- 10: $\kappa_2 \leftarrow \mathbb{Z}_q^*$ s.t. $H_2(g_{\text{id}^*}^{\kappa_2}) = c^*$
- 11: $\text{act}_1 := (r_1, c_1)$ where $r_1 := r^*, c_1 := \text{msg}_1 \oplus H_2(e(Q_{\text{id}^*}, r^*)^{\text{amsk}})$
- 12: $\text{act}_2 := (r_2, c_2) := \text{BF.Enc}(\text{ampk}, \text{msg}_2, \text{id}; \kappa_2)$
- // $r_2 = P^{\kappa_2}, c_2 = \text{msg}_2 \oplus H_2(g_{\text{id}^*}^{\kappa_2})$
- 13: Repeat step 5
- 14: $\mathcal{B}$ simulates all the hash queries of $\mathcal{A}$ using its own challenger
- 15: **return** $\beta' \leftarrow \mathcal{A}(\text{act}^* = (\text{act}_1^*, \text{act}_2^*))$ if $\text{id}^* \notin \mathcal{S}$

It can be observed that if (r^*, c^*) takes the form $(r^* = P^{\kappa_1}, c^* = \text{amsg}_\beta \oplus H_2(\widehat{g}_{\text{id}^*}^{\kappa_1}))$, i.e. the BasicIdent scheme encrypts amsg_β using mpk (or dmpk in the view of $\mathcal{A}$), then $\mathcal{B}$ perfectly simulates $\text{GsIND-CPA}_{\text{IBAE}}^{\mathcal{A}}(\lambda, \beta)$ for $\mathcal{A}$. The only difference is in step 11, where act_1 is computed as $\text{act}_1 = (r_1, c_1)$. However, we observe that

$$r_1 = r^* = P^{\kappa_1}, c_1 = \text{msg}_1 \oplus H_2(e(Q_{\text{id}^*}, r^*)^{\text{amsk}}) = \text{msg}_1 \oplus H_2(g_{\text{id}^*}^{\kappa_1})$$

which is a valid BF ciphertext encrypting msg_1 under ampk . Hence, if $\mathcal{A}$ breaks the sIND-CPA security of Π_{aBF} with a non-negligible probability, then $\mathcal{B}$ can also break the IND-CPA security of the Boneh-Franklin BasicIdent scheme under the public key mpk . This completes the proof. $\square$

6.2 Extension of FO-Based IBE

Variants of the FO transformation are also a viable tool to construct CCA secure IBE schemes. This has started with the initial construction due to Boneh and Franklin on IBE [BF01] and several works have explicitly investigated various FO-type transformations for lifting OW-CPA or IND-CPA secure IBE to CCA secure ones [Gal05, YKH⁺06, KYH⁺06]. Analogous to the discussion of the FO transformation in Section 5.3, it is natural to ask whether we can equivalently apply these results to the IBAE setting. We first want to briefly recall how the variant of the FO transform that is used in the Boneh-Franklin (BF) IBE and the Sakai-Kasahara (SK) IBE [SOK01, CC05] exactly work. We note that both works as specified below are suggested in an ETSI technical report for standardization [ETS22]⁶.

For our presentation let $\Pi_{\text{IBE}} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ and $\Pi_{\text{SKE}} = (\text{Gen}, \text{Enc}, \text{Dec})$ be the underlying IBE and symmetric encryption schemes. Analogous to Section 5.3 we now denote by $\Pi_{\text{FO,IBE}} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ the IND-CCA secure version where $\text{FO}_{\text{IBE}}.\text{Setup}$ and $\text{FO}_{\text{IBE}}.\text{KeyGen}$ just run $\text{IBE}.\text{Setup}$ and $\text{IBE}.\text{KeyGen}$ respectively. $\text{FO}_{\text{IBE}}.\text{Enc}(\text{mpk}, \text{id}, \text{msg})$ returns ciphertext $c = (c_1, c_2)$ where $c_1 = \text{IBE}.\text{Enc}(\text{mpk}, \text{id}, \text{msg}; \sigma)$ and $c_2 = \text{SKE}.\text{Enc}(G(r), \text{msg})$ for uniformly random $r \in \{0, 1\}^n$ (and suitable n) and $\sigma = H(r, \text{msg}) \in \mathcal{M}_{\text{IBE}}$. Note that H and G are hash functions modeled as random oracles mapping to the space of random coins $\mathcal{R}_{\text{IBE}}$ of IBE and the key space $\mathcal{K}_{\text{SKE}}$ of SKE, respectively. For decryption, $\text{FO}_{\text{IBE}}.\text{Dec}(\text{sk}_{\text{id}}, c)$ parses $c = (c_1, c_2)$ computes $r' = \text{IBE}.\text{Dec}(\text{sk}_{\text{id}}, c_1)$ and $\text{msg}' = \text{SKE}.\text{Dec}(G(r'), c_2)$ and returns msg' if $c'_1 = \text{IBE}.\text{Enc}(\text{mpk}, \text{id}, H(r', \text{msg}'))$ or $\perp$ otherwise.

We present the anamorphic IBE extension based on $\Pi_{\text{FO,IBE}}$ in Figure 5.

Theorem 5 *Let $\Pi_{\text{FO,IBE}}$ be an IND-ID-CCA secure FO-based IBE where the underlying IBE Π_{IBE} is IND\$-CPA secure and has an exponentially large message space $\mathcal{M}_{\text{IBE}}$ (i.e., $|\mathcal{M}_{\text{IBE}}| \geq 2^{\text{poly}(\lambda)}$ for non-constant poly), and the underlying SKE Π_{SKE} is wIND\$-CPA secure. Furthermore, let $(\text{Encode}, \text{Decode})$ be a pseudo-random $(\mathcal{C}_{\text{FO,IBE}}, \mathcal{M}_{\text{IBE}}^\mu)$ -encoding, where $\mathcal{C}_{\text{FO,IBE}}$ denotes the ciphertext space of $\Pi_{\text{FO,IBE}}$. Then $\Pi_{\text{FOAE,IBE}}$ as defined in Figure 5 is a μ -message anamorphic identity-based encryption extension satisfying anamorphic security and sIND-RCCA security in the ROM.*

The proof of this theorem works analogous to that of Theorem 3 and is thus omitted. The intuition is that $\Pi_{\text{FO,IBE}}$ satisfies anamorphic security as the embedded ciphertexts components containing amsg

⁶The only difference with the standard, as with the papers, is that they explicitly consider XOR as symmetric encryption scheme SKE.

<u>aSetup(1^λ) :</u> 1. $(\text{amsk}, \text{ampk}) \leftarrow \text{FO}_{\text{IBE}}.\text{Setup}(1^\lambda)$ 2. $(\text{dmsk}, \text{dmpk}) \leftarrow \text{FO}_{\text{IBE}}.\text{Setup}(1^\lambda)$ 3. return $(\text{amsk}, \text{ampk}, \text{dmpk}, \text{dmsk})$ <u>aDec($\text{ask}_{\text{id}}, \text{dsk}_{\text{id}}, \text{act}$) :</u> 1. for $j \in \{1, \dots, \mu\}$: Parse $\text{act}_j = (c_1, c_2)$ $r_j := \text{IBE}.\text{Dec}(\text{ask}_{\text{id}}, c_1)$ if $r_j = \perp$ return $\perp$ 2. $c := \text{Decode}(r_1, \dots, r_\mu)$ 3. if $c = \perp$ return $\perp$ 4. return $\text{FO}_{\text{IBE}}.\text{Dec}(\text{dsk}_{\text{id}}, c)$	<u>aKeyGen(dmsk, id) :</u> 1. $\text{dsk}_{\text{id}} = \text{FO}_{\text{IBE}}.\text{KeyGen}(\text{dmsk}, \text{id})$ 2. return dsk_{id} <u>aEnc($\text{ampk}, \text{dmpk}, \text{id}, \text{msg}, \text{amsg}$) :</u> 1. Parse $\text{msg} = (\text{msg}_1, \dots, \text{msg}_\mu)$ 2. $c \leftarrow \text{FO}_{\text{IBE}}.\text{Enc}(\text{dmpk}, \text{id}, \text{amsg})$ 3. $(r_1, \dots, r_\mu) \leftarrow \text{Encode}(c)$ 4. for $j \in \{1, \dots, \mu\}$: $\text{act}_j := \text{FO}_{\text{IBE}}.\text{Enc}(\text{ampk}, \text{id}, \text{msg}_j; r_j)$ 5. return $\text{act} := (\text{act}_1, \dots, \text{act}_\mu)$
---	--

Figure 5: μ -AIBE $\Pi_{\text{FOAE}, \text{IBE}}$

are indistinguishable from a regular $r \leftarrow \mathcal{M}_{\text{IBE}}$ due to the pseud-randomness of the pseudo-random encoding. Moreover, the IND-CCA security of $\Pi_{\text{FO}, \text{IBE}}$ for encrypting amsg (i.e., the “inner” layer encryption) implies sIND-RCCA for the overall construction $\Pi_{\text{FOAE}, \text{IBE}}$ as an adversary (including the dictator) is not able to maul amsg .

Instantiation with BF Full-Indent. Let us consider $\Pi_{\text{FO}, \text{IBE}}$ based on Boneh-Frankin (BF) as IBE and XOR for the SKE part. BF satisfies IND $\mathbb{S}$ -CPA security under DBDH in the ROM and XORing a message with a random key of the same length satisfies wIND $\mathbb{S}$ -CPA security. When instantiated in type-3 pairings $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, g_1, g_2, q)$, let us assume that c_1 elements are in $\mathbb{G}_1$ and secret keys sk_{id} are in $\mathbb{G}_2$. Note that, $\text{FO}_{\text{IBE}}.\text{Enc}(\text{mpk}, \text{msg})$ would produce a ciphertext $c = (g_1^{H(r, \text{msg})}, H'(g_{2, \text{id}}^{H(r, \text{msg})}) \oplus r, G(r) \oplus \text{msg})$, where $H : \{0, 1\}^* \times \{0, 1\}^* \rightarrow \mathbb{Z}_q$, $H' : \mathbb{G}_T \rightarrow \{0, 1\}^{\ell_1}$, $G : \{0, 1\}^* \rightarrow \{0, 1\}^{\ell_2}$. As for Π_{FOAE} , we can use Elligator Squared [Tib14] as the pseudo-random encoding and considering BN curves [BN06] for instantiating the type-3 pairing we have $\mathbb{G}_1 = \mathbb{E}(\mathbb{F}_p)$ is a cyclic group of prime order q . Moreover, if we again set $\ell_1 = \ell_2 = \lambda$, then we end up with a 4-message anamorphic extension.

7 μ -Message Public Key Anamorphic Signature Extension

In this section, we formalize the notion of μ -message anamorphic extension of a signature scheme along with its security definitions. For brevity, we refrain from defining public key anamorphic signatures first and immediately establish security notions for the extension. To keep our exposition and constructions of this new concept simple, we refrain from considering a robustness notion for

now. However, Section B will informally discuss such a notion and the difficulties to achieve it regarding our designs.

Definition 19 (μ -Message Public Key Anamorphic Signature Extension) A tuple $\Pi_{\text{PKASE}} = (\text{aGenS}, \text{aGenR}, \text{aSig}, \text{aDec})$ of PPT algorithms is a μ -message public key anamorphic signature extension (μ -PKASE) with anamorphic message space $\widehat{\mathcal{M}}$ associated with a signature scheme $\Pi_{\text{SS}} = (\text{Setup}, \text{Gen}, \text{Sig}, \text{Verify})$ with normal message space $\mathcal{M}$ and is defined as follows, where $\text{gp} \leftarrow \text{SS.Setup}(1^\lambda)$:

$(\text{ask}, \text{avk}, \text{td}) \leftarrow \text{aGenS}(\text{gp})$: The anamorphic key generation algorithm aGenS is executed by the sender of the signature, takes as input the global parameters gp generated by Setup of the associated signature scheme, and outputs a signature key pair (ask, avk) ⁷ as well as some secret trapdoor information td .

$(\text{dk}, \text{tk}) \leftarrow \text{aGenR}(\text{gp})$: The anamorphic key generation algorithm aGenR is executed by the receiver of the signature, takes as input the global parameters gp generated by Setup of the associated signature scheme, and outputs a double key pair (dk, tk) .

$\text{asig} \leftarrow \text{aSig}(\text{ask}, \text{td}, \text{dk}, \text{msg}, \text{amsg})$: The anamorphic signing algorithm aSig takes as input the sender's signing key ask with associated trapdoor information td , and the receiver's encryption double key dk , a regular message vector $\text{msg} \in \mathcal{M}^\mu$ and an anamorphic message $\text{amsg} \in \widehat{\mathcal{M}}$, and outputs an anamorphic signature vector asig .

$\text{amsg} \leftarrow \text{aDec}(\text{tk}, \text{avk}, \text{asig}, \text{msg})$: The anamorphic decryption algorithm takes as input the receiver's secret decryption double key tk , the sender's signature verification key avk , and an anamorphic signature vector asig along with a message vector msg , and outputs an anamorphic message amsg or $\perp$.

It must satisfy the following properties.

Correctness: For all $\lambda \in \mathbb{N}$, $\text{gp} \leftarrow \text{Setup}(1^\lambda)$, $(\text{msg}, \text{amsg}) \in \mathcal{M}^\mu \times \widehat{\mathcal{M}}$, it holds that

$$\Pr \left[\text{amsg} \neq \text{aDec}(\text{tk}, \text{avk}, \text{asig}, \text{msg}) : \begin{array}{l} (\text{ask}, \text{avk}, \text{td}) \leftarrow \text{aGenS}(\text{gp}) \\ (\text{dk}, \text{tk}) \leftarrow \text{aGenR}(\text{gp}) \\ \text{asig} \leftarrow \text{aSig}(\text{ask}, \text{td}, \text{dk}, \text{msg}, \text{amsg}) \end{array} \right] \leq \text{negl}(\lambda)$$

where the probability is taken over the random coins of aGenS , aGenR and aSig .

Anamorphic security: For every PPT dictator $\mathcal{D}$ it holds that

$$|\Pr[\text{GReal}_{\text{SS}}^{\mathcal{D}}(\lambda) = 1] - \Pr[\text{GAnam}_{\text{PKASE}}^{\mathcal{D}}(\lambda) = 1]| \leq \text{negl}(\lambda)$$

where the security games are defined as follows, and $\text{Sig}(\text{sk}, \text{msg}) := (\text{Sig}(\text{sk}, \text{msg}))_{\text{msg} \in \text{msg}}$:

⁷We assume that gp is included in vk and avk . It is also available to all the algorithms.

$\text{GReal}_{\text{SS}}^{\mathcal{D}}(\lambda) :$	$\text{GAnam}_{\text{PKASE}}^{\mathcal{D}}(\lambda) :$
1. $\text{gp} \leftarrow \text{Setup}(1^\lambda)$	1. $\text{gp} \leftarrow \text{Setup}(1^\lambda)$
2. $(\text{sk}, \text{vk}) \leftarrow \text{Gen}(\text{gp})$	2. $(\text{ask}, \text{avk}, \text{td}) \leftarrow \text{aGenS}(\text{gp})$
3. return $\mathcal{D}^{\text{OSig}(\cdot, \cdot)}(\text{sk}, \text{vk})$	3. $(\text{dk}, \text{tk}) \leftarrow \text{aGenR}(\text{gp})$
where $\text{OSig}(\text{msg}, \text{amsg}) :=$	4. return $\mathcal{D}^{\text{OaSig}(\cdot, \cdot)}(\text{ask}, \text{avk})$
$\text{Sig}(\text{sk}, \text{msg})$.	where $\text{OaSig}(\text{msg}, \text{amsg}) :=$
	$\text{aSig}(\text{ask}, \text{td}, \text{dk}, \text{msg}, \text{amsg})$.

Anamorphic security roughly guarantees that the two distributions $\{\text{sk}, \text{vk}, \text{Sig}(\text{sk}, \text{msg})\}$ and $\{\text{ask}, \text{avk}, \text{aSig}(\text{ask}_S, \text{dk}_R, \text{msg}, \text{amsg})\}$ are indistinguishable.

Preserving EUF/SUF-CMA security. We note that anamorphic security clearly also holds for an adversary not having access to ask . Such an adversary does not notice whether it interacts with the regular signature scheme or the anamorphic version. This clearly implies that the EUF-CMA or SUF-CMA security of Π_{SS} carries over to the individual signatures of the anamorphic extension.

Choice of gp. As opposed to PKAE, the benefits of deploying *asymmetric* cryptography to realize the anamorphic channel for signatures would be highly reduced if each and every signer chose its own unique gp . In this case, a receiver of an anamorphic message needs to generate an individual key pair (dk, tk) for each signer it wants to receive anamorphic messages from. To this end, we make the realistic assumption that signers only choose from a small set of gp settings, e.g., standardized elliptic curves, or the dictator fixes some gp for everyone.

IND-CPA security of anamorphic signatures. Another natural security property for an anamorphic signature scheme is to guarantee that within a group of people that communicate via this embedded covert channel, no information about a message is leaked to parties other than the intended receiver. More concretely, if a party A knows dk_B of party B because it previously has sent a hidden message to it, then no information about the hidden message contained in an anamorphic signature from party C to party B should be leaked to party A. This notion is captured by the following IND-CPA-style security definition.

Definition 20 (IND-CPA Security for PKASE) A μ -PKASE scheme Π_{PKASE} associated with signature scheme Π_{SS} is said to satisfy IND-CPA security if for every PPT adversary $\mathcal{A}$ it holds that

$$|\Pr[\text{GIND-CPA}_{\text{PKASE}}^{\mathcal{A}}(\lambda, 1) = 1] - \Pr[\text{GIND-CPA}_{\text{PKASE}}^{\mathcal{A}}(\lambda, 0) = 1]| \leq \text{negl}(\lambda)$$

where the security game is defined as follows:

$\text{GIND-CPA}_{\text{PKASE}}^{\mathcal{A}}(\lambda, \beta) :$

1. $\text{gp} \leftarrow \text{Setup}(1^\lambda)$
2. $(\text{ask}, \text{avk}, \text{td}) \leftarrow \text{aGenS}(\text{gp})$
3. $(\text{dk}, \text{tk}) \leftarrow \text{aGenR}(\text{gp})$
4. $(\text{msg}, \text{amsg}_0, \text{amsg}_1) \leftarrow \mathcal{A}^{\text{OaSig}(\cdot, \cdot)}(\text{avk}, \text{dk})$
5. $\text{asig}_\beta \leftarrow \text{aSig}(\text{ask}, \text{td}, \text{dk}, \text{msg}, \text{amsg}_\beta)$
6. **return** $\beta' \leftarrow \mathcal{A}^{\text{OaSig}(\cdot, \cdot)}(\text{asig}_\beta)$ where
 $\text{OaSig}(\text{msg}, \text{amsg}) := \text{aSig}(\text{ask}, \text{td}, \text{dk}, \text{msg}, \text{amsg}).$

(s)IND-(R)CCA security of anamorphic signatures. Similar to the anamorphic encryption case, we can also define stronger security properties for anamorphic signatures by providing a (restricted) aDec oracle or ask as input to $\mathcal{A}$. So we define IND-CCA security by providing an $\text{aDec}(\text{tk}, \text{avk}, \cdot, \cdot)$ oracle (rejecting if queried with challenge $(\text{asig}_\beta, \text{msg})$), sIND-CCA by additionally giving ask as input to $\mathcal{A}$, and sIND-RCCA like sIND-CCA but restricting the oracle to reject if $\text{aDec}(\text{tk}, \text{avk}, \text{asig}, \text{msg}) \in \{\text{amsg}_0, \text{amsg}_1\}$.

7.1 Waters Signatures with Linear Encryption

Subsequently, we present a PKASE construction based on Water's signatures [Wat05] and Linear Encryption (LE) [BBS04]. LE is a variant of ElGamal relying on the Decision linear (DLIN) assumption that can be instantiated in a group $\mathbb{G}$ where the DDH assumption is easy, e.g., in symmetric pairing groups. We consider the Waters scheme over symmetric pairings groups $(\mathbb{G}, \mathbb{G}_T, g, e, q)$ for simplicity, but it can straightforwardly be translated to an asymmetric pairing setting (e.g., type-3 pairings). However, we note that the anamorphic extension still requires LE.⁸

We first recall that Waters signatures are pairing-based signatures EUF-CMA secure under the CDH assumption. They rely on the Waters hash $H(\text{msg}) := h_0 \prod_{i=1}^n h_i^{m_i}$, where m_i denotes the i 'th bit of message $\text{msg} \in \{0, 1\}^n$ and $h_i \leftarrow \mathbb{G}$. A signature consists of $(\sigma_1, \sigma_2) := (g^r, g^{\alpha\beta} H(\text{msg})^r)$ for random $r \in \mathbb{Z}_q$, secret key $g^{\alpha\beta}$ and public key $(g^\alpha, g^\beta, h_0, \dots, h_n)$. A signature is verified by checking the equality $e(g^\alpha, g^\beta) \cdot e(\sigma_1, H(\text{msg})) = e(g, \sigma_2)$. Also recall that in LE, the public key is $(u, v, h) \in \mathbb{G}^3$ and the secret key $(x, y) \in \mathbb{Z}_q$ such that $u^x = v^y = h$ holds. Encrypting a message $\text{msg} \in \mathbb{G}$ amounts to choosing random κ_1, κ_2 and computing ciphertext $(c_1, c_2, c_3) := (u^{\kappa_1}, v^{\kappa_2}, h^{\kappa_1 + \kappa_2} \cdot \text{msg})$ and decryption computes $\text{msg} = c_3 \cdot (c_1^x \cdot c_2^y)^{-1}$. LE is IND-CPA secure under DLIN.

The essential idea is to embed a ciphertext (c_1, c_2, c_3) of an anamorphic message amsg as the first components of three Waters signatures as this component is uniformly chosen from $\mathbb{G}$ and a LE ciphertext is indistinguishable from a random $\mathbb{G}^3$ element under DLIN (cf. IND-CPA, Definition 9). However, we face the problem that the discrete logarithm r of this component is needed to compute the second component of the signature and for the c_i the DLs are unknown. To circumvent this problem, we make the assumption that every user generates its own parameters for the Waters hash

⁸We cannot rely on DDH as elements of the ElGamal ciphertext would be required in both source groups.

and thus knows a_i such that $h_i = g^{a_i}$ for $i \in \{0, \dots, n\}$ and that this is accepted by the dictator.⁹ Using the a_i allows us to compute the second signature component without knowing the DL r .

The resulting scheme is presented in Figure 6. We note that the same strategy can also be applied to the SUF-CMA secure variant by Boneh, Shen and Waters [BSW06]. The security of anamorphic signature extensions following this strategy is implied by our master theorem (Theorem 6) presented in Section 7.3.

<u>aGenS(gp) :</u> 1. Parse $\text{gp} = (\mathbb{G}, \mathbb{G}_T, e, g, g_T, p)$ 2. $\alpha, \beta, a_0, \dots, a_n \leftarrow \mathbb{Z}_p$ 3. $h_0 := g^{a_0}, \dots, h_n := g^{a_n}$ and $H : \{0, 1\}^n \rightarrow \mathbb{G}$ with $\text{msg} \mapsto h_0 \prod_{i=1}^n h_i^{m_i}$ 4. $\text{ask} := g^{\alpha\beta}, \text{avk} := (g^\alpha, g^\beta, h_0, \dots, h_n), \text{td} := (a_0, \dots, a_n)$ 5. return $(\text{ask}, \text{avk}, \text{td})$ <u>aGenR(gp) :</u> 1. Parse $\text{gp} = (\mathbb{G}, \mathbb{G}_T, e, g, g_T, p)$ 2. $(\text{pk}, \text{sk}) \leftarrow \text{LinE.Gen}(\text{gp})$ $// a, b, x \leftarrow \mathbb{Z}_p$, and $y := axb^{-1}$ $// u := g^a, v := g^b, h := g^{ax}$ $// \text{pk} := (u, v, h), \text{tk} := (x, y)$ 3. $\text{dk} := \text{pk}, \text{tk} := \text{sk}$ 4. return (dk, tk)	<u>aSig(ask, td, dk, msg, amsg) :</u> 1. Parse $\text{msg} = (\text{msg}_1, \text{msg}_2, \text{msg}_3)$ 2. $(c_1, c_2, c_3) \leftarrow \text{LinE.Enc}(\text{dk}, \text{amsg})$ $// \kappa_1, \kappa_2 \leftarrow \mathbb{Z}_p$ $// c_1 = u^{\kappa_1}, c_2 = v^{\kappa_2},$ $// c_3 = h^{\kappa_1 + \kappa_2} \cdot \text{amsg}$ 3. for $j \in \{1, 2, 3\}$: $\text{sig}_j := (c_j, \text{ask} c_j^{a_0} \prod_{i=1}^n c_j^{a_i m_{j,i}})$ 4. return $\text{asig} := (\text{sig}_1, \text{sig}_2, \text{sig}_3)$ <u>aDec(tk, avk, asig, msg) :</u> 1. Parse $\text{asig} = ((c_1, s_1), (c_2, s_2), (c_3, s_3))$ 2. $\text{amsg} \leftarrow \text{LinE.Dec}(\text{tk}, (c_1, c_2, c_3))$ $// \text{amsg} = c_3 \cdot (c_1^x \cdot c_2^y)^{-1}$ 3. return amsg
---	---

Figure 6: 3-PKASE Π_{aWaters}

7.2 BBS with ElGamal and Pseudo-Random Encodings

BBS signatures, as introduced in [BBS04], are a very well-known signature scheme and currently in the process of *standardization* by the IETF [LKW24]. A formal proof of its SUF-CMA security under the k -SDH assumption in the standard model has only recently been given in [TZ23]. We

⁹We are aware that this is a somewhat strong assumption as this could be suspicious for the dictator. Our second PKASE from BBS does not suffer from this limitation

consider BBS defined in asymmetric (type-3) pairing groups $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, g_1, g_2, q)$ of prime order q . The public key is g_2^x for secret key $x \in \mathbb{Z}_q$. For a signature on message $\mathbf{msg} \in \mathbb{Z}_q^\ell$, one samples a random $d \in \mathbb{Z}_q$ and computes $\sigma = (\sigma_1, \sigma_2) = (d, H(\mathbf{msg})^{\frac{1}{x+d}}) \in \mathbb{Z}_q \times \mathbb{G}_1$. Verification is straightforward by checking whether $e(\sigma_2, g_2) = e(H(\mathbf{msg}), g_2^{\sigma_1} \cdot g_2^x)$. Regarding our construction, we may omit the detail that the hash $H(\mathbf{msg}) := g_1 \cdot \prod_{i=1}^\ell h_i^{\mathbf{msg}_i}$ is as in Waters scheme (though with $\mathbf{msg}_i \in \mathbb{Z}_q$) as this is irrelevant. We set $\ell = 1$ in the following.

The basic idea (to obtain an IND-CPA secure extension) is to apply ElGamal encryption over $\mathbb{G}_1$ and to embed a ciphertext, which is pseudo-random over $\mathbb{G}_1^2$ under the DDH assumption, into a random component of two BBS signatures. The first component d of such a signature is randomly chosen, but unfortunately over $\mathbb{Z}_q$ instead of $\mathbb{G}_1$. To this end, we need to specify a pseudo-random $(\mathbb{G}_1^2, \mathbb{Z}_q^\mu)$ -encoding. Again, we can deploy Elligator Squared [Tib14] which also provides a suitable encoding for pairing-friendly curves $\mathbb{E}(\mathbb{F}_p)$ to $\mathbb{F}_p^2$. For instance, let us consider BN curves [BN06] for instantiating the asymmetric pairing-based setting of our BBS signatures. Then $\mathbb{G}_1 = \mathbb{E}(\mathbb{F}_p)$ is a cyclic group of prime order q . Note that for a random $h \in \mathbb{G}_1$, Elligator Squared will return a close to random element from $\mathbb{F}_p^2$ but not necessarily $\mathbb{F}_q^2$ as desired. However, since p is very close to q , more precisely $p - q \approx \sqrt{p}$ (cf. Hasse's bound), the probability, assuming the standard element representation for $\mathbb{F}_p$ and $\mathbb{F}_q$ as $\{0, \dots, p-1\}$ and $\{0, \dots, q-1\}$, that we hit an element which is not in $\mathbb{F}_q^2$ is negligible, namely at most $\frac{2}{\sqrt{p}}$.¹⁰ In this context, observe that we are actually not forced to do ElGamal encryption in the pairing group that BBS uses. We could alternatively use ElGamal (or any other secure encryption) over any other group as long as we are able to find a suitable pseudo-random encoding of a ciphertext as $\mathbb{Z}_q^\mu$ element.

The 4-message anamorphic extension Π_{aBBS} resulting from a pseudo-random $(\mathbb{G}_1^2, \mathbb{Z}_q^4)$ -encoding (e.g., using Elligator Squared for $\mathbb{G}_1$ of BN curves) is shown in Figure 7. Note that no trapdoor is required here to compute the second signature component. Π_{aBBS} achieves anamorphic security and IND-CPA security. To achieve sIND-RCCA security, it suffices to replace plain ElGamal by an FO-based PKE from ElGamal and XOR-based symmetric encryption and extend the pseudo-random encoding described above to also cover the symmetric ciphertext component. For q close to 2^λ , a random bitstring $G(r) \oplus \mathbf{amsg} \in \{0, 1\}^\lambda$ can simply be interpreted as $\mathbb{F}_q$ -element s_5 which causes an encoding error (integer $> p-1$) with negligible probability. This leads to a 5-message anamorphic signature extension satisfying sIND-RCCA security. To additionally achieve IND-CCA security, one needs to make use of the SUF-CMA security of BBS, by checking the validity of the signatures **asig** wrt. to the messages **msg** in **aDec**.

Also for other variants of BBS signatures like BBS+ [ASM06], such anamorphic extensions can be obtained in the same way. The security of all these extensions is covered by our master theorem in Section 7.3.

7.3 Anamorphism-Friendly Signatures with Compliant Encryption

Generalizing the ideas from Section 7.1 and Section 7.2, the following definition covers signature schemes which are canonical candidates for an anamorphic extension. The signatures generated by these schemes have at least one uniformly (and freshly) chosen component and allow the rest of the signature to be computed (cf. **RemSig** below) by treating the uniform components in a black-box way, that means, without using additional information related to the generation of these components (e.g., the DL d used to create a random group element g^d). To enable the latter, some trapdoor

¹⁰To be compliant to the definition of our pseudo-random encoding, we could output $\perp$ in this case.

<u>aGenS(gp) :</u> 1. Parse $\text{gp} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, q, H)$ 2. $x \leftarrow \mathbb{Z}_q$ 3. $\text{ask} := x, \text{avk} := g_2^x$ 4. return (ask, avk, td) <u>aGenR(gp) :</u> 1. Parse $\text{gp} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, q, H)$ 2. $(\text{pk}, \text{sk}) \leftarrow \text{El.Gen}(\text{gp})$ // $x \leftarrow \mathbb{Z}_q$ // $\text{pk} = g_1^x, \text{tk} = x$ 3. $\text{dk} := \text{pk}, \text{tk} := \text{sk}$ 4. return (dk, tk)	<u>aSig(ask, td, dk, msg, amsg) :</u> 1. Parse $\text{msg} = (\text{msg}_1, \text{msg}_2, \text{msg}_3, \text{msg}_4)$ 2. $c := \text{El.Enc}(\text{dk}, \text{amsg})$ // $\kappa \leftarrow \mathbb{Z}_q$ // $c = (c_1, c_2) = (g_1^\kappa, \text{dk}^\kappa \text{amsg})$ 3. $(s_1, s_2, s_3, s_4) := \text{Encode}(c)$ // In the negligible event that // $\text{Encode}(c) = \perp$, we return to 2. 4. for $j \in \{1, 2, 3, 4\}$: $\text{sig}_j := (s_j, (H(\text{msg}_j))^{\frac{1}{\text{ask} + s_j}})$ 5. return $\text{asig} := (\text{sig}_1, \text{sig}_2, \text{sig}_3, \text{sig}_4)$ <u>aDec(tk, avk, asig, msg) :</u> 1. Parse $\text{asig} = ((s_1, s'_1), (s_2, s'_2), (s_3, s'_3), (s_4, s'_4))$ 2. $c := \text{Decode}((s_1, s_2), (s_3, s_4))$ 3. if $c = \perp$ return $\perp$ 4. $\text{amsg} := \text{EL.Dec}(\text{tk}, c)$ 5. return amsg
---	--

Figure 7: 4-PKASE Π_{aBBS}

information from the key generation might be used (cf. $\text{SS.Gen}'$ below). An encryption scheme is compliant to such a signature scheme, in the sense that it easily allows for the construction of an anamorphic extension, if its ciphertext components are pseudo-random (IND\$-CPA, see Definition 9) and can be completely embedded in one or multiple signatures.

Definition 21 (Anamorphism-Friendly Sign. & Compliant Enc.) *Let Π_{SS} be a signature scheme with a signature space that can be written as $S_0^{e_0} \times \dots \times S_{m-1}^{e_{m-1}} \times R$, where $S_i \neq S_j$ and $e_i > 0$, such that the following conditions hold:*

1. *Random components: $m - 1 \geq 0$ and for signatures $(s_0, \dots, s_{m-1}, r)$ generated by SS.Sig , it holds that all $s_i \leftarrow S_i^{e_i}$ are uniformly and freshly chosen.*
2. *Black-box computation of remainder: There exist two PPT algorithms $\text{SS.Gen}'$, SS.RemSig such that $(\text{sk}, \text{vk}) \approx_c (\text{sk}', \text{vk}')$, where $(\text{sk}, \text{vk}) \leftarrow \text{SS.Gen}(\text{gp})$ and $(\text{sk}', \text{vk}', \text{td}) \leftarrow \text{SS.Gen}'(\text{gp})$,*

<u>aGenS(gp) :</u> 1. $(\text{ask}, \text{avk}, \text{td}) \leftarrow \text{SS.Gen}'(\text{gp})$ 2. return $(\text{ask}, \text{avk}, \text{td})$ <u>aGenR(gp) :</u> 1. $(\text{dk}, \text{tk}) \leftarrow \text{ES.Gen}(\text{gp})$ 2. return (dk, tk) <u>aDec(tk, avk, asig, msg) :</u> 0. for $0 \leq i \leq \mu - 1$: if $\text{SS.Ver}(\text{avk}, \text{sig}_i, \text{msg}_i) = \perp$ return $\perp$ 1. for $0 \leq i \leq m - 1, 0 \leq j \leq e'_i - 1$: $c_{i,j} := s_{i,j \bmod e_i}^{(\lfloor \frac{j}{e_i} \rfloor)}$ 2. return $\text{amsg} \leftarrow \text{ES.Dec}(\text{tk}, (c_i)_i)$	<u>aSig(ask, td, dk, msg, amsg) :</u> 1. Parse $\text{msg} = (\text{msg}_0, \dots, \text{msg}_{\mu-1})$ 2. $\text{act} = (c_0, \dots, c_{m-1}) \leftarrow \text{ES.Enc}(\text{dk}, \text{amsg})$ where $c_i = (c_{i,0}, \dots, c_{i,e_i-1}) \in S_i^{e'_i}$ 3. for $0 \leq i \leq \mu - 1, 0 \leq \ell \leq m - 1, 0 \leq j \leq e_\ell - 1$: $s_{\ell,j}^{(i)} := \begin{cases} c_{\ell, ie_\ell + j}, & ie_\ell + j < e'_\ell \\ a \leftarrow S_\ell, & \text{else} \end{cases}$ 4. for $0 \leq i \leq \mu - 1$: $r^{(i)} \leftarrow \text{SS.RemSig}(\text{ask}, \text{td}, \text{msg}_i, (s_j^{(i)})_j)$ $\text{sig}_i := ((s_j^{(i)})_j, r^{(i)})$ 5. return $\text{asig} := (\text{sig}_0, \dots, \text{sig}_{\mu-1})$
---	--

Figure 8: Generic μ -PKASE Π_{aGAE} and variant $\Pi_{\text{aGAE}'}$

and

$$\Pr \left[b = b' : \begin{array}{l} \text{gp} \leftarrow \text{SS.Setup}(1^\lambda) \\ (\text{sk}, \text{vk}, \text{td}) \leftarrow \text{SS.Gen}'(\text{gp}) \\ \text{msg} \leftarrow \mathcal{A}(\text{sk}, \text{vk}) \\ (s_0, \dots, s_{m-1}, r_0) \leftarrow \text{SS.Sig}(\text{sk}, \text{msg}) \\ r_1 \leftarrow \text{SS.RemSig}(\text{sk}, \text{td}, \text{msg}, (s_0, \dots, s_{m-1})) \\ b \leftarrow \{0, 1\} \\ b' \leftarrow \mathcal{A}(s_0, \dots, s_{m-1}, r_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda)$$

In this case, we call Π_{SS} anamorphism-friendly. We call an encryption scheme Π_{ES} compliant to Π_{SS} if it is IND\$-CPA secure and its message space can be written as $S_0^{e'_0} \times \dots \times S_{m-1}^{e'_{m-1}}$ with $e'_i > 0$.

In this situation, we can securely embed a ciphertext containing the anamorphic message into the components of one or multiple signatures. This is specified by the **aSig** algorithm in Figure 8. In Step 3, as many ciphertext components as possible are packed into the same signature. As it may happen that the designated signature components cannot be completely “filled” with ciphertext components, we use truly random elements in this case. The remainder of the signature is computed

in Step 4 using **RemSig**. When receiving μ signatures, the ciphertext components are first unpacked in Step 2 of **aDec** before **amsg** is recovered. By following this blueprint using an anamorphism-friendly signature and compliant encryption, we obtain a secure anamorphic extension Π_{aGAE} also satisfying IND-CPA security. If the underlying encryption additionally satisfies IND-RCCA security then Π_{aGAE} achieves sIND-RCCA security. Note that an IND-CCA secure PKE still leads to an sIND-RCCA but not IND-CCA secure extension when only following Steps 1 and 2 of **aDec** in Figure 8 as the adversary (against the extension) may tamper with the $r^{(i)}$ of the challenge which would lead to a different **asig** decrypting to **amsg**. To additionally achieve IND-CCA security, **aDec** needs to verify the signatures **asig** (Step 0 in Figure 8) and the signature scheme needs to be SUF-CMA secure. Still sIND-CCA security cannot be achieved in this way as, given **ask**, the adversary may re-use the challenge $c_{i,j}$ to generate a new signature.

Theorem 6 *Let Π_{SS} be an anamorphism-friendly signature scheme and Π_{ES} an encryption scheme compliant to Π_{SS} . Then Π_{aGAE} as defined in Figure 8 is a μ -message anamorphic signature extension, where $\mu = \max\{\lceil \frac{e'_i}{e_i} \rceil\}$, satisfying anamorphic and IND-CPA security. If Π_{ES} additionally achieves IND-RCCA security then Π_{aGAE} achieves sIND-RCCA security. Moreover, if Π_{ES} satisfies IND-CCA security and Π_{SS} satisfies SUF-CMA security, then the variant $\Pi_{\text{aGAE}'}$ in Figure 8 achieves IND-CCA security.*

Proof. We prove this theorem in two parts.

Anamorphic Security. In the following sequence of hybrid games let $\mathcal{D}$ be a PPT dictator and Q its total number of oracle queries.

Game 0: This is the real game $\text{GReal}_{\text{SS}}^{\mathcal{D}}(\lambda)$ in the anamorphic security definition, where the challenger generates $(\text{sk}, \text{vk}) \leftarrow \text{SS.Gen}(\text{gp})$ and handles signature queries $\text{OSig}(\text{msg}, \text{amsg})$ of the dictator by computing $(\text{SS.Sig}(\text{sk}, \text{msg}_i))$ for $0 \leq i \leq \mu - 1$.

Game 1: In this game the challenger generates the key pair by computing $(\text{ask}, \text{avk}, \text{td}) \leftarrow \text{SS.Gen}'(\text{gp})$ and provides (ask, avk) as input to $\mathcal{D}$.

Game (2, j) for $j \in [0, \mu Q]$: In this series of games we slightly modify Game 1. We define Game (2, 0) to be identical to Game 1. In Game (2, j), the challenger handles the first j signature computations (in order to respond to oracle calls) as follows: It computes $(s_0, \dots, s_{m-1}, r) \leftarrow \text{SS.Sig}(\text{ask}, \text{msg}_j)$ but replaces r with $r' \leftarrow \text{SS.RemSig}(\text{ask}, \text{td}, \text{msg}_j, (s_0, \dots, s_{m-1}))$, i.e., it returns $(s_0, \dots, s_{m-1}, r')$. The remaining signature computations are left untouched in comparison to Game (2, 0). Note that in Game (2, μQ) we have modified all signature computations in this way.

Game 3: In this game, we modify all signature computations done by the challenger (in comparison to Game (2, μQ)) at once: instead of calling SS.Sig to compute $(s_0, \dots, s_{m-1})$ being input to SS.RemSig , we choose them uniformly at random, i.e., $(s_0, \dots, s_{m-1}) \leftarrow S_0^{e_0} \times S_{m-1}^{e_{m-1}}$.

Game (4, j) for $j \in [0, Q]$: We define Game (4, 0) to be identical to Game 3, except that the challenger additionally generates the key pair $(\text{dk}, \text{tk}) \leftarrow \text{ES.Gen}(\text{gp})$ of the encryption scheme. In Game (4, j), the challenger handles the first j oracle calls $\text{OSig}(\text{msg}, \text{amsg})$ as follows: it computes $(c_0, \dots, c_{m-1}) \leftarrow \text{ES.Enc}(\text{dk}, \text{amsg})$ and embeds the ciphertext components into $(s_0, \dots, s_{m-1})$ as defined by Steps 2 and 3 of the **aSig** definition in Figure 8. Then SS.RemSig is called as in Game 3. Note that in Game (4, Q) we have embedded ciphertexts in all responses to oracle calls.

Game 5: This is the anamorphic game $\text{GAnam}_{\text{GAE}}^{\mathcal{D}}(\lambda)$, where the challenger generates $(\text{ask}, \text{avk}, \text{td}) \leftarrow \text{GAE.aGenS}(\text{gp})$, $(\text{dk}, \text{tk}) \leftarrow \text{GAE.aGenR}(\text{gp})$ and handles signature queries $\text{OaSig}(\mathbf{msg}, \text{amsg})$ of the dictator by computing $\text{aSig}(\text{ask}, \text{td}, \text{dk}, \mathbf{msg}, \text{amsg})$.

The indistinguishability of Game 0 and Game 1 follows immediately from the second property of an anamorphism-friendly signature. Game 1 and Game (2, 0) are identical.

Game (2, 0) is indistinguishable from Game (2, Q), as a distinguisher $\mathcal{D}$ for Game (2, j) and Game (2, $j + 1$) (for random $j \in \{0, \dots, \mu Q - 1\}$) can be turned in to an adversary against the black-box computation of remainder property of an anamorphism-friendly signature. Game (2, μQ) is perfectly indistinguishable from Game 3 due to the random component property of the signature scheme. Game 3 and Game (4, 0) are perfectly indistinguishable as the modification is purely conceptional. Game (4, 0) is indistinguishable from Game (4, Q), as from a distinguisher $\mathcal{D}$ for Game (4, j) and Game (4, $j + 1$) (for random $j \in \{0, \dots, Q - 1\}$) an adversary against the IND\$-CPA security of Π_{ES} can be constructed. Finally, Game (4, Q) and Game 5 are identical.

IND-CPA Security of Π_{aGAE} . This follows immediately from the IND\$-CPA security of Π_{ES} : Let Game 0 be the IND-CPA game for $\beta = 1$. In Game 1, when computing the challenge signature $\text{asig}_1 \leftarrow \text{aSig}(\text{ask}, \text{td}, \text{dk}, \mathbf{msg}, \text{amsg}_1)$, we replace the ciphertexts $(c_0, \dots, c_{m-1})$ to be embedded by aSig by randomly chosen values from the ciphertext space $S_0^{e_0} \times \dots \times S_{m-1}^{e_{m-1}}$. Game 0 and Game 1 are indistinguishable if Π_{ES} is IND\$-CPA. Note that the reduction algorithm can handle signature queries as it can choose ask, td itself. In Game 2, we immediately revert this change by embedding ciphertexts encrypting amsg_0 , i.e., the challenger computes $\text{asig}_0 \leftarrow \text{aSig}(\text{ask}, \text{td}, \text{dk}, \mathbf{msg}, \text{amsg}_0)$. Again, Game 1 and Game 2 are indistinguishable if Π_{ES} is IND\$-CPA. Game 2 is the IND-CPA game for $\beta = 0$.

sIND-RCCA Security of Π_{aGAE} . This follows immediately from the assumed IND-RCCA security of Π_{ES} . The reduction algorithm $\mathcal{B}$ is given gp, dk as input from its IND-RCCA challenger and has access to a $\text{Dec}(\text{tk}, \cdot)$ oracle. It can generate $(\text{avk}, \text{ask}, \text{td}) \leftarrow \text{aGenR}(\text{gp})$ itself, and thus provide $(\text{avk}, \text{dk}, \text{ask})$ as input to the sIND-RCCA adversary $\mathcal{A}$. Clearly, $\mathcal{B}$ can handle $\mathcal{A}$'s $\text{aSig}(\text{ask}, \text{td}, \text{dk}, \cdot, \cdot)$ oracle queries since it knows all necessary inputs. $\mathcal{B}$ can also handle $\text{aDec}(\text{tk}, \text{avk}, \cdot, \cdot)$ queries by extracting the $c_{i,j}$ from the given $\mathbf{asig}$ as specified in Figure 8 and querying its own $\text{Dec}(\text{tk}, \cdot)$ oracle this input. Clearly, this simulated aDec oracle behaves exactly as the real aDec oracle (in particular it rejects an input exactly when the real one does). To generate the challenge signature for the given $(\mathbf{msg}, \text{amsg}_0, \text{amsg}_1)$, $\mathcal{B}$ provides $(\text{amsg}_0, \text{amsg}_1)$ to its own challenger and embeds the response into signatures for $\mathbf{msg}$ as specified by Steps 3 to 5 of aSig in Figure 8. Finally $\mathcal{B}$ outputs whatever $\mathcal{A}$ outputs.

IND-CCA Security of Π_{aGAE} . This proof is similar to the sIND-RCCA security proof for Π_{aGAE} we have seen above, where the SUF-CMA security of Π_{SS} now ensures that the simulation of the aDec using a IND-CCA Dec oracle works out.

So let us consider how the reduction algorithm $\mathcal{B}$ (the adversary against the IND-CCA security of Π_{ES}) handles $\mathcal{A}$'s queries of the $\text{aDec}(\text{tk}, \text{avk}, \cdot, \cdot)$ IND-CCA oracle: For given $(\mathbf{asig}, \mathbf{msg})$, it first checks whether $(\mathbf{asig}, \mathbf{msg})$ equals the challenge signature and message vectors $(\mathbf{asig}^*, \mathbf{msg}^*)$ and rejects if it does. Then it verifies each of the μ signatures contained $\mathbf{asig}$ and returns $\perp$ if at least one of these checks fail. Otherwise, it extracts the $c_{i,j}$ from the given $\mathbf{asig}$ as specified in Figure 8

and queries its own $\text{Dec}(\text{tk}, \cdot)$ oracle for this input. Obviously, the only thing this simulation does differently compared to a real aDec oracle is calling the $\text{Dec}(\text{tk}, \cdot)$ oracle instead of executing the $\text{Dec}(\text{tk}, \cdot)$ algorithm. This can lead to the following simulation failure: The $\text{Dec}(\text{tk}, \cdot)$ oracle on input of the challenge ciphertext $c^* = (c_{i,j}^*)_{i,j}$ (generated by $\mathcal{B}$'s IND-CCA challenger) will reject while the algorithm will return the corresponding decryption result. Hence, we need to show that the checks (particularly the signature verification) performed before this point ensure that the $\text{Dec}(\text{tk}, \cdot)$ oracle will never be queried with c^* as input.

Let us consider the different inputs $(\text{asig}, \text{msg})$ to the aDec oracle which could lead to c^* :

1. $(\text{asig}, \text{msg}) = (\text{asig}^*, \text{msg}^*)$: In this case $\mathcal{B}$ immediately rejects.
2. $\text{asig} = \text{asig}^*$ and $\text{msg} \neq \text{msg}^*$: In this case, we have at least one verifying signature sig_i^* for a non-challenge message $\text{msg}_i \neq \text{msg}_i^*$. There is only a negligible chance that a call to the aSig oracle (including msg_i as part of the input) could have produced the same signature since the used randomness for producing this signature is freshly chosen. In particular, if we write sig_i^* as $\text{sig}_i^* = ((s_j^{(i)}, r^{(i)}))$ as in Figure 8, for the newly produced signature the same $s_j^{(i)}$ would need to be chosen. Hence, in this case the SUF-CMA security is broken and thus we could build an adversary against the underlying signature scheme.
3. $\text{asig} \neq \text{asig}^*$ and $\text{msg} = \text{msg}^*$: In this case, we have at least one signature $\text{sig}_i \neq \text{sig}_i^*$ which verifies for the challenge message msg_i^* . Since also with this signature sig_i , the extraction of the ciphertext need to yield c^* which is composed of the $s_j^{(i)}$, it holds that at least one of the random components of sig_i needs to coincide with the one of sig_i^* . Similar to the previous case, we can argue that such sig_i cannot stem from a call to the aSig oracle and thus sig_i is a forgery of a new signature for the challenge message msg_i^* . Thus, in this case the SUF-CMA security is broken.
4. $\text{asig} \neq \text{asig}^*$ and $\text{msg} \neq \text{msg}^*$: In this case, we are either in one of the two previous situations (for which we have already shown that they violate SUF-CMA security) or for all i it holds that $\text{sig}_i \neq \text{sig}_i^*$ and $\text{msg}_i \neq \text{msg}_i^*$. So let us consider the latter. Using the same argument as in the previous case, it follows that there is a negligible probability that sig_i stems from a call to the aSig oracle and thus we either have a signature forgery for a new message or a message for which the aSig oracle has been queried before. Both cases violate the SUF-CMA security of the signature scheme.

To summarize, assuming SUF-CMA security, the IND-CCA adversary $\mathcal{B}$ against Π_{ES} can perfectly simulate the aDec oracle for $\mathcal{A}$ and thus $\mathcal{A}$'s IND-CCA challenger for Π_{aGAE} . Finally, $\mathcal{B}$ outputs whatever $\mathcal{A}$ outputs.

To make the above argument more formal, one can define a sequence of games, where the first game is the IND-CCA game for Π_{aGAE} for $\beta = 1$ and the last game is the IND-CCA game for Π_{aGAE} for $\beta = 0$. In the first three intermediate games, we modify the aDec oracle by considering the cases 2-4 from above and let the oracle reject if the corresponding event happens. Indistinguishability between the games can then be shown using the SUF-CMA security of Π_{SS} (based on the arguments made above). After that, we can be sure that an aDec oracle can be simulated by a Dec oracle and thus we can change the challenge message to be encrypted from amsg_1 to amsg_0 in the next and final game. Here indistinguishability follows from the IND-CCA security for Π_{ES} . $\square$

The anamorphic extension of Waters signatures using linear encryption presented in Section 7.1 is an instance of our general blueprint and thus its security follows from Theorem 6. The same holds for BBS and the encryption scheme obtained by combining ElGamal with a suitable pseudo-random encoding from Section 7.2.

Theorem 7 *Waters signatures (as defined in Section 7.1) are anamorphism-friendly and linear encryption is compliant to them under the DLIN assumption.*

Proof. We prove the theorem in two parts.

Anamorphism-Friendliness. The signature space of Waters is $\mathbb{G}^2$ where $(\mathbb{G}, \mathbb{G}_T, e, p)$ is a bilinear group setting. In the notation of Definition 21, we can write it as $S_0 \times R$, with $S_0 = \mathbb{G}$ and $R = \mathbb{G}$. Considering only the first component of a signature $(g^d, g^{\alpha\beta} H(\text{msg})^d)$, we observe that it is uniformly and freshly as $d \leftarrow \mathbb{Z}_p$. Hence, the first property is satisfied. Regarding the second property, we observe that $g^{\alpha\beta} H(\text{msg})^d = g^{\alpha\beta} (h_0 \prod_{i=1}^{\ell} h_i^{m_i})^d$, where $h_i = g^{a_i}$ are elements chosen during signature key generation, can alternatively be computed by setting $\text{td} := (a_0, \dots, a_n)$ and $\text{RemSig}(\text{ask}, \text{td}, \text{msg}, s = g^d) := \text{ask} s^{a_0} \prod_{i=1}^{\ell} s^{a_i m_i}$. Gen' outputs a key pair with the same distribution as Gen and additionally td . This implies the second property.

Compliance of Linear Encryption. It is easy to see that the linear encryption scheme as defined in Section 7.1 is IND\$-CPA over the group $\mathbb{G}$ under the DLIN assumption. The DLIN problem asks an adversary to distinguish h^{x+y} from h^z , where $z \leftarrow \mathbb{Z}_p$, given (u, v, h, u^x, v^y) , where $u, v, h \leftarrow \mathbb{G}$ and $x, y \leftarrow \mathbb{Z}_p$. First we observe that the public key (u, v, h) of the linear encryption scheme can be generated by choosing $a, b, x \leftarrow \mathbb{Z}_p$ and setting $y := axb^{-1}$, $u := g^a$, $v := g^b$, $h := g^{ax}$. In this way, (u, v, h) are indeed random generators as in the DLIN problem and they satisfy $u^x = v^y = h$ as needed for decryption. Now, the reduction to the DLIN assumption is very simple: given an instance of the DLIN problem (u, v, h, u^x, v^y, w) , we compute the challenge ciphertext for the IND\$-CPA adversary as $(c_1, c_2, c_3) = (u^x, v^y, w \cdot \text{msg})$ and return the output of the adversary as our guess. Clearly, if w is random then also the ciphertext, if $w = h^{x+y}$ then it is a proper encryption of msg . Finally, observe that the ciphertext space of the linear encryption is $\mathbb{G}^3$ and is thus compliant with Waters signature scheme. $\square$

Theorem 8 *BBS signatures (as defined in Section 7.2) are anamorphism-friendly and ElGamal encryption, where ciphertexts are encoded during encryption and decoded during decryption using a pseudo-random $\mathbb{Z}_p^k$ -encoding, is compliant to BBS under the DDH assumption.*

Proof. We prove the theorem in two parts.

Anamorphism-Friendliness. The signature space of BBS can be written in the form of Definition 21 as $S_0 \times R$, where $S_0 = \mathbb{Z}_p$ and $R = \mathbb{G}$. As the first component of each signature $(s_0, H(\text{msg})^{\frac{1}{x+s_0}})$ is uniformly and freshly chosen, the first property is satisfied. The second one is also trivially satisfied by setting $\text{td} := \perp$ and $\text{RemSig}(\text{ask} = x, \text{td}, \text{msg}, s_0) := H(\text{msg})^{\frac{1}{x+s_0}}$.

Compliance of ElGamal+Pseudo-Random Encoding. Let us consider ElGamal encryption over $\mathbb{G}_1$ of an asymmetric bilinear groups setting $\text{gp} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, g_1, g_2, e, p)$ and Encode and Decode algorithms mapping elements from $\mathbb{G}_1$ to $\mathbb{Z}_p^k$ and vice versa. Then the encryption scheme defined by $\text{Gen}(\text{gp}) := \text{El.Gen}(\text{gp})$, $\text{Enc}(\text{pk}, \text{msg}) := (\text{Encode}(c_1), \text{Encode}(c_2))$, where $(c_1, c_2) \leftarrow \text{El.Enc}(\text{pk}, \text{msg})$, $\text{Dec}(\text{sk}, (c'_1, c'_2)) := \text{El.Dec}(\text{sk}, (\text{Decode}(c'_1), \text{Decode}(c'_2)))$ has ciphertext

space $\mathbb{Z}_p^{2k}$, provides correctness up to a negligible error probability, and satisfies IND\$-CPA security. Regarding IND\$-CPA security, we first observe that ElGamal is IND\$-CPA secure under the DDH assumption (the proof is similar to the one for linear encryption under DLIN), and that the output of Encode for some uniform $c_i \in \mathbb{G}_1$ is indistinguishable from an uniform element of $\mathbb{Z}_p^k$. Finally, observe that the ciphertext space of the combined encryption scheme is $\mathbb{Z}_p^{2k}$ and is thus compliant to the signature space component $S_0 = \mathbb{Z}_p$ of the BBS signature scheme. $\square$

References

- [ABR99] Michel Abdalla, Mihir Bellare, and Phillip Rogaway. Dhaes: An encryption scheme based on the diffie-hellman problem. *IACR Cryptol. ePrint Arch.*, 1999:7, 1999.
- [ADKL19] Prabhanjan Ananth, Apoorvaa Deshpande, Yael Tauman Kalai, and Anna Lysyanskaya. Fully homomorphic NIZK and NIWI proofs. In Dennis Hofheinz and Alon Rosen, editors, *TCC 2019: 17th Theory of Cryptography Conference, Part II*, volume 11892 of *Lecture Notes in Computer Science*, pages 356–385. Springer, Cham, December 2019. doi:[10.1007/978-3-030-36033-7_14](https://doi.org/10.1007/978-3-030-36033-7_14).
- [ASM06] Man Ho Au, Willy Susilo, and Yi Mu. Constant-size dynamic k-TAA. In Roberto De Prisco and Moti Yung, editors, *SCN 06: 5th International Conference on Security in Communication Networks*, volume 4116 of *Lecture Notes in Computer Science*, pages 111–125. Springer, Berlin, Heidelberg, September 2006. doi:[10.1007/11832072_8](https://doi.org/10.1007/11832072_8).
- [BBS04] Dan Boneh, Xavier Boyen, and Hovav Shacham. Short group signatures. In Matthew Franklin, editor, *Advances in Cryptology – CRYPTO 2004*, volume 3152 of *Lecture Notes in Computer Science*, pages 41–55. Springer, Berlin, Heidelberg, August 2004. doi:[10.1007/978-3-540-28628-8_3](https://doi.org/10.1007/978-3-540-28628-8_3).
- [BF01] Dan Boneh and Matthew K. Franklin. Identity-based encryption from the Weil pairing. In Joe Kilian, editor, *Advances in Cryptology – CRYPTO 2001*, volume 2139 of *Lecture Notes in Computer Science*, pages 213–229. Springer, Berlin, Heidelberg, August 2001. doi:[10.1007/3-540-44647-8_13](https://doi.org/10.1007/3-540-44647-8_13).
- [BGH⁺24] Fabio Banfi, Konstantin Gegier, Martin Hirt, Ueli Maurer, and Guilherme Rito. Anamorphic encryption, revisited. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024, Part II*, volume 14652 of *Lecture Notes in Computer Science*, pages 3–32. Springer, Cham, May 2024. doi:[10.1007/978-3-031-58723-8_1](https://doi.org/10.1007/978-3-031-58723-8_1).
- [BN06] Paulo S. L. M. Barreto and Michael Naehrig. Pairing-friendly elliptic curves of prime order. In Bart Preneel and Stafford Tavares, editors, *SAC 2005: 12th Annual International Workshop on Selected Areas in Cryptography*, volume 3897 of *Lecture Notes in Computer Science*, pages 319–331. Springer, Berlin, Heidelberg, August 2006. doi:[10.1007/11693383_22](https://doi.org/10.1007/11693383_22).
- [Bon98] Dan Boneh. The decision diffie-hellman problem. In *International algorithmic number theory symposium*, pages 48–63. Springer, 1998.

- [BR95] Mihir Bellare and Phillip Rogaway. Optimal asymmetric encryption. In Alfredo De Santis, editor, *Advances in Cryptology – EUROCRYPT’94*, volume 950 of *Lecture Notes in Computer Science*, pages 92–111. Springer, Berlin, Heidelberg, May 1995. doi:[10.1007/BFb0053428](https://doi.org/10.1007/BFb0053428).
- [BSW06] Dan Boneh, Emily Shen, and Brent Waters. Strongly unforgeable signatures based on computational Diffie-Hellman. In Moti Yung, Yevgeniy Dodis, Aggelos Kiayias, and Tal Malkin, editors, *PKC 2006: 9th International Conference on Theory and Practice of Public Key Cryptography*, volume 3958 of *Lecture Notes in Computer Science*, pages 229–240. Springer, Berlin, Heidelberg, April 2006. doi:[10.1007/11745853_15](https://doi.org/10.1007/11745853_15).
- [CC05] Liqun Chen and Zhaohui Cheng. Security proof of sakai-kasahara’s identity-based encryption scheme. In Nigel P. Smart, editor, *Cryptography and Coding, 10th IMA International Conference, Cirencester, UK, December 19-21, 2005, Proceedings*, volume 3796 of *Lecture Notes in Computer Science*, pages 442–459. Springer, 2005. doi:[10.1007/11586821_29](https://doi.org/10.1007/11586821_29).
- [CCLZ25] Wonseok Choi, Daniel Collins, Xiangyu Liu, and Vassilis Zikas. A unified treatment of anamorphic encryption. Cryptology ePrint Archive, Paper 2025/309, 2025. URL <https://eprint.iacr.org/2025/309>.
- [CGM24a] Dario Catalano, Emanuele Giunta, and Francesco Migliaro. Anamorphic encryption: New constructions and homomorphic realizations. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024, Part II*, volume 14652 of *Lecture Notes in Computer Science*, pages 33–62. Springer, Cham, May 2024. doi:[10.1007/978-3-031-58723-8_2](https://doi.org/10.1007/978-3-031-58723-8_2).
- [CGM24b] Dario Catalano, Emanuele Giunta, and Francesco Migliaro. Limits of black-box anamorphic encryption. Cryptology ePrint Archive, Paper 2024/1098, 2024. URL <https://eprint.iacr.org/2024/1098>.
- [CGM25] Dario Catalano, Emanuele Giunta, and Francesco Migliaro. Generic anamorphic encryption, revisited: New limitations and constructions, 2025. doi:[10.1007/978-3-031-91124-8_10](https://doi.org/10.1007/978-3-031-91124-8_10).
- [CKN03] Ran Canetti, Hugo Krawczyk, and Jesper Buus Nielsen. Relaxing chosen-ciphertext security. In Dan Boneh, editor, *Advances in Cryptology – CRYPTO 2003*, volume 2729 of *Lecture Notes in Computer Science*, pages 565–582. Springer, Berlin, Heidelberg, August 2003. doi:[10.1007/978-3-540-45146-4_33](https://doi.org/10.1007/978-3-540-45146-4_33).
- [CS98] Ronald Cramer and Victor Shoup. A practical public key cryptosystem provably secure against adaptive chosen ciphertext attack. In Hugo Krawczyk, editor, *Advances in Cryptology – CRYPTO’98*, volume 1462 of *Lecture Notes in Computer Science*, pages 13–25. Springer, Berlin, Heidelberg, August 1998. doi:[10.1007/BFb0055717](https://doi.org/10.1007/BFb0055717).
- [DH21] Whitfield Diffie and Martin Hellman. New directions in cryptography (1976). 2021.
- [DL25] Amit Deo and Benoît Libert. Fully asymmetric anamorphic homomorphic encryption from LWE. 2025. URL <https://eprint.iacr.org/2025/328>.

- [ElG86] Taher ElGamal. On computing logarithms over finite fields. In Hugh C. Williams, editor, *Advances in Cryptology – CRYPTO’85*, volume 218 of *Lecture Notes in Computer Science*, pages 396–402. Springer, Berlin, Heidelberg, August 1986. doi:[10.1007/3-540-39799-X_28](https://doi.org/10.1007/3-540-39799-X_28).
- [ETS22] ETSI. Etsi tr 103 719: Guide to identity-based cryptography. https://www.etsi.org/deliver/etsi_tr/103700_103799/103719/01.01.01_60/tr_103719v010101p.pdf, 2022.
- [FO99] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael J. Wiener, editor, *Advances in Cryptology – CRYPTO’99*, volume 1666 of *Lecture Notes in Computer Science*, pages 537–554. Springer, Berlin, Heidelberg, August 1999. doi:[10.1007/3-540-48405-1_34](https://doi.org/10.1007/3-540-48405-1_34).
- [FO13] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. *Journal of Cryptology*, 26(1):80–101, January 2013. doi:[10.1007/s00145-011-9114-1](https://doi.org/10.1007/s00145-011-9114-1).
- [Gal05] David Galindo. Boneh-Franklin identity based encryption revisited. In Luís Caires, Giuseppe F. Italiano, Luís Monteiro, Catuscia Palamidessi, and Moti Yung, editors, *ICALP 2005: 32nd International Colloquium on Automata, Languages and Programming*, volume 3580 of *Lecture Notes in Computer Science*, pages 791–802. Springer, Berlin, Heidelberg, July 2005. doi:[10.1007/11523468_64](https://doi.org/10.1007/11523468_64).
- [GPV08] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In *STOC 2008*, pages 197–206. ACM, 2008.
- [GSW13] Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013, Part I*, volume 8042 of *Lecture Notes in Computer Science*, pages 75–92. Springer, Berlin, Heidelberg, August 2013. doi:[10.1007/978-3-642-40041-4_5](https://doi.org/10.1007/978-3-642-40041-4_5).
- [ILL89] Russell Impagliazzo, Leonid A. Levin, and Michael Luby. Pseudo-random generation from one-way functions (extended abstracts). In *21st Annual ACM Symposium on Theory of Computing*, pages 12–24. ACM Press, May 1989. doi:[10.1145/73007.73009](https://doi.org/10.1145/73007.73009).
- [KLR19] Michael Klooß, Anja Lehmann, and Andy Rupp. (R)CCA secure updatable encryption with integrity protection. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2019, Part I*, volume 11476 of *Lecture Notes in Computer Science*, pages 68–99. Springer, Cham, May 2019. doi:[10.1007/978-3-030-17653-2_3](https://doi.org/10.1007/978-3-030-17653-2_3).
- [KPP⁺23a] Mirosław Kutyłowski, Giuseppe Persiano, Duong Hieu Phan, Moti Yung, and Marcin Zawada. Anamorphic signatures: Secrecy from a dictator who only permits authentication! In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part II*, volume 14082 of *Lecture Notes in Computer Science*, pages 759–790. Springer, Cham, August 2023. doi:[10.1007/978-3-031-38545-2_25](https://doi.org/10.1007/978-3-031-38545-2_25).

- [KPP⁺23b] Mirosław Kutyłowski, Giuseppe Persiano, Duong Hieu Phan, Moti Yung, and Marcin Zawada. The self-anti-censorship nature of encryption: on the prevalence of anamorphic cryptography. *Proceedings on Privacy Enhancing Technologies*, 2023(4):170–183, 2023.
- [KW19] Venkata Koppula and Brent Waters. Realizing chosen ciphertext security generically in attribute-based encryption and predicate encryption. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part II*, volume 11693 of *Lecture Notes in Computer Science*, pages 671–700. Springer, Cham, August 2019. doi:[10.1007/978-3-030-26951-7_23](https://doi.org/10.1007/978-3-030-26951-7_23).
- [KYH⁺06] Takashi Kitagawa, Peng Yang, Goichiro Hanaoka, Rui Zhang, Hajime Watanabe, Kanta Matsuura, and Hideki Imai. Generic transforms to acquire cca-security for identity based encryption: The cases of fopkc and REACT. In Lynn Margaret Batten and Reihaneh Safavi-Naini, editors, *Information Security and Privacy, 11th Australasian Conference, ACISP 2006, Melbourne, Australia, July 3-5, 2006, Proceedings*, volume 4058 of *Lecture Notes in Computer Science*, pages 348–359. Springer, 2006. doi:[10.1007/11780656_29](https://doi.org/10.1007/11780656_29).
- [LKWL24] Tobias Looker, Vasilis Kalos, Andrew Whitehead, and Mike Lodder. The BBS Signature Scheme. Internet-Draft draft-irtf-cfrg-bbs-signatures-07, Internet Engineering Task Force, September 2024. URL <https://datatracker.ietf.org/doc/draft-irtf-cfrg-bbs-signatures/07/>. Work in Progress.
- [NY90] Moni Naor and Moti Yung. Public-key cryptosystems provably secure against chosen ciphertext attacks. In *22nd Annual ACM Symposium on Theory of Computing*, pages 427–437. ACM Press, May 1990. doi:[10.1145/100216.100273](https://doi.org/10.1145/100216.100273).
- [PPY22] Giuseppe Persiano, Duong Hieu Phan, and Moti Yung. Anamorphic encryption: Private communication against a dictator. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part II*, volume 13276 of *Lecture Notes in Computer Science*, pages 34–63. Springer, Cham, May / June 2022. doi:[10.1007/978-3-031-07085-3_2](https://doi.org/10.1007/978-3-031-07085-3_2).
- [PPY24] Giuseppe Persiano, Duong Hieu Phan, and Moti Yung. Public-key anamorphism in (CCA-secure) public-key encryption and beyond. Cryptology ePrint Archive, Paper 2024/1327, 2024. doi:[10.1007/978-3-031-68379-4_13](https://doi.org/10.1007/978-3-031-68379-4_13).
- [Reg09] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM (JACM)*, 56(6):1–40, 2009.
- [SAB⁺22] Peter Schwabe, Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Gregor Seiler, Damien Stehlé, and Jintai Ding. CRYSTALS-KYBER. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
- [SOK01] Ryuichi Sakai, Shigeo Ohgishi, and Masao Kasahara. Cryptosystems based on pairing over elliptic curve (in japanese). In Nigel P. Smart, editor, *The 2001 Symposium on Cryptography and Information Security*, Lecture Notes in Computer Science, 2001.

- [Tib14] Mehdi Tibouchi. Elligator squared: Uniform points on elliptic curves of prime order as uniform random strings. In Nicolas Christin and Reihaneh Safavi-Naini, editors, *FC 2014: 18th International Conference on Financial Cryptography and Data Security*, volume 8437 of *Lecture Notes in Computer Science*, pages 139–156. Springer, Berlin, Heidelberg, March 2014. doi:[10.1007/978-3-662-45472-5_10](https://doi.org/10.1007/978-3-662-45472-5_10).
- [TZ23] Stefano Tessaro and Chenzhi Zhu. Revisiting BBS signatures. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part V*, volume 14008 of *Lecture Notes in Computer Science*, pages 691–721. Springer, Cham, April 2023. doi:[10.1007/978-3-031-30589-4_24](https://doi.org/10.1007/978-3-031-30589-4_24).
- [Wat05] Brent R. Waters. Efficient identity-based encryption without random oracles. In Ronald Cramer, editor, *Advances in Cryptology – EUROCRYPT 2005*, volume 3494 of *Lecture Notes in Computer Science*, pages 114–127. Springer, Berlin, Heidelberg, May 2005. doi:[10.1007/11426639_7](https://doi.org/10.1007/11426639_7).
- [WCHY23] Yi Wang, Rongmao Chen, Xinyi Huang, and Moti Yung. Sender-anamorphic encryption reformulated: Achieving robust and generic constructions. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 135–167. Springer, 2023.
- [WHL25] Weihao Wang, Shuai Han, and Shengli Liu. Anamorphic authenticated key exchange: Double key distribution under surveillance. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 168–200. Springer, 2025.
- [YKH⁺06] Peng Yang, Takashi Kitagawa, Goichiro Hanaoka, Rui Zhang, Kanta Matsuura, and Hideki Imai. Applying fujisaki-okamoto to identity-based encryption. In Marc P. C. Fossorier, Hideki Imai, Shu Lin, and Alain Poli, editors, *Applied Algebra, Algebraic Algorithms and Error-Correcting Codes, 16th International Symposium, AAECC-16, Las Vegas, NV, USA, February 20-24, 2006, Proceedings*, volume 3857 of *Lecture Notes in Computer Science*, pages 183–192. Springer, 2006. doi:[10.1007/11617983_18](https://doi.org/10.1007/11617983_18).

A Robustness of Public-Key Anamorphic Encryptions

A PKAE could also satisfy the following robustness property.

Robustness: For every PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$, such that for all $\lambda \in \mathbb{N}$, the following holds:

$$|\Pr[\text{Robust}_{\text{pkae}}^{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\text{Robust}_{\text{pkae}}^{\mathcal{A}}(\lambda, 1) = 1]| \leq \text{negl}(\lambda)$$

where the security games are defined as follows:

$\text{Robust}_{\text{pkae}}^{\mathcal{A}}(\lambda, \beta) :$

1. $(\text{apk}, \text{ask}, \text{dk}, \text{tk}) \leftarrow \text{aGen}(1^\lambda)$
2. **return** $\mathcal{A}^{\mathcal{O}^\beta(\text{apk}, \text{ask}, \text{tk}, \text{dk}, \cdot)}(\text{apk}, \text{ask})$ where

$$\mathcal{O}^0(\text{apk}, \text{ask}, \text{tk}, \text{dk}, \text{msg}) = \text{aDec}(\text{tk}, \text{Enc}(\text{apk}, \text{msg}))$$

$$\mathcal{O}^1(\text{apk}, \text{ask}, \text{tk}, \text{dk}, \text{msg}) = \perp$$

Robustness of Π_{aEl} from Section 5.1. To obtain robustness in the ElGamal anamorphic construction, we simply allow the decryption algorithm aEl.aDec to exhaustively search the anamorphic message space, and return $\perp$ if $\text{amsg} \notin \widehat{\mathcal{M}}$. To see why this technique is effective, observe that algorithm 2El.Enc samples k_2 uniformly at random from $\mathbb{Z}_q$, which for a fixed α and a fixed amsg , collides with $\kappa\alpha + \text{amsg}$ with probability $\frac{1}{q}$ (a negligible function in λ).

Robustness of Π_{aDReg} from Section 5.2. To obtain robustness in the Dual-Regev public key anamorphic 2-message extension, we could simply redefine the anamorphic decryption algorithm aDReg.aDec to include an additional sanity-check for the ciphertext $F(\text{act}) = (\widehat{\mathbf{r}}^T, \widehat{c})$ as follows:

1. **if** $|\widehat{c} - \widehat{\mathbf{r}}^T \cdot \widehat{\mathbf{k}}| \leq \frac{7q}{10}$ **then**
2. **return** $\text{amsg} := \text{aDReg.aDec}(\text{tk}, F(\text{act}))$
3. **else return** $\perp$

To see why this technique is effective, observe that

$$|\widehat{c} - \widehat{\mathbf{r}}^T \cdot \widehat{\mathbf{k}}| = \mathbf{s}_1^T \cdot \mathbf{A} \cdot \widehat{\mathbf{k}} + \widehat{y} + \text{amsg} \cdot \lfloor \frac{q}{2} \rfloor - \mathbf{s}_1^T \cdot \mathbf{A} \cdot \widehat{\mathbf{k}} - \mathbf{e}_2^T \cdot \widehat{\mathbf{k}} \pmod{q}$$

(the anamorphic decryption algorithm knows $\text{tk} = \widehat{\mathbf{k}}$). If the ciphertext contains amsg , then $\mathbf{s}_1^T \cdot \mathbf{A} \cdot \widehat{\mathbf{k}}$ cancels out, and the above term reduces to $t = \widehat{y} + \text{amsg} \cdot \lfloor \frac{q}{2} \rfloor - \mathbf{e}_2^T \cdot \widehat{\mathbf{k}} \pmod{q}$. Now, following our parameter choices $m \geq 2n \log q$, $r = \omega(\sqrt{\log m})$, $q \geq 5r(m+1)$, $\alpha \leq 1/(r\sqrt{m+1} \cdot \omega(\sqrt{\log n}))$ and $\chi = \Psi_\alpha$, the norm of the error $\widehat{y} - \mathbf{e}_2^T \cdot \widehat{\mathbf{k}} \pmod{q}$ is smaller than $\frac{q}{5}$ with overwhelming probability in λ . This implies that t is upper-bounded by $\frac{7q}{10}$.

When the ciphertext does not contain amsg , then $\widehat{c}$ is a uniformly random element of $\mathbb{Z}_q$, and hence the term $\mathbf{s}_1^T \cdot \mathbf{A} \cdot \widehat{\mathbf{k}}$ does not cancel out with high probability over the choices of $\mathbf{A}$ and $\mathbf{s}$, which implies that the norm of $\widehat{c} - \widehat{\mathbf{r}}^T \cdot \widehat{\mathbf{k}}$ is larger than $\frac{7q}{10}$.

B Informal: Robustness of Public-Key Anamorphic Signatures

For a PKASE or more precisely for anamorphism-friendly signatures with compliant encryption we could define robustness analogous to [BGH⁺24]. This captures that it must be hard to find μ messages $(\text{msg}_1, \dots, \text{msg}_\mu)$ for which when generating a vector of normal signatures viewed as an anamorphic signature $\text{asig} = (\text{SS.Sig}(\text{ask}, \text{msg}_1), \dots, \text{SS.Sig}(\text{ask}, \text{msg}_\mu))$, which is then subsequently anamorphically decrypted into $\text{amsg} := \text{aDec}(\text{tk}, \text{asig})$, it holds that $\text{amsg} \neq \perp$. Formally, as done for an anamorphic extension of a PKE in [BGH⁺24], this can be done by defining a distinguishing problem between an oracle implementing the aforementioned and an oracle that always simply returns $\perp$.

Unfortunately, it is not clear how robustness could generically be related to the IND\$-CPA security of the compliant PKE scheme used to construct the PKASE according to Definition 21. When looking at our concrete constructions, however, we can observe the following.

It is easy to see that the 3-PKASE Π_{aWaters} in Figure 6 uses an ElGamal type encryption (namely linear encryption) and thus we can follow the ElGamal public key anamorphic 2-message extension Π_{aEl} in Figure 1. Namely, we could use linear encryption in the exponent, i.e., instead of encrypting $\text{amsg} \in \mathbb{G}$ we encrypt g^{amsg} for amsg coming from a polynomially bounded subset of $\mathbb{Z}_p$, and follow the exact same idea.

For the 4-PKASE Π_{aBBS} in 7, although using ElGamal encryption and thus suggesting the use of the same strategy as above, it is not obvious how to achieve this. More precisely, when we consider a vector of μ normal signatures, then elements s_i, s_j , $i \neq j$ from any such two signatures when decoded, need to yield a random element in $\mathbb{G}_1$. However, this depends on the concrete encoding and is something that would need to be satisfied by the Elligator Squared encoding in our specific case or in general by the algorithm Decode of the pseudo-random $\mathbb{Z}_p^k$ -encoding.

We leave a thorough study of robustness in context of PKASE and anamorphism-friendly signatures with compliant encryption for future work.